ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 10
АДМИНИСТРАТИВНАЯ ПАНЕЛЬ DJANGO И РАБОТА С ДАННЫМИ ЧЕРЕЗ ORM

ЦЕЛЬ РАБОТЫ

Цель практической работы – освоить возможности административной панели Django и углубить навыки работы с данными через Django ORM на примере индивидуального сквозного проекта Roomly – системы бронирования помещений.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ 2
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 3
  2.1. Подготовка к работе 3
  2.2. Выполнение материалов Яндекс Практикума 3
  2.3. Работа над индивидуальным проектом 4
  2.4. Контрольная точка Git 5
  2.5. Требования к выполнению и оформлению 6
3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 7
  3.1. Административная панель Django 7
  3.2. Регистрация моделей и настройка ModelAdmin 10
  3.3. Локализация административного интерфейса 12
  3.4. Связанные объекты и встроенные формы 13
4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА 15
5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ 16
  5.1. Общий порядок выполнения работы 16
  5.2. Работа с административной панелью 17
  5.3. Работа с данными через Django ORM 66
  5.4. Дополнительные возможности Django в индивидуальном проекте 128
6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 166
  6.1. Проверка административной панели Django 167
  6.2. Проверка работы Django ORM 167
  6.3. Проверка пользовательских сценариев 169
  6.4. Проверка качества кода и автоматизированных тестов 170
7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ 173
  7.1. Контроль выполнения материалов Яндекс Практикума 173

7.2. Чек-лист выполнения практической работы 174
7.3. Таблица оценивания практической работы 175
7.4. Состав отчёта 179
8. КОНТРОЛЬНЫЕ ВОПРОСЫ 180
9. ИТОГ 181

1. ЗАДАЧИ РАБОТЫ

Цель практической работы – освоить возможности административной панели Django и углубить навыки работы с данными через Django ORM на примере индивидуального сквозного проекта Roomly – системы бронирования помещений.

В рамках работы необходимо решить следующие задачи:

1. Изучить принципы работы административной панели Django и порядок предоставления доступа к ней.
2. Освоить регистрацию моделей в Django Admin и настройку их отображения с помощью класса ModelAdmin.
3. Изучить способы локализации административного интерфейса, настройки названий приложений, моделей и полей.
4. Освоить настройку поиска, фильтрации, сортировки и отображения связанных объектов в административной панели.
5. Закрепить навыки построения ORM-запросов с использованием методов all(), filter(), exclude(), order_by(), get() и других средств Django ORM.
6. Научиться формировать составные условия запросов с помощью объектов Q.
7. Изучить получение данных через связанные модели, применение select_related() и использование полей связанных моделей в фильтрации и сортировке.
8. Освоить анализ SQL-запросов, сформированных Django ORM, с помощью Django Debug Toolbar.
9. Проверить корректность и производительность существующих ORM-представлений индивидуального проекта.
10. Сохранить ранее реализованную в ПР9 функциональность Roomly, устранить обнаруженные недостатки и актуализировать документацию проекта.

В результате выполнения практической работы необходимо получить работоспособный Django-проект Roomly с настроенной административной панелью, проверенными ORM-запросами и актуальной документацией.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

2.1. Подготовка к работе

Перед началом практической работы необходимо подготовить рабочую среду и проверить состояние индивидуального проекта.

Выполните следующие действия.

1. Откройте проект Roomly, созданный и доработанный в рамках ПР9. Не создавайте новый проект Django.
2. Откройте терминал в корневой директории проекта и активируйте виртуальное окружение, используемое для разработки Roomly.
3. Проверьте установленную версию Python и Django командами:

```
python --version
python -m django --version
```

4. Выполните проверку конфигурации проекта:

```
python manage.py check
```

5. Проверьте состояние миграций:

```
python manage.py showmigrations
```

6. Проверьте, что проект использует целевую базу данных db.sqlite3, а не исходную базу data/roomly.sqlite, применявшуюся до перехода на Django ORM.
7. Запустите сервер разработки и проверьте доступность главной страницы, списка помещений, страницы отдельного помещения, списка бронирований и страницы отдельного бронирования.
8. Убедитесь, что ранее реализованные фильтрация, сортировка и пагинация списка бронирований продолжают работать.
9. Сохраните резервную копию актуальной базы данных перед выполнением операций, которые могут изменить данные.
10. Проверьте состояние репозитория Git и зафиксируйте уже выполненные изменения, если они ещё не сохранены в коммите.

Если при выполнении подготовительных действий обнаружены ошибки, сначала устраните их. Не переходите к настройке административной панели, пока проект не запускается и его исходная функциональность не проверена.

2.2. Выполнение материалов Яндекс Практикума

Практическая работа включает материалы курса Яндекс Практикума «Бэкенд-разработка», относящиеся к трём тематическим блокам:

- «Админ-зона Django»;
- «Django ORM. Получение информации из БД»;

- «Финальный проект спринта: публикации для Блогикум».

Изучайте уроки в указанной последовательности. Выполняйте предложенные задания и проверяйте результаты на учебном примере курса.

При переносе полученных знаний в Roomly учитывайте различие между учебным проектом Яндекс Практикума и индивидуальным проектом. Не копируйте модели, поля, маршруты и бизнес-правила учебного приложения без анализа их назначения.

В частности, в Roomly уже реализованы модели Room, User и Booking, связи между ними, миграции и основные операции с данными через ORM. Эти элементы не требуется создавать заново. Их необходимо проверить и при необходимости доработать в соответствии с задачами ПР10.

В ходе изучения материалов курса:

1. Создайте суперпользователя Django, если он ещё не создан.
2. Зарегистрируйте модели Roomly в административной панели.
3. Настройте отображение объектов, поиск и фильтрацию.
4. Изучите способы формирования запросов и получения отдельных объектов.
5. Разберите построение составных условий с помощью Q.
6. Проверьте способы обращения к связанным моделям.
7. Изучите работу Django Debug Toolbar и анализ SQL-запросов.
8. Выполните предусмотренные курсом задания по доработке проекта.
9. Зафиксируйте основные результаты и возникшие затруднения.

Подробный порядок выполнения уроков приведён в разделе 4 методички.

2.3. Работа над индивидуальным проектом

Индивидуальный сквозной проект – система бронирования помещений Roomly. На этом этапе необходимо развивать существующую реализацию, не нарушая архитектуру, сформированную в предыдущих практических работах.

В рамках ПР10 выполните следующие работы:

1. Настройте административную панель для моделей Room, User и Booking.
2. Обеспечьте понятное русскоязычное отображение приложений, моделей, полей и объектов.
3. Настройте столбцы административных списков, поиск, фильтрацию и сортировку.
4. Организуйте просмотр связанных бронирований в карточках помещений и пользователей.
5. Проверьте существующие ORM-запросы в представлениях помещений и бронирований.

6. Проверьте фильтрацию по дате, помещению и пользователю, а также сортировку и пагинацию списка бронирований.
7. Проанализируйте количество и содержание SQL-запросов, выполняемых при обработке пользовательских сценариев.
8. При необходимости оптимизируйте обращения к связанным объектам и устраните избыточные запросы.
9. Выполните автоматизированные тесты и ручную проверку основных сценариев.
10. Актуализируйте README.md с учётом фактической реализации проекта.

Все изменения должны соответствовать предметной области Roomly. Например, отменённое бронирование не должно блокировать помещение на выбранную дату, а активное бронирование должно учитываться при определении доступности помещения.

Не считайте настройку административной панели заменой бизнес-логики приложения. Возможность создавать и редактировать бронирования через Django Admin не гарантирует соблюдение всех правил предметной области. Если проверка доступности помещения при создании бронирования не реализована, не утверждайте, что административная панель автоматически предотвращает конфликты.

2.4. Контрольная точка Git

В ходе выполнения практической работы необходимо сохранять промежуточные результаты в системе контроля версий Git.

Перед началом изменений проверьте состояние репозитория:

```
git status
```

После завершения настройки административной панели и проверки ORM-запросов просмотрите изменённые файлы:

```
git diff
```

Убедитесь, что в коммит не попали временные файлы, локальные настройки, секретные данные, виртуальное окружение или резервные копии базы данных.

Зафиксируйте завершённый этап работы:

```
git add .
git commit -m "Настроена админ-панель и проверены ORM-запросы"
```

Перед выполнением git add . проверьте содержимое git status и при необходимости добавьте только те файлы, которые относятся к работе.

Если в проекте используется удалённый репозиторий, отправьте коммит в соответствующую ветку после проверки его содержимого.

В ходе работы рекомендуется выделить как минимум две контрольные точки:

- после завершения настройки Django Admin;
- после проверки ORM-запросов, анализа SQL и актуализации документации.

Такой подход позволит восстановить предыдущее состояние проекта при возникновении ошибок.

2.5. Требования к выполнению и оформлению

При выполнении практической работы соблюдайте следующие требования.

Требования к коду:

- Сохраняйте существующую структуру Django-проекта.
- Используйте Django ORM для обращения к данным приложения.
- Не заменяйте ORM-запросы прямыми SQL-запросами без обоснованной необходимости.
- Соблюдайте единый стиль именования классов, функций, переменных и атрибутов.
- Не дублируйте регистрацию моделей, представления, маршруты и логику, которые уже реализованы.
- Не удаляйте существующую функциональность без обоснования и проверки последствий.
- При изменении моделей проверяйте необходимость создания и применения миграций.
- При оптимизации запросов подтверждайте результат проверкой фактического поведения приложения и SQL-запросов.

Требования к функциональности:

- Административная панель должна открываться по адресу /admin/.
- Модели Room, User и Booking должны быть доступны для администрирования.
- Поиск и фильтрация должны работать по предусмотренным полям.
- Связанные помещения и пользователи должны корректно отображаться в бронированиях.
- Фильтрация доступности помещений должна учитывать только активные бронирования на выбранную дату.
- Сортировка и пагинация списка бронирований должны сохранять выбранные параметры фильтрации.
- Существующие маршруты и шаблоны должны продолжать работать после внесения изменений.

Требования к документации и отчёту:

- Укажите цель и задачи практической работы.
- Кратко опишите выполненные изменения.

- Приведите результаты проверки административной панели.
- Отразите результаты проверки ORM-запросов и анализа SQL.
- Приведите подтверждение успешного выполнения автоматизированных тестов.
- Укажите коммиты, которыми зафиксированы результаты работы.
- При наличии ошибок или ограничений опишите их и укажите принятые решения.

Скриншоты используйте для подтверждения работы интерфейса и выполнения пользовательских сценариев. Они не заменяют исходный код, результаты тестирования и объяснение принятых технических решений.

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1. Административная панель Django

Django Admin – встроенная административная панель Django, предназначенная для просмотра, создания, изменения и удаления записей базы данных через веб-интерфейс.

Административная панель позволяет управлять данными без разработки отдельных HTML-страниц для каждой операции редактирования. Она автоматически формирует формы и списки объектов на основе зарегистрированных моделей и их полей.

В типичном Django-проекте административная панель используется для:

- просмотра записей базы данных;
- добавления новых объектов;
- редактирования существующих объектов;
- удаления объектов с учётом ограничений базы данных;
- поиска и фильтрации записей;
- перехода между связанными объектами;
- выполнения административных операций пользователями с соответствующими правами.

Схема обработки запроса к административной панели показана на рис. 1.

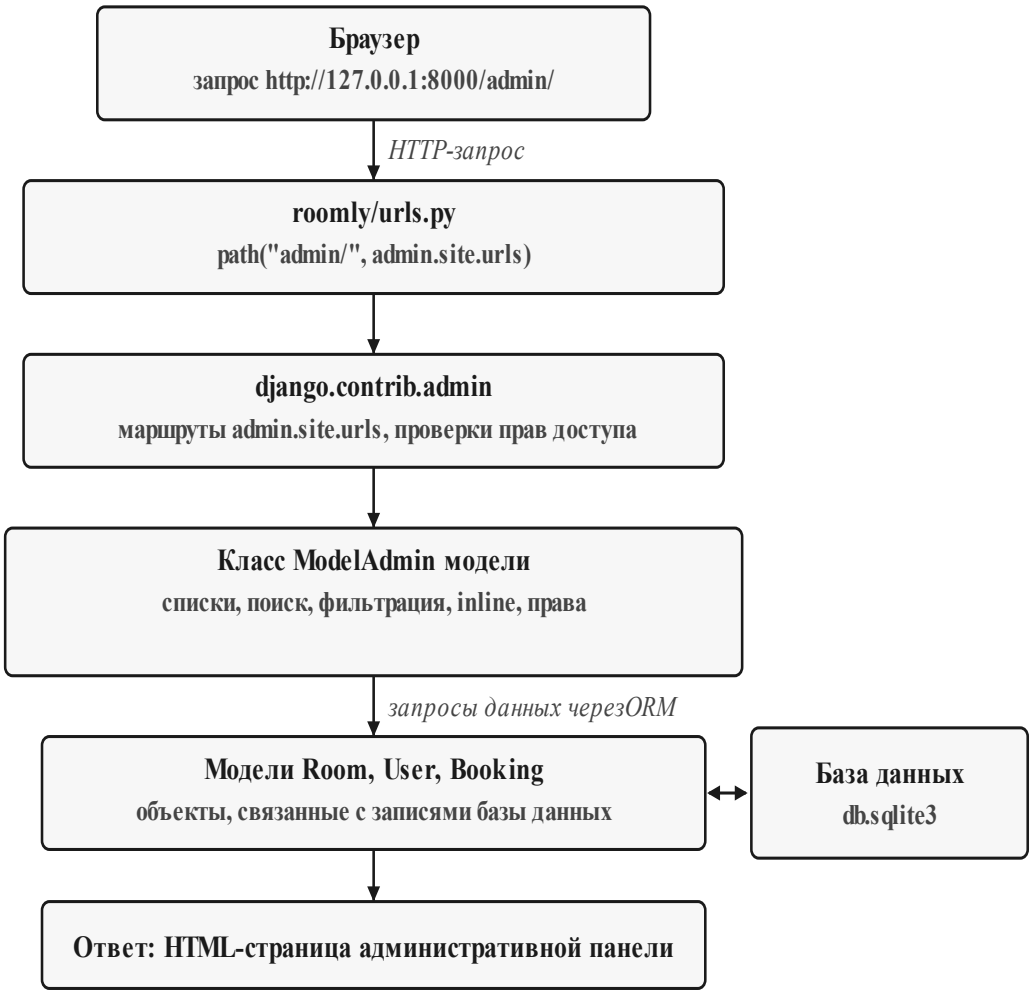


Рисунок 1 – Обработка запроса к административной панели Django

3.1.1. Подключение административной панели

В проекте Roomly административная панель подключается через приложение `django.contrib.admin`, включённое в список `INSTALLED_APPS`.

Маршрут административной панели задаётся в корневом файле roomly/urls.py:

```
from django.contrib import admin
from django.urls import include, path
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("homepage.urls")),
    path("rooms/", include("rooms.urls")),
    path("bookings/", include("bookings.urls")),
]
```

Строка `path("admin/", admin.site.urls)` связывает адрес `/admin/` со встроенным административным интерфейсом Django.

Для работы панели также необходимы стандартные компоненты Django, включая систему аутентификации, сессии и механизм сообщений. В типовом проекте они подключены в `INSTALLED_APPS` и `MIDDLEWARE`.

3.1.2. Аутентификация и права доступа

Для входа в административную панель используется учётная запись Django. Она может быть создана командой `createsuperuser`.

Суперпользователь обладает административными правами, позволяющими управлять зарегистрированными моделями. Для других пользователей доступ можно ограничивать с помощью разрешений Django.

Следует различать три понятия:

- *Аутентификация* – установление личности пользователя, например по имени и паролю.
- *Авторизация* – проверка того, разрешено ли пользователю выполнить конкретную операцию.
- *Пользователь предметной области* – объект, описывающий участника прикладной системы.

В Roomly модель users.User представляет пользователя системы бронирования и хранит его имя и электронную почту. Она не является учётной записью Django и сама по себе не предоставляет права входа в административную панель.

3.1.3. Регистрация моделей

Чтобы модель появилась в административной панели, её необходимо зарегистрировать в файле admin.py соответствующего приложения.

Например:

```
from django.contrib import admin
from .models import Room
admin.site.register(Room)
```

После регистрации Django предоставляет стандартный интерфейс для работы с объектами модели.

Если требуется настроить столбцы, поиск, фильтрацию и другие параметры, используют класс, наследующийся от `admin.ModelAdmin`.

```
from django.contrib import admin
from .models import Room
@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
    search_fields = ("name",)
```

Декоратор `@admin.register(Room)` регистрирует модель `Room` с указанным классом настроек.

Регистрацию одной модели нельзя без необходимости выполнять несколько раз. Если модель уже зарегистрирована, её настройки следует изменить в существующей регистрации.

3.2. Регистрация моделей и настройка ModelAdmin

Класс ModelAdmin позволяет управлять тем, как объекты модели отображаются и редактируются в административной панели.

Настройки можно условно разделить на параметры списка объектов, поиска, фильтрации, сортировки и редактирования.

Основные группы параметров класса ModelAdmin показаны на рис. 2.

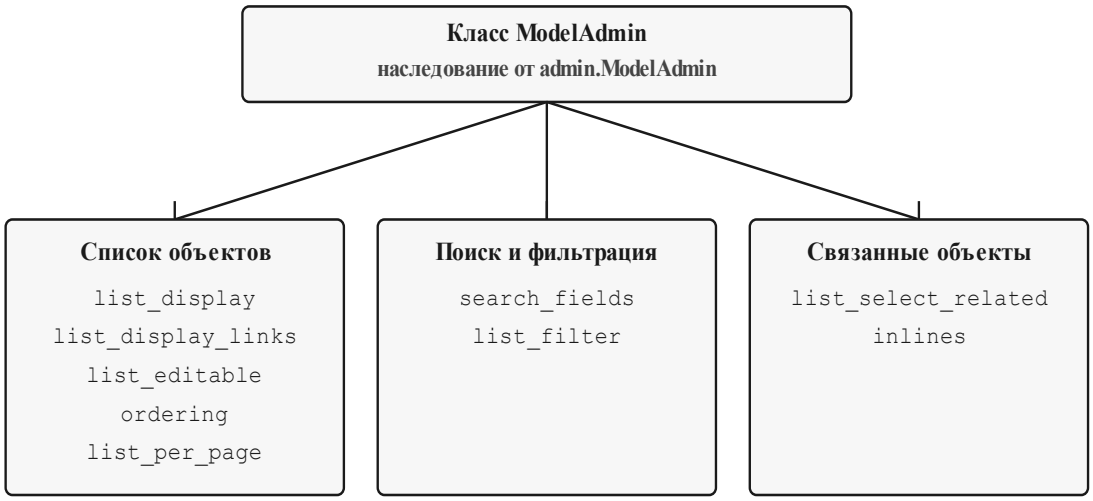


Рисунок 2 – Основные параметры класса ModelAdmin

3.2.1. Отображение списка объектов

Параметр `list_display` задаёт поля и методы, которые отображаются в таблице объектов.

Например, для модели Room:

```
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
```

В списке помещений будут отображаться идентификатор, название и вместимость.

Параметр `list_display_links` задаёт поля, по нажатию на которые можно перейти к карточке объекта.

```
list_display_links = ("id", "name")
```

Если параметр не задан, Django обычно использует первое поле `list_display` как ссылку на страницу изменения объекта.

3.2.2. Поиск и фильтрация

Параметр `search_fields` определяет поля, по которым выполняется поиск:

```
search_fields = ("name",)
```

Параметр `list_filter` добавляет боковую панель фильтрации:

```
list_filter = ("capacity",)
```

Поиск и фильтрация решают разные задачи. Поиск позволяет найти записи по введённой строке, а фильтрация – выбрать объекты, удовлетворяющие заданным значениям или условиям.

Для связанных полей можно использовать двойное подчёркивание:

```
search_fields = (
    "room__name",
    "user__name",
)
```

Такой поиск может использоваться в административном списке бронирований, чтобы находить записи по названию помещения и имени пользователя.

3.2.3. Сортировка и количество записей

Параметр `ordering` задаёт порядок отображения объектов:

```
ordering = ("name",)
```

Для сортировки по убыванию перед именем поля ставится дефис:

```
ordering = ("-booking_date",)
```

Можно задавать сортировку по нескольким полям:

```
ordering = ("booking_date", "id")
```

В этом случае записи сортируются сначала по дате, а затем по идентификатору внутри одинаковых дат.

Параметр `list_per_page` задаёт количество записей, отображаемых на одной странице административного списка:

```
list_per_page = 20
```

Он относится к списку объектов в Django Admin и не заменяет пагинацию, реализованную в пользовательских представлениях приложения.

3.2.4. Редактирование полей непосредственно в списке

Параметр `list_editable` позволяет изменять выбранные поля прямо в таблице объектов:

```
list_display = ("id", "name", "capacity")
list_editable = ("capacity",)
```

В таком случае вместимость можно редактировать без открытия карточки помещения.

При использовании `list_editable` необходимо соблюдать ограничения Django: редактируемое поле должно присутствовать в `list_display` и не может одновременно быть полем ссылки на карточку объекта.

Используйте редактирование из списка только там, где оно действительно удобно и безопасно. Например, изменение вместимости помещения может быть допустимо, но не должно приводить к нарушению других правил предметной области.

3.3. Локализация административного интерфейса

Локализация – адаптация интерфейса приложения к языку и региональным настройкам пользователя.

В Django локализация может включать:

- язык стандартного интерфейса;
- отображаемые названия приложений и моделей;
- подписи полей;
- представление дат и чисел;
- перевод сообщений и системных строк.

3.3.1. Язык и часовой пояс

В settings.py языка и часовой пояс задаются соответствующими параметрами:

```
LANGUAGE_CODE = "ru-RU"
TIME_ZONE = "Europe/Moscow"
USE_TZ = True
```

- `LANGUAGE_CODE` определяет язык по умолчанию для стандартных переводимых строк Django.
- `TIME_ZONE` задаёт часовой пояс проекта.
- `USE_TZ = True` включает использование часовых поясов при работе Django с датой и временем.

Настройки языка и часового пояса не следует смешивать: язык влияет на локализацию интерфейса, а часовой пояс – на представление и обработку времени.

3.3.2. Названия приложений

Название приложения для административного интерфейса задаётся через `verbose_name` в классе конфигурации приложения, унаследованном от AppConfig.

Например:

```
from django.apps import AppConfig
class RoomsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "rooms"
    verbose_name = "Помещения"
```

Здесь `name` остаётся техническим именем Python-пакета, а `verbose_name` задаёт человекочитаемое название.

Конфигурацию приложения можно явно указать в `INSTALLED_APPS`:

```
"rooms.apps.RoomsConfig"
```

3.3.3. Названия моделей и полей

Для модели можно задать параметры `verbose_name` и `verbose_name_plural` в классе `Meta`:

```
class Meta:
    verbose_name = "Помещение"
    verbose_name_plural = "Помещения"
Для поля название задаётся при объявлении:
name = models.CharField(
    max_length=200,
    verbose_name="Название помещения",
)
```

Параметр `verbose_name` используется Django при построении форм и административных страниц.

Метод `__str__()` определяет строковое представление объекта:

```
def __str__(self):
    return self.name
```

Для Room это позволяет отображать название помещения вместо стандартного представления, содержащего имя класса и идентификатор объекта. В Roomly эти механизмы уже частично реализованы. При доработке необходимо проверять существующие настройки и дополнять их, а не создавать дублирующие определения.

3.4. Связанные объекты и встроенные формы

Реляционные базы данных позволяют связывать записи разных таблиц. Django ORM представляет такие связи через поля моделей, например `ForeignKey`.

В Roomly модель `Booking` содержит ссылки на помещение и пользователя:

```
room = models.ForeignKey(
    Room,
    on_delete=models.PROTECT,
    related_name="bookings",
    verbose_name="Помещение",
)
user = models.ForeignKey(
    User,
    on_delete=models.PROTECT,
    related_name="bookings",
    verbose_name="Пользователь", )
```

Один объект `Room` может быть связан со многими объектами `Booking`. Аналогично один объект `User` может иметь несколько бронирований.

Связи моделей Roomly и встроенные формы для просмотра бронирований показаны на рис. 3.

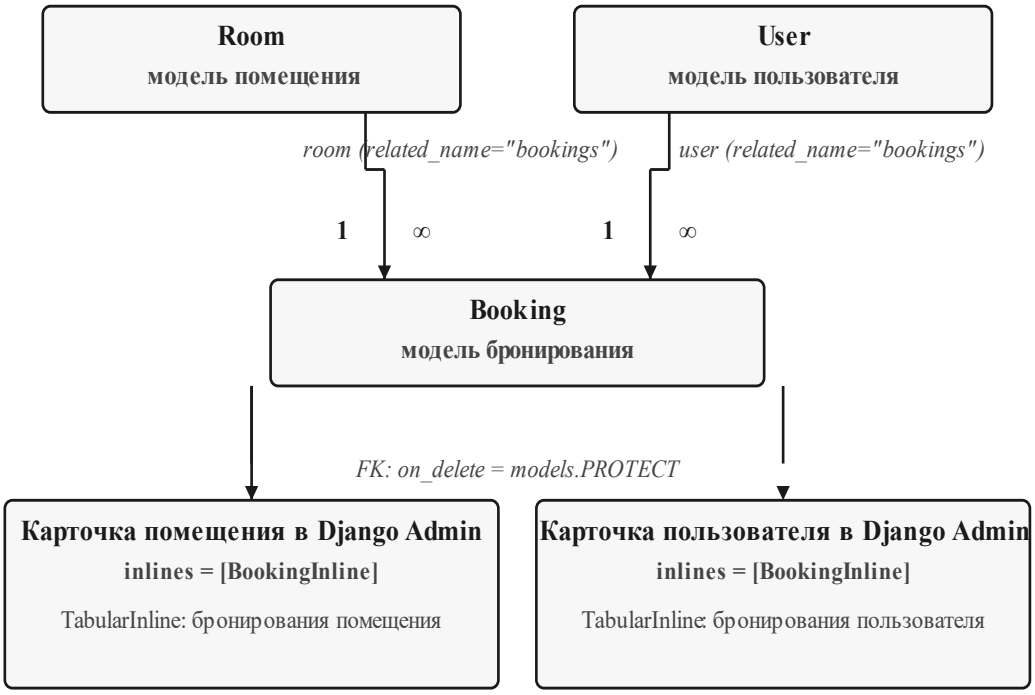


Рисунок 3 – Связи моделей Roomly и встроенные формы (inline)

3.4.1. Обращение к связанным объектам

Если объект бронирования уже получен из базы данных, связанное помещение можно получить через `booking.room`, а пользователя – через `booking.user`.

Например:

```
booking.room.name
booking.user.name
```

Обратное обращение обеспечивается параметром `related_name`:

```
room.bookings.all()
user.bookings.all()
```

Таким образом, можно получить все бронирования конкретного помещения или пользователя.

3.4.2. Встроенные формы в Django Admin

Для редактирования связанных объектов на одной странице Django предоставляет встроенные формы – `inline`.

Существует два основных варианта:

- `admin.TabularInline` – отображение связанных объектов в табличной форме;
- `admin.StackedInline` – отображение связанных объектов в вертикальной форме, когда каждый связанный объект выводится отдельным блоком полей.

Состав встроенных форм задаётся в классе `ModelAdmin` через параметр `inlines`. В индивидуальном проекте встроенные формы применяются для просмотра бронирований в карточках помещений и пользователей (подраздел 5.1.7).

4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

Практическая часть включает изучение трёх тематических блоков курса Яндекс Практикума «Бэкенд-разработка»:

- «Админ-зона Django»;
- «Django ORM. Получение информации из БД»;
- «Финальный проект спринта: публикации для Блогикум».

Выполняйте уроки последовательно, поскольку материалы по административной панели и ORM опираются на знания о моделях, связях между ними и работе с базой данных.

Для каждого урока придерживайтесь следующего порядка:

1. Изучите теоретические материалы и примеры кода.
2. Выполните задания в учебном проекте, если они предусмотрены уроком.
3. Проверьте результат и разберите возникшие ошибки.
4. Определите, какие приёмы применимы к проекту Roomly.
5. При необходимости выполните аналогичные действия в Roomly, сохраняя существующие модели, представления, шаблоны и маршруты.
6. Зафиксируйте результат в Git, если завершён самостоятельный этап работы.

Не переносите в Roomly учебные модели и бизнес-правила из примеров курса. В частности, модели публикаций, категорий и местоположений из «Блогикума» не должны заменять модели Room, User и Booking.

Для проверки результатов используйте команды:

```
python manage.py check
python manage.py showmigrations
pytest -v
```

После выполнения материалов Яндекс Практикума необходимо подтвердить освоение следующих навыков.

Направление	Ожидаемый результат
Django Admin	Умеете создавать суперпользователя и регистрировать модели
Настройка интерфейса	Умеете задавать столбцы, поиск, фильтрацию и сортировку
Локализация	Понимаете назначение LANGUAGE_CODE, verbose_name и AppConfig.verbose_name
Работа с QuerySet	Умеете получать данные через all(), filter() и exclude()
Логические условия	Умеете строить составные запросы с помощью Q

Направление	Ожидаемый результат
Сортировка и ограничения	Умеете использовать order_by() и понимать ограничения выборки
Получение отдельных объектов	Понимаете различия между get() и get_object_or_404()
Связанные модели	Умеете обращаться к связанным полям и применять select_related()
Анализ SQL	Умеете находить запросы, сформированные ORM, и анализировать их
Доработка проекта	Умеете проверять изменения и сохранять существующую функциональность

Результаты выполнения учебных заданий должны быть использованы при работе над индивидуальным проектом в разделе 5.

5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

5.1. Общий порядок выполнения работы

В рамках ПР10 необходимо продолжить разработку системы бронирования помещений Roomly. В предыдущей практической работе были созданы модели `Room`, `User` и `Booking`, настроены связи между ними, выполнены миграции и реализована работа с данными через Django ORM.

В этой работе необходимо закрепить навыки работы с Django ORM и административной панелью на примере приложения Roomly. Научиться получать и фильтровать данные, работать со связанными моделями, формировать страницы по слагу, определять абсолютный URL объекта, передавать общие данные в шаблоны и использовать вычисляемые поля базы данных. Отработать создание и применение миграций, а также проверку SQL-запросов с помощью Django Debug Toolbar.

В приложении Roomly должны быть реализованы и проверены:

- Управление помещениями, пользователями и бронированиями через административную панель.
- Получение, фильтрация и сортировка данных с помощью ORM.
- Страница помещения, открывающаяся по уникальному слагу.
- Метод `get_absolute_url()` для формирования адреса страницы помещения.
- Пользовательский контекстный процессор, передающий общие данные в шаблоны.
- Вычисляемое поле `GeneratedField`, значение которого формируется на стороне базы данных.
- Создание и применение миграций при изменении моделей.
- Проверка SQL-запросов и связанных объектов с помощью Django Debug Toolbar.

5.2. Работа с административной панелью

5.2.1. Подготовка административной панели к работе

5.2.1.1. Подготовка административной панели и создание суперпользователя

Административная панель Django – это встроенный интерфейс для управления данными приложения. Она позволяет просматривать записи, создавать новые объекты, изменять существующие и удалять их без разработки отдельных страниц для выполнения этих операций.

В проекте Roomly административная панель будет использоваться для управления помещениями, пользователями и бронированиями. Например, администратор сможет добавить новое помещение, изменить его вместимость, проверить сведения о бронировании или отметить бронирование как отменённое.

При этом административная панель не заменяет пользовательский интерфейс Roomly. Посетители сайта продолжают работать с обычными страницами приложения, а административная панель предназначена для пользователей, которым предоставлены соответствующие права.

В Django административная панель реализована приложением django.contrib.admin. Оно входит в состав Django, поэтому для базового использования не требуется устанавливать дополнительный пакет или самостоятельно создавать интерфейс управления данными.

Однако наличие административной панели в проекте ещё не означает, что она готова к использованию. Необходимо проверить настройки Django, применить миграции базы данных, создать административного пользователя и убедиться, что вход в систему работает.

5.2.1.2. Проверка настроек административной панели

Откройте файл settings.py проекта Roomly и найдите параметр `INSTALLED_APPS`.

В стандартном проекте Django этот список содержит следующие приложения:

```
INSTALLED_APPS = [
"django.contrib.admin",
"django.contrib.auth",
"django.contrib.contenttypes",
"django.contrib.sessions",
"django.contrib.messages",
"django.contrib.staticfiles",
"homepage",
"rooms",
"users",
"bookings",
]
```

Это пример структуры настроек. Если в проекте уже присутствуют дополнительные приложения, менять их состав только ради административной панели не требуется.

Рассмотрим назначение основных компонентов Django, связанных с административной панелью.

Приложение	Назначение
`django.contrib.admin`	Предоставляет административный интерфейс Django.
`django.contrib.auth`	Обеспечивает аутентификацию пользователей, работу с паролями, группами и разрешениями.
`django.contrib.contenttypes`	Позволяет Django учитывать типы моделей и связывать разрешения с конкретными моделями.
`django.contrib.sessions`	Обеспечивает хранение данных пользовательских сессий.
`django.contrib.messages`	Позволяет отображать пользователю временные уведомления, например сообщение об успешном сохранении объекта.
`django.contrib.staticfiles`	Обеспечивает работу со статическими файлами, в том числе с ресурсами оформления административной панели.

Эти приложения выполняют разные задачи, но вместе обеспечивают работу стандартной административной панели.

Например, django.contrib.admin формирует страницы управления объектами, а django.contrib.auth позволяет определить, кто может войти в систему и какие действия ему разрешены.

Не удаляйте стандартные приложения Django из `INSTALLED_APPS` без необходимости. Даже если они не используются непосредственно в пользовательских страницах Roomly, они могут быть необходимы для работы административной панели и других механизмов фреймворка.

5.2.1.3. Проверка маршрута административной панели

Откройте главный файл маршрутизации проекта roomly/urls.py. В нём должен быть подключён маршрут административной панели.

Пример стандартной конфигурации:

```
from django.contrib import admin
from django.urls import include, path
urlpatterns = [
path("admin/", admin.site.urls),
path("", include("homepage.urls")),
path("rooms/", include("rooms.urls")),
path("bookings/", include("bookings.urls")),
]
```

Приведённый фрагмент показывает принцип подключения маршрутов. При работе с проектом сохраните существующие маршруты и добавьте подключение административной панели, если его ещё нет. Не заменяйте весь файл приведённым примером, если в нём уже настроены другие страницы и пространства имён.

Разберём строку:

```
path("admin/", admin.site.urls),
```

- `path()` объявляет маршрут, по которому Django определяет, какой обработчик должен обслуживать запрос.
- `"admin/"` задаёт адрес административной панели относительно корневого адреса сайта.
- `admin.site.urls` подключает набор маршрутов, предоставляемых административным интерфейсом Django.

После запуска локального сервера панель будет доступна по адресу:

```
http://127.0.0.1:8000/admin/
```

Завершающий символ / соответствует стандартной конфигурации маршрута. Если административная панель не открывается, прежде всего проверьте, что адрес указан правильно и маршрут присутствует в главном urls.py.

5.2.1.4. Проверка базы данных и применение миграций

Перед созданием административного пользователя необходимо убедиться, что база данных подготовлена к работе.

Django хранит сведения о пользователях, группах, разрешениях и сессиях в таблицах базы данных. Эти таблицы создаются на основании миграций встроенных приложений.

Миграции – это механизм, с помощью которого Django описывает и применяет изменения структуры базы данных. Например, миграция может создавать новую таблицу, добавлять поле или изменять ограничения существующего поля.

Откройте терминал в каталоге проекта, где находится файл manage.py. Активируйте виртуальное окружение проекта, если оно используется, и выполните:

```
python manage.py migrate
```

Команда `migrate` применяет миграции, которые ещё не были применены к текущей базе данных.

При выполнении команды Django определяет, какие изменения необходимо внести, и создаёт соответствующие таблицы или изменяет их структуру. Если миграции уже применены, повторный запуск обычно не приводит к повторному созданию таблиц: Django учитывает состояние миграций.

В выводе команды могут присутствовать сообщения о применении миграций приложений `contenttypes`, `auth`, `admin` и `sessions`.

Если проект уже использует собственные модели Roomly и для них созданы миграции, Django также может применить миграции этих приложений. Не следует

удалять базу данных или пересоздавать её только ради настройки административной панели.

В частности, если проект уже содержит важные данные, сначала убедитесь, что команда использует нужную базу данных. Применение миграций к другой базе не подготовит ту базу, с которой фактически работает приложение.

После завершения команды проверьте, что в терминале нет ошибок.

5.2.1.5. Создание суперпользователя Django

Для входа в административную панель Django необходима учётная запись с соответствующими правами. На учебном проекте для этого обычно создают суперпользователя.

Суперпользователь – это учётная запись стандартной системы авторизации Django, обладающая полными правами в административной панели. Она не является объектом предметной области users.User.

Выполните команду создания учётной записи:

- В терминале, находясь в корневой папке Roomly, выполните:

```
python manage.py createsuperuser
```

Django последовательно запросит данные для создания учётной записи.

- Укажите имя пользователя. В терминале появится запрос вида:

```
Username:
```

- Введите имя, под которым будете входить в административную панель, например:

```
admin
```

Если Django сообщает, что такое имя уже используется, выберите другое или используйте уже созданную учётную запись.

- Укажите адрес электронной почты. Если Django запрашивает адрес электронной почты, введите адрес, который будет связан с создаваемой учётной записью.

Этот адрес относится к учётной записи стандартной системы авторизации Django, а не к модели users.User из Roomly.

- Задайте пароль. Введите пароль и повторите его по запросу.

При вводе пароля в терминале символы обычно не отображаются. Это нормальное поведение. Вводите пароль внимательно и нажимайте Enter после завершения.

- Django может предупредить, что пароль слишком простой или похож на имя пользователя. По возможности используйте надёжный пароль.

При успешном создании появится сообщение о том, что суперпользователь создан.

- Если суперпользователь уже существует повторно создавать учётную запись не нужно. Используйте существующие имя пользователя и пароль.

- Если пароль забыт, его можно изменить командой:

```
python manage.py changepassword admin
```

- Вместо admin укажите имя своей учётной записи.
- Если при создании суперпользователя возникает ошибка

Проверьте следующее:

- команда выполняется из папки, где находится manage.py;
- активировано виртуальное окружение проекта;
- Django подключён и запускается без ошибок;
- миграции стандартных приложений auth и admin применены;
- команда использует ту же базу данных, с которой работает сервер.

Не создавайте новый файл базы данных только ради устранения ошибки. Сначала проверьте параметр DATABASES в roomly/settings.py.

5.2.1.6. Что означает статус суперпользователя?

В системе аутентификации Django пользователь представлен объектом модели User, которая по умолчанию относится к приложению django.contrib.auth.

У такой учётной записи есть несколько важных характеристик:

- `is_active` – указывает, активна ли учётная запись;
- `is_staff` – определяет, может ли пользователь входить в административный интерфейс Django;
- `is_superuser` – определяет, обладает ли пользователь всеми разрешениями системы Django.

Команда `createsuperuser` создаёт учётную запись с правами суперпользователя. В стандартной конфигурации такой пользователь активен, имеет право входа в административную панель и обладает всеми разрешениями.

Важно различать два понятия.

- *Суперпользователь Django* – это учётная запись, предназначенная для управления приложением и его административными функциями.
- *Пользователь Roomly* – это объект предметной области, представляющий человека, который использует систему бронирования. В данном проекте для него предусмотрена собственная модель User приложения users.

Несмотря на одинаковое название, эти сущности решают разные задачи. Собственная модель `users.User` содержит данные пользователя Roomly, например имя и адрес электронной почты. Модель аутентификации Django отвечает за вход в систему, проверку пароля и управление правами доступа.

Поэтому создание суперпользователя не означает автоматического создания записи в таблице пользователей Roomly. Аналогично добавление записи в `users.User`

само по себе не создаёт учётную запись, с помощью которой человек сможет войти в административную панель.

Это различие особенно важно в проекте, где уже существует собственная модель `User`. Не следует создавать административную учётную запись вручную через эту модель: для входа в стандартную панель Django используется настроенная система аутентификации.

5.2.1.7. Запуск сервера и вход в административную панель

После создания суперпользователя запустите локальный сервер разработки:

```
python manage.py runserver
```

По умолчанию Django запускает сервер на адресе 127.0.0.1:8000.

Откройте в браузере:

```
http://127.0.0.1:8000/admin/
```

Должна отобразиться страница входа в административную панель Django.

Введите имя пользователя и пароль, указанные при выполнении команды `createsuperuser`. После успешной аутентификации Django откроет главную страницу административного интерфейса.

На этом этапе панель может отображать только стандартные модели Django, например группы и пользователей системы аутентификации. Собственные модели Roomly появятся в административном интерфейсе после их регистрации. Этому будет посвящён следующий пункт работы.

Не следует считать отсутствие моделей `Room`, `User` и `Booking` ошибкой на данном этапе. Django не добавляет пользовательские модели в административную панель автоматически.

5.2.1.8. Проверка результата

После выполнения предыдущих действий необходимо убедиться, что административная панель действительно готова к дальнейшей настройке.

Проверьте следующие условия:

1. Команда `python manage.py migrate` завершилась без ошибок.
2. Команда `python manage.py createsuperuser` завершилась сообщением об успешном создании пользователя.
3. Команда `python manage.py runserver` запустила сервер разработки.
4. Адрес `http://127.0.0.1:8000/admin/` открывает страницу входа.
5. Вход с созданной учётной записью выполняется успешно.
6. После входа отображается главная страница административной панели Django.

Если какое-либо условие не выполнено, сначала устраните проблему, а затем переходите к регистрации моделей Roomly.

5.2.1.9. Возможные ошибки и способы их устранения

Ошибка: *no such table* или сообщение об отсутствии таблицы.

Обычно это означает, что необходимая таблица отсутствует в используемой базе данных. Проверьте настройки DATABASES в settings.py и выполните python manage.py migrate. Убедитесь, что команда использует ту же базу данных, с которой работает приложение.

Ошибка: *That username is already taken*.

Пользователь с таким именем уже существует в выбранной базе данных. Можно указать другое имя или проверить существующую учётную запись. Не нужно удалять базу данных только ради повторного создания суперпользователя.

Ошибка: *You don't have permission to view or edit anything*.

Такое сообщение может появиться, если пользователь вошёл в административную панель, но не имеет разрешений на просмотр доступных моделей. Проверьте, действительно ли используется суперпользователь и не была ли изменена стандартная конфигурация аутентификации. Если в панели пока отсутствуют собственные модели Roomly, это само по себе не означает, что вход выполнен неправильно.

Ошибка *404 Not Found при открытии /admin/.*

Проверьте наличие строки path("admin/", admin.site.urls) в главном файле urls.py. Убедитесь, что вы открываете адрес именно того Django-проекта, сервер которого запущен в терминале.

Ошибка *ModuleNotFoundError: No module named 'django'*.

Вероятно, команда выполняется не в том Python-окружении, в котором установлен Django. Активируйте виртуальное окружение проекта и повторите команду.

Страница административной панели открывается без оформления.

Проверьте настройки django.contrib.staticfiles, параметр STATIC_URL и сообщения в консоли браузера. При локальной разработке Django обычно самостоятельно обслуживает статические файлы при стандартной конфигурации DEBUG=True.

После успешного выполнения всех проверок административная панель готова к настройке. Следующий шаг – регистрация моделей Room, User и Booking, чтобы управлять объектами предметной области непосредственно через интерфейс Django.

5.2.2. Регистрация модели Room и настройка списка помещений

Административная панель Django предоставляет стандартный интерфейс для управления объектами моделей. Однако для удобной работы с конкретной

предметной областью необходимо настроить отображение данных, поиск, фильтрацию, сортировку и навигацию по списку записей.

В проекте Roomly начнём с модели Room, которая представляет помещение. Администратор должен иметь возможность просматривать список помещений, видеть их вместимость, быстро находить нужные записи и переходить к редактированию выбранного объекта.

Для этого необходимо зарегистрировать модель в административной панели и настроить её представление с помощью класса ModelAdmin.

5.2.2.1. Проверка модели Room

Перед настройкой административного интерфейса необходимо убедиться, что модель Room уже создана и содержит поля, необходимые для работы с помещениями.

Откройте файл rooms/models.py. Если модель уже реализована, изучите её текущий код и сохраните все существующие поля, методы и настройки. Следующий пример показывает структуру модели, которую можно использовать в качестве ориентира:

```
from django.db import models
class Room(models.Model):
    name = models.CharField(
        max_length=100,
        verbose_name="Название",
    )
    capacity = models.PositiveIntegerField(
        verbose_name="Вместимость",
    )
    def __str__(self):
        return self.name
    class Meta:
        verbose_name = "Помещение"
        verbose_name_plural = "Помещения"
```

Рассмотрим основные элементы модели.

- `models.Model` – базовый класс Django, от которого наследуются модели, связанные с базой данных.
- `name` – поле, содержащее название помещения.
- `capacity` – поле, содержащее вместимость помещения.
- `__str__()` – метод, возвращающий человекочитаемое представление объекта.
- `Meta` – вложенный класс, предназначенный для настройки метаданных модели.

Параметры `verbose_name` и `verbose_name_plural` задают название модели в единственном и множественном числе соответственно. Благодаря этому в административной панели можно отображать понятное название раздела – «Помещения».

Параметр `verbose_name` у каждого поля определяет его отображаемое название. Например, вместо технического имени `capacity` в формах будет использоваться слово «Вместимость».

Метод `__str__()` возвращает название помещения. Это удобно, когда Django должен показать объект в текстовом виде, например при выборе связанного помещения в форме бронирования.

Если в модели уже определён класс `Meta`, не создавайте второй: добавьте необходимые параметры в существующий класс. Не заменяйте всю модель примером, если в ней уже реализованы дополнительные поля и методы.

5.2.2.2. Регистрация модели в административной панели

Django позволяет зарегистрировать модель в административной панели несколькими способами. Прежде чем настраивать список помещений, необходимо понять, чем они отличаются и в каких ситуациях применяются.

Способ 1. Регистрация без отдельного класса настройки

Для простой регистрации достаточно вызвать `admin.site.register()`.

В файле rooms/admin.py можно написать:

```
from django.contrib import admin
from .models import Room
admin.site.register(Room)
```

Рассмотрим этот код.

- `from django.contrib import admin` импортирует модуль административного интерфейса Django.
- `from .models import Room` импортирует модель `Room` из текущего приложения. Точка перед `models` означает, что модуль находится внутри пакета `rooms`.

Вызов:

```
admin.site.register(Room)
```

передаёт модель стандартному административному сайту Django. После этого Django может сформировать страницы просмотра, создания, изменения и удаления объектов модели в соответствии с настройками и разрешениями пользователя.

При такой регистрации используется стандартная конфигурация административного интерфейса. Отдельный класс настройки создавать не требуется.

Этот способ подходит, когда необходимо быстро подключить модель к панели или проверить базовую работу административного интерфейса.

Однако для проекта Roomly стандартного представления недостаточно: желательно явно указать, какие столбцы отображать, по каким полям искать помещения и как сортировать список.

Способ 2. Регистрация с помощью класса ModelAdmin

Для настройки административного интерфейса используется класс, унаследованный от `admin.ModelAdmin`.

Пример:

```
from django.contrib import admin
from .models import Room
@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
    search_fields = ("name",)
    list_filter = ("capacity",)
    ordering = ("name",)
```

В этом случае класс `RoomAdmin` содержит настройки административного интерфейса модели `Room`, а декоратор `@admin.register(Room)` регистрирует модель с использованием этих настроек.

`ModelAdmin` – базовый класс Django, предоставляющий стандартные возможности управления объектами. При наследовании от него можно переопределить отдельные параметры, не создавая собственные страницы редактирования и не реализуя механизмы работы с базой данных вручную.

В данном примере настраиваются столбцы списка, поиск, фильтрация и сортировка.

Альтернативный вариант регистрации класса

Ту же настройку можно выполнить без декоратора:

```
from django.contrib import admin
from .models import Room
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
    search_fields = ("name",)
    list_filter = ("capacity",)
    ordering = ("name",)
admin.site.register(Room, RoomAdmin)
```

В этом варианте класс сначала объявляется, а затем передаётся методу `admin.site.register()` вместе с моделью.

Оба способа регистрации с `ModelAdmin` дают один и тот же результат. Далее будем использовать декоратор `@admin.register(Room)`, поскольку он позволяет связать модель и класс настройки непосредственно в объявлении класса.

5.2.2.3. Настройка столбцов списка – list_display

Параметр `list_display` определяет, какие столбцы отображаются на странице списка объектов. Добавьте в класс `RoomAdmin`:

```
list_display = ("id", "name", "capacity")
```

В результате список помещений будет представлен таблицей.

ID	Название	Вместимость
1	Аудитория 301	30
2	Конференц-зал	60
3	Лаборатория 205	24

Значения приведены для примера. Фактический список зависит от записей в базе данных.

В данном случае отображаются три поля:

- `id` – идентификатор записи;
- `name` – название помещения;
- `capacity` – вместимость помещения.

Если `list_display` не задан, Django обычно использует строковое представление объекта, определяемое методом `__str__()`. Для модели `Room` это будет название помещения.

Однако одного названия может быть недостаточно. Например, если администратору нужно подобрать помещение для мероприятия на 40 человек, удобнее видеть вместимость сразу в таблице, не открывая каждую запись.

Поэтому `list_display` позволяет сделать список информативнее и уменьшить количество переходов между страницами.

В качестве элементов `list_display` можно использовать поля модели, некоторые методы модели, методы административного класса и другие поддерживаемые Django средства отображения. В рамках текущего задания достаточно полей `id`, `name` и `capacity`.

5.2.2.4. Настройка ссылок на редактирование – list_display_links

По умолчанию Django обычно делает первое поле списка ссылкой на страницу изменения объекта. Если первым указано `id`, нажатие на идентификатор открывает страницу редактирования помещения. Параметр `list_display_links` позволяет явно указать, какие столбцы должны содержать такие ссылки.

Например:

```
list_display = ("id", "name", "capacity")
list_display_links = ("name",)
```

В этом случае ссылка на страницу изменения будет размещена в столбце `name`.

При нажатии на название помещения администратор перейдёт к форме изменения объекта. Идентификатор и вместимость останутся обычными значениями в таблице. Можно указать несколько полей:

```
list_display_links = ("id", "name")
```

Тогда перейти к редактированию можно будет по идентификатору или названию.

Если `list_display_links` не задан, Django применяет стандартное поведение. Если указать `list_display_links = None`, Django не будет автоматически делать столбцы ссылками на изменение. В таком случае следует учитывать, как администратор будет открывать форму редактирования объекта.

Поля, указанные в `list_display_links`, должны присутствовать в `list_display`. Также необходимо учитывать ограничения при совместном использовании `list_display_links` и `list_editable`: поле, которое редактируется непосредственно в списке, не может одновременно быть ссылкой на страницу изменения.

Для Roomly удобно сделать ссылкой название помещения, поскольку оно информативнее идентификатора.

5.2.2.5. Настройка поиска – search_fields

Когда помещений становится много, просматривать весь список вручную неудобно. Для поиска по выбранным полям используется параметр `search_fields`.

Добавьте:

```
search_fields = ("name",)
```

Django добавит строку поиска по названию помещения. Например, администратор сможет ввести 301, чтобы найти «Аудитория 301», или часть названия, чтобы найти соответствующие записи.

Для текстовых полей стандартный поиск Django обычно выполняется по вхождению строки без учёта регистра, если это поддерживается используемой базой данных. Можно указать несколько полей:

```
search_fields = ("name",)
```

В текущей модели Room основным текстовым атрибутом является название, поэтому такого поиска достаточно.

Не следует путать поиск с фильтрацией. Поиск предназначен для ввода строки, которую необходимо найти в заданных полях. Фильтры позволяют выбирать условия из предложенных интерфейсом вариантов.

В дальнейшем при настройке `BookingAdmin` будет рассмотрен поиск по связанным моделям, например по названию помещения через `room__name`.

5.2.2.6. Настройка фильтрации – list_filter

Параметр `list_filter` добавляет на страницу списка средства фильтрации записей. Добавьте:

```
list_filter = ("capacity",)
```

Django создаст фильтр по полю `capacity`. Его конкретный вид зависит от типа поля и возможностей административного интерфейса.

Фильтрация позволяет ограничить список объектов по выбранным условиям. Например, администратор может использовать фильтр, чтобы быстрее найти помещения с нужной вместимостью. В отличие от `search_fields`, где пользователь вводит поисковую строку, `list_filter` предоставляет элементы интерфейса для выбора условий фильтрации.

Если в модели появятся дополнительные характеристики, например корпус или тип помещения, их тоже можно будет использовать для фильтрации.

При выборе полей необходимо учитывать их назначение. Для числовой вместимости фильтрация обычно удобнее, чем попытка искать значение через строку поиска. Не указывайте в list_filter поле, которого нет в модели. Django проверяет конфигурацию административного интерфейса и сообщает о некорректных настройках.

5.2.2.7. Настройка сортировки – `ordering`

Параметр `ordering` задаёт исходный порядок отображения записей. Добавьте:

```
ordering = ("name",)
```

В этом случае помещения будут выводиться в порядке возрастания значения поля `name`.

Чтобы задать обратный порядок, перед именем поля ставится дефис:

```
ordering = ("-name",)
```

Порядок определяется базой данных. При сортировке строк результат может зависеть от используемой системы и её правил сортировки.

Для помещений сортировка по названию помогает быстрее ориентироваться в списке. Для бронирований может быть удобна сортировка по дате, а для других моделей – по статусу или времени создания.

Параметр `ordering` определяет порядок по умолчанию. Он не изменяет данные в базе и не влияет на правила доступности помещений. Если администратор меняет сортировку через заголовок столбца, Django применяет соответствующий порядок для отображения списка.

5.2.2.8. Настройка количества записей на странице – `list_per_page`

Если в базе данных хранится много помещений, выводить все записи на одной странице неудобно. Страница может становиться слишком длинной, а поиск нужного

объекта среди множества строк – затруднительным. Для управления размером страницы используется параметр `list_per_page`.

Добавьте:

```
list_per_page = 20
```

Теперь Django будет показывать по 20 помещений на одной странице списка. Если записей больше, в административном интерфейсе появится постраничная навигация. Например, если в базе данных 53 помещения, список будет разделён на три страницы:

- первая страница – записи с 1-й по 20-ю;
- вторая страница – записи с 21-й по 40-ю;
- третья страница – записи с 41-й по 53-ю.

`list_per_page` принимает целое число, задающее размер страницы. При выборе значения нужно учитывать удобство работы: слишком маленькое число увеличивает количество переходов между страницами, а слишком большое делает список громоздким. Значение 20 подходит для учебного проекта и позволяет продемонстрировать работу пагинации.

`list_per_page` определяет размер страницы, а не максимальное количество записей, которое может храниться в базе данных. Все объекты остаются в базе, но административный интерфейс показывает их частями. Этот параметр действует на список объектов в административной панели. Он не меняет количество помещений на обычной странице сайта Roomly и не заменяет настройки пагинации пользовательских представлений.

5.2.2.9. Итоговая конфигурация RoomAdmin

После добавления всех параметров класс `RoomAdmin` должен выглядеть следующим образом:

```
from django.contrib import admin
from .models import Room
@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
    list_display_links = ("name",)
    search_fields = ("name",)
    list_filter = ("capacity",)
    ordering = ("name",)
    list_per_page = 20
```

Обобщим назначение всех параметров.

Параметр	Назначение
`list_display`	Определяет столбцы списка помещений.

Параметр	Назначение
`list_display_links`	Определяет, по каким столбцам можно перейти к странице изменения объекта.
`search_fields`	Настраивает поиск по выбранным полям.
`list_filter`	Добавляет фильтры для ограничения списка записей.
`ordering`	Задаёт порядок отображения записей по умолчанию.
`list_per_page`	Определяет количество записей на одной странице.

Все перечисленные параметры относятся к административному интерфейсу. Они не добавляют поля в модель, не изменяют структуру таблицы базы данных и не заменяют правила предметной области.

Если в проекте уже существует класс `RoomAdmin`, внесите в него необходимые настройки, сохранив остальные параметры. Не создавайте второй класс с тем же назначением и не регистрируйте одну модель повторно.

5.2.2.10. Проверка результата

После сохранения rooms/admin.py откройте административную панель Django:

```
http://127.0.0.1:8000/admin/
```

Войдите под созданной учётной записью суперпользователя и перейдите к списку помещений.

Выполните следующие проверки:

- Проверка отображения столбцов – убедитесь, что список содержит идентификатор, название и вместимость помещения.
- Проверка ссылки на редактирование – нажмите на название помещения. Должна открыться страница изменения соответствующего объекта.
- Проверка поиска – введите часть названия существующего помещения. В списке должны остаться записи, соответствующие поисковому запросу.
- Проверка фильтрации – используйте фильтр по вместимости и убедитесь, что список изменяется в соответствии с выбранным условием.
- Проверка сортировки – убедитесь, что при первоначальном открытии списка помещения упорядочены по названию.
- Проверка количества записей на странице. Если в базе больше 20 помещений, убедитесь, что список разделён на страницы. Если записей меньше, все они могут отображаться на одной странице – это нормальное поведение.

Если база данных пока не содержит помещений, пустой список не является ошибкой регистрации. Для проверки можно добавить тестовую запись через административную панель.

5.2.2.11. Результат выполнения пункта

После выполнения задания административный интерфейс модели Room должен предоставлять:

- отображение идентификатора, названия и вместимости помещения;
- переход к редактированию по названию;
- поиск по названию;
- фильтрацию по вместимости;
- сортировку по названию;
- постраничное отображение по 20 записей.

Если часть возможностей уже реализована, проверьте их работу и внесите только необходимые изменения.

5.2.3. Настройка административного интерфейса User и Booking

После настройки административного интерфейса модели Room необходимо аналогичным образом настроить модели `User` и `Booking`. Это позволит просматривать пользователей и бронирования, искать записи, фильтровать результаты и переходить к редактированию объектов непосредственно из административной панели Django.

При настройке необходимо учитывать связи между моделями. Объект `Booking` связан с помещением `Room` и пользователем `User`, поэтому в списке бронирований можно отображать не только идентификаторы связанных объектов, но и их понятные человеку характеристики: название помещения, имя пользователя, дату бронирования и признак отмены.

5.2.3.1. Проверьте модели User и Booking

Перед настройкой административного интерфейса изучите фактические определения моделей в проекте. В Roomly используется собственная модель `User` из приложения `users`. Она представляет пользователя предметной области – человека, который может оформлять бронирования. Её не следует путать с моделью `User`, которую Django предоставляет в приложении `django.contrib.auth`.

В данном задании необходимо работать именно с моделью Roomly:

```
from users.models import User
```

Импортировать `User` из `django.contrib.auth.models` вместо этого не нужно: это другая модель, предназначенная для аутентификации пользователей сайта.

Проверьте, что модель `User` содержит необходимые атрибуты, например `name` и `email`, а модель `Booking` имеет поля или связи, позволяющие получить помещение, пользователя, дату бронирования и признак отмены. Названия полей в административных настройках должны соответствовать фактическому коду проекта.

Пример структуры связей:

```
Room
  ├── id
  ├── name
  └── capacity
User
  ├── id
  ├── name
  └── email
Booking
  ├── id
  ├── room ──────→ Room
  ├── user ──────→ User
  ├── booking_date
  └── is_cancelled
```

Связи `room` и `user` позволяют обращаться к связанным объектам из экземпляра `Booking`. Например, выражение `booking.room.name` возвращает название помещения, а `booking.user.name` – имя пользователя.

Примечание. Приведённые далее примеры предполагают, что связи в Booking называются room и user, а соответствующие поля связанных моделей – name. Если в проекте используются другие имена, адаптируйте примеры, не заменяя существующие модели целиком.

5.2.3.2. Настройте административный интерфейс модели User

Откройте файл users/admin.py. Для простой регистрации модели достаточно одной команды:

```
from django.contrib import admin
from .models import User
admin.site.register(User)
```

После этого Django добавит модель `User` в административную панель. Однако стандартный список не позволяет явно задать набор столбцов, параметры поиска и сортировки. Для более удобной работы используйте класс `ModelAdmin`.

Класс `ModelAdmin` позволяет настраивать представление и обработку записей определённой модели в административном интерфейсе. Он не заменяет саму модель User и не определяет её поля: он описывает, как эти поля показываются и редактируются в админ-панели.

Замените простой вариант регистрации следующим кодом:

```
from django.contrib import admin
from .models import User
@admin.register(User)
class UserAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "email")
    list_display_links = ("name",)
```

```
    search_fields = ("name", "email")
    ordering = ("name",)
    list_per_page = 20
```

Приведенные настройки аналогичны рассмотренным в п. 5.3.

5.2.3.3. Настройте административный интерфейс модели `Booking`

В отличие от пользователя, бронирование связано с двумя другими объектами. Поэтому в списке бронирований особенно полезно показывать название помещения и имя пользователя, а не только значения внешних ключей.

Откройте файл bookings/admin.py.

Создайте класс `BookingAdmin`:

```
from django.contrib import admin
from .models import Booking
@admin.register(Booking)
class BookingAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "room",
        "user",
        "booking_date",
        "is_cancelled",
    )
    list_display_links = ("id",)
    search_fields = (
        "room__name",
        "user__name",
    )
    list_filter = (
        "booking_date",
        "is_cancelled",
    )
    ordering = ("-booking_date",)
    list_per_page = 20
```

Этот пример подходит для модели, в которой используются поля `room`, `user`, `booking_date` и `is_cancelled`. Если названия отличаются, скорректируйте настройки по фактической реализации.

Атрибут `list_display` отвечает за отображение связанных объектов.

```
list_display = (
    "id",
    "room",
    "user",
    "booking_date",
```

```
    "is_cancelled",
)
```

В списке отображаются идентификатор бронирования, помещение, пользователь, дата и признак отмены.

При указании `room` и `user` Django обращается к соответствующим объектам модели. Их строковое представление определяется методом `__str__()` каждой модели.

Например, если в модели `Room` реализован метод:

```
def __str__(self):
    return self.name
```

то в списке бронирований вместо технического представления объекта будет показано название помещения. Аналогично метод `__str__()` модели `User` позволяет отобразить имя пользователя. Поэтому полезно проверить, что все три модели имеют понятные строковые представления. Они используются не только в административных списках, но и в выпадающих списках связанных объектов при создании и редактировании бронирования.

`search_fields` – поиск по полям связанных моделей.

```
search_fields = (
    "room__name",
    "user__name",
)
```

Здесь используется двойное подчёркивание `__`. В Django ORM оно позволяет переходить от поля одной модели к полю связанной модели при построении запросов.

Выражение `room__name` означает: перейти от `Booking` по связи `room` к модели `Room`, затем обратиться к полю `name`. Выражение `user__name` работает аналогично: от бронирования перейти к связанному пользователю и использовать его имя.

Таким образом, в административном интерфейсе можно найти бронирование по названию помещения или имени пользователя, хотя эти значения не хранятся непосредственно в таблице бронирований.

Синтаксис можно представить так:

```
Booking
  ├── room__name ──→ Room.name
  └── user__name ──→ User.name
```

Обратите внимание, что двойное подчёркивание – это специальный синтаксис Django ORM, а не часть имени поля. В самой модели поле по-прежнему называется `room` или `user`.

`list_filter` – фильтрация бронирований.

```
list_filter = (
```

```
    "booking_date",
    "is_cancelled",
)
```

Позволяет отбирать бронирования по дате и признаку отмены. Фильтрация по `is_cancelled` особенно полезна для проверки бизнес-правила `Roomly`: отменённое бронирование не должно блокировать помещение на соответствующую дату.

В зависимости от типа поля и версии Django вид фильтра может отличаться. Не следует предполагать, что любое поле автоматически предоставляет диапазон дат или числовой интервал: проверьте фактически отображаемые элементы фильтрации в административной панели.

При необходимости в список фильтров можно добавить `room` или `user`:

```
list_filter = (
    "booking_date",
    "is_cancelled",
    "room",
    "user",
)
```

Это позволит выбирать записи по конкретному помещению или пользователю. Используйте такой вариант, если он действительно упрощает работу с данными; большое количество фильтров может сделать интерфейс менее удобным.

`ordering` – сортировка по дате бронирования.

```
ordering = ("-booking_date",)
```

Минус перед именем поля задаёт сортировку по убыванию. Поэтому сначала будут отображаться более поздние даты, а затем более ранние.

`list_display_links` – переход к записи.

```
list_display_links = ("id",)
```

В этом примере ссылкой на страницу редактирования становится идентификатор бронирования. Можно выбрать другое поле из `list_display`, если оно удобнее для работы. Не указывайте в `list_display_links` поле, которого нет в `list_display`.

`list_per_page` – размер страницы.

```
list_per_page = 20
```

Ограничивает количество отображаемых бронирований на одной странице. При большом количестве записей это помогает работать со списком без необходимости загружать и просматривать его целиком.

5.2.3.4. Проверьте регистрацию моделей

В предыдущих пунктах модель `Room` уже была зарегистрирована в rooms/admin.py. В результате административная панель должна содержать три доменные модели `Roomly`:

- `Room` – помещения;
- `User` – пользователи предметной области;
- `Booking` – бронирования.

Для каждой модели должна использоваться одна регистрация. Если применяется декоратор:

```
@admin.register(Booking)
class BookingAdmin(admin.ModelAdmin):
    ...
```

не нужно дополнительно вызывать:

```
admin.site.register(Booking)
```

Повторная регистрация одной модели в том же экземпляре административного сайта приводит к ошибке `AlreadyRegistered`.

Аналогично не следует одновременно оставлять старую регистрацию `admin.site.register(User)` и добавлять `@admin.register(User)` для той же модели. Выберите один способ регистрации.

5.2.3.5. Проверьте административную панель

Запустите сервер разработки:

```
python manage.py runserver
```

Откройте административную панель по адресу:

```
http://127.0.0.1:8000/admin/
```

Войдите под учётной записью суперпользователя, созданной в пункте 5.1.1.

Последовательно откройте списки `Rooms`, `Users` и `Bookings` (названия в интерфейсе могут отличаться в зависимости от настроек `verbose_name`).

Проверьте следующие действия.

1. В списке пользователей отображаются идентификатор, имя и адрес электронной почты.
2. Переход по имени пользователя открывает страницу редактирования соответствующей записи.
3. Поиск пользователей по имени и электронной почте возвращает подходящие записи.

4. В списке бронирований отображаются помещение, пользователь, дата и признак отмены.
5. Поиск бронирований по названию помещения и имени пользователя работает корректно.
6. Фильтрация позволяет отделить отменённые бронирования от активных.
7. Записи сортируются по дате в заданном порядке.
8. При наличии более 20 записей список разбивается на страницы.

Если в базе данных пока недостаточно записей, создайте несколько тестовых объектов через административную панель, соблюдая обязательные поля и связи. Для проверки поиска создайте записи с различающимися именами, адресами электронной почты и названиями помещений.

5.2.3.6. Разберите возможные ошибки

- Ошибка `AlreadyRegistered`.

Причина: одна и та же модель зарегистрирована повторно. Проверьте, не остался ли вызов `admin.site.register()` рядом с декоратором `@admin.register()`.

- Ошибка `admin.E108` или `admin.E116`.

Причина: в настройках `list_display`, `list_display_links` или `list_filter` указано имя, которое Django не может использовать в соответствующем параметре. Сверьте названия с определением модели и документацией для конкретной настройки.

- Ошибка при поиске по `room__name` или `user__name`.

Проверьте, существуют ли поля `room` и `user` в модели `Booking`, являются ли они связями и действительно ли связанные модели содержат поле `name`. Путь поиска должен соответствовать реальной структуре моделей.

Вместо имени пользователя отображается техническое представление объекта.

Проверьте метод `__str__()` модели User. Аналогично проверьте Room, если вместо названия помещения отображается техническое представление.

- В списке бронирований отсутствуют записи или фильтры не дают ожидаемого результата.

Проверьте, что используете нужную базу данных и в ней действительно существуют записи. Убедитесь, что поле `is_cancelled` содержит корректные значения и что фильтр настроен на реальное поле модели.

- Изменения в admin.py не отражаются в интерфейсе.

Проверьте, сохранён ли файл, не возникли ли ошибки при запуске сервера и действительно ли изменяется admin.py нужного приложения. При необходимости перезапустите сервер и изучите сообщения в терминале.

5.2.3.7. Результат выполнения пункта

К завершению задания административная панель Roomly должна предоставлять удобные списки помещений, пользователей и бронирований. Для пользователей должны быть настроены отображение основных полей, поиск, сортировка и постраничный вывод. Для бронирований дополнительно должны работать поиск по связанным объектам и фильтрация по дате и признаку отмены.

Проверьте, что настройки соответствуют фактическим полям проекта, а модель `users.User` не перепутана со встроенной моделью аутентификации Django.

5.2.4. Управление записями и проверка прав доступа в административной панели

Административная панель Django предназначена не только для просмотра данных. Через неё можно создавать новые записи, изменять существующие объекты и удалять их, если у пользователя есть соответствующие разрешения.

В Roomly административная панель позволяет управлять помещениями, пользователями предметной области и бронированиями. Однако операции над этими объектами имеют разные последствия. Например, изменение названия помещения не должно нарушать связанные бронирования, а удаление помещения, на которое ссылаются бронирования, может быть запрещено на уровне модели.

В этом пункте необходимо проверить полный цикл работы с объектами через Django Admin и разобраться, как разрешения Django и связи между моделями влияют на доступные операции.

5.2.4.1. Проверьте создание объектов через административную панель

Откройте административную панель Roomly и войдите под суперпользователем.

Последовательно проверьте создание записей для моделей `Room`, `User` и `Booking`.

- Создание помещения:

1. Откройте список помещений и перейдите к добавлению нового объекта.
2. Заполните название помещения и вместимость в соответствии с полями фактической модели.
3. Сохраните запись.
4. Вернитесь к списку и убедитесь, что новое помещение появилось в нём.

- Создание пользователя предметной области

1. Откройте список пользователей `Roomly`.
2. Создайте пользователя, указав необходимые поля, например имя и адрес электронной почты.
3. Сохраните запись и проверьте её отображение в списке.

Не путайте эту операцию с созданием учётной записи Django. Создание объекта `users.User` добавляет пользователя предметной области, а не создаёт логин и пароль для входа в административную панель.

- Создание бронирования

Для создания бронирования должны существовать помещение и пользователь, на которые будет ссылаться новый объект `Booking`.

1. Откройте список бронирований и выберите добавление записи.
2. Выберите существующее помещение и пользователя.
3. Укажите дату бронирования.
4. Установите признак отмены в соответствии с требованиями модели и формы.
5. Сохраните запись и убедитесь, что она появилась в списке.

Если при создании бронирования поля помещения и пользователя представлены выпадающими списками, это связано с внешними ключами модели `Booking`. Django использует связи, чтобы пользователь мог выбрать существующие объекты, а не вводить произвольные идентификаторы вручную.

Фактический набор полей формы зависит от определения модели и настроек административного класса. Если обязательное поле не заполнено или значение не проходит проверку, Django не сохранит форму и покажет сообщение об ошибке.

5.2.4.2. Проверьте изменение существующих объектов

Откройте ранее созданные записи и измените некоторые значения.

Для помещения измените название или вместимость. Для пользователя измените имя или адрес электронной почты. Для бронирования измените дату либо признак отмены.

После сохранения проверьте, что новые значения отображаются в списке и сохраняются при повторном открытии объекта.

При изменении связанного объекта необходимо учитывать, что его данные могут отображаться в нескольких местах. Например, если изменить название помещения `Room`, то название, получаемое через `booking.room`, также изменится. При этом само бронирование останется связано с тем же объектом помещения.

Это важное свойство объектной модели: `Booking` хранит ссылку на объект `Room`, а не отдельную копию его названия.

Проверьте также, что изменение признака отмены бронирования отражается в административном списке. В Roomly отмена – это изменение состояния бронирования, а не обязательное удаление записи из базы данных.

Сохранение отменённого бронирования позволяет сохранять историю операций и отличать отменённые записи от активных.

5.2.4.3. Проверьте удаление записей и влияние связей между моделями

Удаление объекта может затрагивать другие записи, поэтому проверять эту операцию необходимо с учётом структуры моделей.

Например, если `Booking` содержит внешний ключ на `Room`, между записями существует зависимость:

```
Room
↑
| ForeignKey
|
Booking
```

В данном случае бронирование ссылается на помещение. Если попытаться удалить помещение, у которого есть связанные бронирования, результат будет зависеть от параметра `on_delete`, указанного во внешнем ключе. В Django параметр `on_delete` определяет поведение при удалении объекта, на который ссылается внешний ключ. Рассмотрим основные варианты.

Значение on_delete	Поведение при удалении связанного объекта
`models.CASCADE`	Связанные объекты также удаляются.
`models.PROTECT`	Удаление запрещается, если существуют защищённые связанные объекты.
`models.SET_NULL`	Внешний ключ устанавливается в NULL; поле должно допускать null=True.
`models.SET_DEFAULT`	Внешний ключ получает значение по умолчанию, которое должно быть допустимым.
`models.DO_NOTHING`	Django не выполняет автоматического изменения связанной записи; ограничения базы данных всё равно могут запретить удаление.

Для Roomly важно не допустить случайного удаления истории бронирований при удалении помещения или пользователя. Поэтому в моделях проекта может использоваться `on_delete=models.PROTECT`.

Например, если фактическая модель `Booking` содержит следующий внешний ключ:

```
room = models.ForeignKey(
"rooms.Room",
on_delete=models.PROTECT,
related_name="bookings",
)
```

то помещение, на которое ссылаются бронирования, нельзя удалить, пока существуют защищённые связанные записи.

Аналогичный принцип может применяться к пользователю:

```
user = models.ForeignKey(
```

```
"users.User",
on_delete=models.PROTECT,
related_name="bookings",
)
```

Эти фрагменты приведены для объяснения механизма. Не заменяйте ими существующие поля модели, если они уже реализованы: сначала проверьте фактические настройки связей.

Проведите проверку удаления.

1. Создайте помещение, на которое не ссылаются бронирования.
2. Попробуйте удалить это помещение через административную панель.
3. Убедитесь, что удаление разрешено, если другие ограничения отсутствуют.
4. Создайте бронирование, связанное с другим помещением.
5. Попробуйте удалить помещение, на которое ссылается бронирование.
6. Если для связи установлен `models.PROTECT`, убедитесь, что Django запрещает удаление и сообщает о наличии связанных объектов.

Аналогично проверьте удаление пользователя, у которого есть бронирования, если для связи `Booking.user` используется `models.PROTECT`.

Не удаляйте существующие учебные данные без необходимости. Для проверки используйте специально созданные тестовые объекты.

Почему отмена бронирования не заменяет его удаление? В Roomly отменённое бронирование должно сохраняться, поскольку оно является частью истории системы. Изменение `is_cancelled` и удаление объекта – разные операции:

- `booking.cancel()` или изменение `is_cancelled` меняет состояние бронирования;
- удаление объекта убирает запись из базы данных, если операция разрешена.

Следовательно, для освобождения помещения по бизнес-правилу следует отменить бронирование, а не удалять его.

5.2.4.4. Разберитесь в системе разрешений Django

Django создаёт стандартные разрешения для моделей при выполнении миграций. Обычно для каждой модели доступны разрешения на просмотр, добавление, изменение и удаление объектов.

Разрешение	Код разрешения	Назначение
Просмотр	`view_modelname`	Просмотр объектов модели.
Добавление	`add_modelname`	Создание объектов модели.
Изменение	`change_modelname`	Изменение существующих объектов.
Удаление	`delete_modelname`	Удаление объектов модели.

В кодах разрешений `modelname` обозначает имя модели в нижнем регистре. Например, для модели `Room` стандартные коды имеют вид `view_room`, `add_room`, `change_room` и `delete_room`.

Разрешения создаются для моделей Django, но возможность конкретного пользователя выполнять действия зависит от его прав и правил доступа.

Суперпользователь Django, созданный командой `createsuperuser`, по умолчанию обладает полным набором разрешений. Поэтому проверка только под суперпользователем не позволяет убедиться, что разграничение доступа работает корректно.

Для проверки обычных разрешений создайте отдельную группу и тестовую учётную запись Django.

5.2.4.5. Создайте группу и проверьте разрешения

1. В административной панели откройте раздел управления группами (Groups). Если раздел доступен, создайте группу, например Roomly Operators.
2. В списке доступных разрешений найдите разрешения модели `Room` и назначьте группе следующие права:

- `view_room` – просмотр помещений;
- `add_room` – создание помещений;
- `change_room` – изменение помещений.

3. Не назначайте разрешение `delete_room`.
4. При выборе разрешений ориентируйтесь на реальные названия приложений и моделей в административной панели.
5. Откройте раздел управления пользователями Django и создайте отдельную учётную запись для проверки разрешений.
6. Установите флаг доступа к административной панели (`is_staff`) и добавьте пользователя в группу `Roomly Operators`.
7. Не устанавливайте флаг суперпользователя (`is_superuser`).
8. Обратите внимание: здесь создаётся учётная запись Django для входа в административную панель, а не объект `users.User` из предметной области `Roomly`.
9. Выйдите из учётной записи суперпользователя и войдите под созданной тестовой учётной записью.
10. Проверьте, что пользователь может:

- просматривать список помещений;
- создавать новые помещения;
- изменять существующие помещения.

11. Убедитесь, что возможность удаления помещений не предоставляется через стандартный административный интерфейс.
12. Если раздел помещений или необходимые действия не отображаются, проверьте назначенные разрешения и наличие у тестовой учётной записи флага `is_staff`.
13. После проверки вернитесь к суперпользователю и убедитесь, что тестовые изменения сохранились.

Примечание: не назначайте тестовой учётной записи статус суперпользователя. Иначе проверка ограничений доступа потеряет смысл.

5.2.4.6. Познакомьтесь с настройками разрешений в ModelAdmin

Помимо стандартной системы разрешений Django, класс `ModelAdmin` предоставляет методы, с помощью которых можно уточнять доступность операций в административном интерфейсе.

Например:

```
def has_add_permission(self, request):
    return super().has_add_permission(request)
```

Этот метод определяет, может ли пользователь добавлять объекты через данный административный класс. В примере стандартное поведение Django сохраняется.

Аналогично существуют методы:

- `has_view_permission(self, request, obj=None)` – проверка права просмотра;
- `has_change_permission(self, request, obj=None)` – проверка права изменения;
- `has_delete_permission(self, request, obj=None)` – проверка права удаления.

Параметр `request` содержит информацию о текущем HTTP-запросе, в том числе о пользователе, который выполняет операцию. Параметр `obj` в методах, где он предусмотрен, позволяет учитывать конкретный объект.

Например, для модели `Room` можно временно запретить удаление объектов через административную панель:

```
from django.contrib import admin
from .models import Room

@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
    list_display_links = ("name",)
    search_fields = ("name",)
    ordering = ("name",)
```

```
    list_per_page = 20

    def has_delete_permission(self, request, obj=None):
        return False
```

В этом примере метод всегда возвращает `False`, поэтому удаление помещений через данный административный интерфейс запрещено, даже если пользователь имеет стандартное разрешение на удаление модели.

Однако такая настройка не является универсальной защитой от удаления данных во всём проекте. Она регулирует действия через соответствующий `ModelAdmin`, но не запрещает удаление через произвольный код приложения или другой интерфейс.

Кроме того, если вы добавляете такой метод в уже существующий `RoomAdmin`, не создавайте второй класс с тем же декоратором. Измените существующий класс, сохранив остальные его настройки.

Для текущего этапа достаточно понять назначение методов. Не оставляйте запрет удаления в проекте постоянно, если он не предусмотрен требованиями Roomly. После проверки удалите временное ограничение или восстановите стандартное поведение:

```
def has_delete_permission(self, request, obj=None):
return super().has_delete_permission(request, obj)
```

5.2.4.7. Сопоставьте разрешения Django и ограничения моделей

Разрешения административной панели и правила целостности данных решают разные задачи.

- Разрешения Django определяют, какие действия пользователь может выполнять через защищённые интерфейсы.
- Настройки `on_delete` определяют, что происходит со связанными объектами при удалении.
- Проверки модели и бизнес-логики обеспечивают соблюдение правил предметной области, например запрет повторного активного бронирования одного помещения на одну дату.

Например, пользователь может иметь разрешение на удаление помещения, но попытка удалить помещение с защищёнными бронированиями всё равно будет отклонена из-за `models.PROTECT`.

И наоборот, отсутствие разрешения на удаление не означает, что объект нельзя удалить программно из другого места приложения. Если требуется обеспечить бизнес-ограничение независимо от интерфейса, его необходимо реализовывать на подходящем уровне приложения и модели данных.

5.2.4.8. Итоговая проверка

Проверьте выполнение следующих действий:

- Создание помещения, пользователя и бронирования через административную панель.
- Изменение существующих объектов и сохранение изменений.
- Отмена бронирования без удаления записи.
- Удаление помещения, не имеющего связанных бронирований.
- Проверка поведения при удалении помещения, на которое ссылается бронирование.
- Проверка поведения при удалении пользователя, на которого ссылается бронирование.
- Создание тестовой группы с ограниченным набором разрешений.
- Проверка административной панели под тестовой учётной записью без прав суперпользователя.
- Проверка влияния методов ModelAdmin на доступность операций, если временное ограничение удаления было добавлено.

Если какая-либо проверка не проходит, установите причину: недостаток разрешений, ограничение внешнего ключа, ошибка в модели или особенность настроек административного класса.

5.2.4.9. Результат выполнения пункта

После выполнения задания необходимо понимать, как Django Admin управляет созданием, изменением и удалением объектов, чем стандартные разрешения отличаются от ограничений связей между моделями и почему отмена бронирования не должна подменяться удалением записи.

В Roomly административная панель должна сохранять целостность связей между Room, User и Booking, а доступные действия должны соответствовать разрешениям текущего пользователя.

5.2.5. Редактирование записей непосредственно в списке административной панели

В предыдущих пунктах была настроена административная панель Roomly, в которой можно просматривать записи, искать объекты и открывать отдельные страницы для их редактирования.

Django позволяет изменять некоторые поля непосредственно в таблице списка, не открывая форму каждого объекта. Для этого используется параметр `list_editable` класса `ModelAdmin`.

В `Roomly` эту возможность удобно применить к модели `Room`: например, разрешить изменять вместимость помещений прямо в списке. Это особенно полезно, когда необходимо скорректировать несколько значений подряд.

5.2.5.1. Принцип работы `list_editable`

Параметр `list_editable` определяет поля модели, которые можно изменять непосредственно в списке объектов административной панели. Например:

```
list_editable = ("capacity",)
```

После добавления этой настройки поле `capacity` будет представлено в списке помещений в виде элемента редактирования. Пользователь с соответствующими правами сможет изменить вместимость и сохранить изменения, не переходя на страницу каждого помещения.

При этом `list_editable` не заменяет стандартное редактирование объекта. Поля, не включённые в этот параметр, по-прежнему можно изменять на странице отдельной записи, если у пользователя есть необходимые разрешения.

Также важно различать два действия:

- Изменение записи через форму объекта. Пользователь открывает отдельную страницу, редактирует поля и сохраняет форму.
- Изменение записи в списке. Пользователь редактирует разрешённые поля непосредственно в таблице и сохраняет изменения с помощью кнопки сохранения, которая появляется в интерфейсе.

В обоих случаях Django проверяет разрешения пользователя и выполняет стандартную обработку формы.

5.2.5.2. Настройка редактирования вместимости помещения

Откройте файл rooms/admin.py и найдите существующий класс `RoomAdmin`.

Не создавайте второй класс для той же модели: измените уже существующую конфигурацию.

Добавьте параметр `list_editable`:

```
from django.contrib import admin
from .models import Room

@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "capacity")
    list_display_links = ("name",)
    list_editable = ("capacity",)
    search_fields = ("name",)
    list_filter = ("capacity",)
    ordering = ("name",)
    list_per_page = 20
```

В этом примере сохраняются ранее настроенные параметры административного интерфейса, а новое поведение задаётся одной строкой:

```
list_editable = ("capacity",)
```

Теперь вместимость можно менять непосредственно в списке помещений.

Обратите внимание: в примере предполагается, что поле модели действительно называется `capacity`. Если в вашей реализации используется другое название, укажите фактическое имя поля.

Шаг 1. Разберитесь в ограничениях `list_editable`

При использовании `list_editable` необходимо соблюдать несколько правил.

1. Редактируемое поле должно присутствовать в `list_display`.

Следующая конфигурация корректна:

```
list_display = ("id", "name", "capacity")
list_editable = ("capacity",)
```

Поле `capacity` отображается в списке и разрешено для редактирования.

Если добавить в `list_editable` поле, которого нет в `list_display`, Django сообщит об ошибке конфигурации.

2. Первое поле списка нельзя сделать редактируемым.

Например, такая конфигурация некорректна:

```
list_display = ("id", "name", "capacity")
list_editable = ("id",)
```

Первое поле списка нельзя включать в `list_editable`.

Кроме того, первичный ключ обычно не следует изменять вручную: он используется для идентификации объекта и связанных с ним записей.

3. Редактируемое поле не должно одновременно быть ссылкой на страницу объекта.

В нашем примере:

```
list_display_links = ("name",)
list_editable = ("capacity",)
```

Название помещения служит ссылкой на страницу редактирования, а вместимость изменяется непосредственно в списке. Нельзя одновременно сделать одно и то же поле ссылкой через `list_display_links` и редактируемым через `list_editable`.

4. Редактирование ограничивается правами пользователя.

Наличие `list_editable` само по себе не предоставляет права на изменение данных. Пользователь должен обладать разрешением `change_room`. Если пользователь может только просматривать помещения, Django не должен предоставлять ему возможность сохранять изменения.

5. Изменения необходимо сохранять.

В административной панели изменения полей списка обычно применяются после нажатия кнопки сохранения внизу таблицы. До сохранения изменённое значение в форме не означает, что оно уже записано в базу данных.

Шаг 2. Проверьте работу редактирования в списке

Запустите сервер разработки:

```
python manage.py runserver
```

Откройте административную панель и перейдите к списку помещений.

Выполните следующие действия:

1. Найдите одно из существующих помещений.
2. Измените его вместимость непосредственно в таблице.
3. Сохраните изменения.
4. Обновите страницу.
5. Убедитесь, что новое значение сохранилось.
6. Откройте страницу этого помещения и проверьте, что там отображается то же значение.

Затем проверьте поведение под тестовой учётной записью, если в предыдущем пункте вы настраивали ограниченные разрешения.

Пользователь без разрешения change_room не должен иметь возможности изменять и сохранять вместимость помещения.

Если в базе данных нет помещений, создайте несколько тестовых объектов через административную панель.

Шаг 3. Разберите возможные ошибки

- Ошибка `admin.E121`.

Она может возникнуть, если поле, указанное в `list_editable`, отсутствует в `list_display`. Проверьте, что редактируемое поле включено в список столбцов.

- Ошибка `admin.E124`.

Проверьте, не указано ли одно и то же поле одновременно в `list_editable` и `list_display_links`.

Ошибка `admin.E125`.

Проверьте, не включено ли первое поле `list_display` в `list_editable`.

- Поле отображается, но изменить его нельзя.

Проверьте конфигурацию `RoomAdmin`, наличие `list_editable` и права пользователя. Также убедитесь, что поле не исключено из формы административного интерфейса другими настройками.

- Изменение не сохраняется.

Убедитесь, что после редактирования нажата кнопка сохранения списка. Если возникает сообщение об ошибке, проверьте введённое значение и ограничения поля модели. Например, если `capacity` определено как `PositiveIntegerField`, отрицательное значение не соответствует требованиям поля.

Результат выполнения пункта

В административной панели Roomly должна появиться возможность изменять вместимость помещения непосредственно в списке.

Убедитесь, что изменения сохраняются, отображаются после обновления страницы и недоступны пользователям без права изменения помещений.

5.2.5.3. Добавление подсказок к полям модели с помощью help_text

При работе с административной панелью важно не только предоставить пользователю доступ к полям модели, но и объяснить, какие данные необходимо вводить.

Например, в Roomly поле `capacity` хранит вместимость помещения. Само название поля не всегда объясняет, какое значение следует указать: количество рабочих мест, максимальное число участников или другую характеристику. Для пояснения назначения поля в Django предусмотрен параметр `help_text`.

Шаг 1. Изучите назначение `help_text`

Параметр `help_text` задаётся при объявлении поля модели. Он содержит поясняющий текст, который Django может отображать рядом с соответствующим полем в форме. Например:

```
capacity = models.PositiveIntegerField(
verbose_name="Вместимость",
help_text="Укажите максимальное количество человек, которое может находиться в помещении.",
)
```

В административной панели под полем «Вместимость» будет отображаться соответствующая подсказка.

`help_text` помогает пользователю понять назначение поля, но сам по себе не проверяет введённое значение и не запрещает сохранять некорректные данные.

Для проверки данных в Django используются другие механизмы:

- тип поля модели и его ограничения;
- валидаторы;
- проверки, реализованные в формах;
- ограничения базы данных, если они предусмотрены моделью и поддерживаются используемой СУБД.

Таким образом, подсказка объясняет, какое значение ожидается, а проверка определяет, допустимо ли введённое значение.

Шаг 2. Добавьте подсказку к полю вместимости

Откройте файл rooms/models.py и найдите существующее определение модели Room. Добавьте параметр `help_text` к полю `capacity`, сохранив остальные параметры и поля модели.

Например, если модель имеет следующую структуру:

```
from django.db import models


class Room(models.Model):
    name = models.CharField(
        max_length=100,
        verbose_name="Название",
    )
    capacity = models.PositiveIntegerField(
        verbose_name="Вместимость",
    )
    def __str__(self):
        return self.name

    class Meta:
        verbose_name = "Помещение"
        verbose_name_plural = "Помещения"
измените только объявление поля capacity:
capacity = models.PositiveIntegerField(
    verbose_name="Вместимость",
    help_text="Укажите максимальное количество человек, которое может находиться в помещении.",
)
```

измените только объявление поля `capacity`:

```
capacity = models.PositiveIntegerField(
verbose_name="Вместимость",
help_text="Укажите максимальное количество человек, которое может находиться в помещении.",
)
```

Не заменяйте существующую модель целиком, если она отличается от приведённого примера. Сохраните остальные поля, методы и настройки.

Почему подсказку добавляют в модель, а не непосредственно в admin.py? Потому что `help_text` является характеристикой поля модели. Django может использовать его при построении стандартных форм на основе модели, в том числе в административной панели.

Если для поля применяется специально написанная форма или переопределено его отображение, поведение может отличаться. В рамках текущего задания используется стандартная форма Django Admin.

Шаг 3. Проверьте, требуется ли миграция

После изменения модели Django необходимо проверить состояние миграций:

```
python manage.py makemigrations
```

Затем примените миграции:

```
python manage.py migrate
```

В этом примере изменяется метаинформация поля – добавляется `help_text`. Тип поля, имя столбца и структура хранимых значений не меняются. Поэтому Django обычно создаёт миграцию, отражающую изменение состояния модели, без необходимости изменять физическую структуру таблицы базы данных.

Тем не менее не следует пропускать проверку миграций. Убедитесь, что Django обнаружил изменение и корректно обработал его.

Если команда `makemigrations` сообщает, что изменений нет, проверьте, сохранён ли файл модели и относится ли изменяемая модель к подключённому приложению.

Шаг 4. Проверьте отображение подсказки

- Запустите сервер разработки:

```
python manage.py runserver
```

- Откройте административную панель и перейдите к созданию или редактированию помещения.
- Найдите поле «Вместимость».
- Проверьте, что:

1. Под полем отображается добавленный поясняющий текст.
2. Подсказка помогает понять, какое значение требуется указать.
3. Поле продолжает работать как раньше.
4. Существующие записи доступны для просмотра и редактирования.
5. Добавление `help_text` не изменило значения вместимости в базе данных.

Если подсказка не отображается, убедитесь, что изменено именно поле используемой модели, сохранён файл и административная форма не переопределена другими настройками.

Шаг 5. Разграничьте подсказку и проверку данных

Рассмотрим два примера.

Пример 1. Подсказка

```
help_text="Укажите максимальное количество человек, которое может нахо-
диться в помещении."
```

Этот текст объясняет назначение поля. Он не гарантирует, что пользователь введёт разумное значение.

Пример 2. Ограничение типа поля

```
capacity = models.PositiveIntegerField(
    verbose_name="Вместимость",
    help_text="Укажите максимальное количество человек, которое может
находиться в помещении.",
)
```

Здесь тип поля задаёт требования к данным, а help_text объясняет их назначение. При этом точный набор проверок зависит от формы, валидаторов и ограничений базы данных.

Если бизнес-правила Roomly требуют, чтобы вместимость была строго больше нуля, одного `help_text` недостаточно: соответствующее ограничение следует обеспечить отдельным механизмом проверки.

Не добавляйте дополнительные проверки в рамках этого пункта без необходимости. Его задача – освоить добавление поясняющего текста и понять его назначение.

Результат выполнения пункта

В форме создания и редактирования помещения должна отображаться подсказка к полю вместимости. Она должна помогать пользователю правильно интерпретировать поле, не изменяя существующую логику хранения данных.

5.2.5.4. Отображение связанных бронирований на странице помещения с помощью `Inline`

В административной панели Django связанные модели обычно редактируются на отдельных страницах. Например, для просмотра бронирований определённого помещения приходится открывать раздел `Bookings` и искать нужные записи.

Django позволяет отображать связанные объекты непосредственно на странице родительской модели. Для этого используются встроенные формы – `inline forms`. Они особенно удобны, когда одна модель связана с несколькими объектами другой модели.

В Roomly эту возможность можно применить к связи между Room и Booking: на странице редактирования помещения будут отображаться связанные бронирования.

Шаг 1. Изучите принцип работы Inline

Предположим, что модель `Booking` содержит внешний ключ на `Room`:

```
# bookings/models.py
```

```
from django.db import models


class Booking(models.Model):
    # Каждое бронирование связано с одним помещением.
    # PROTECT запрещает удалять помещение, если на него ссылаются бронирования.
    room = models.ForeignKey(
        "rooms.Room",
        on_delete=models.PROTECT,
        related_name="bookings",
    )

    # Остальные поля модели Booking остаются без изменений.
```

В этом примере `Booking` является дочерней моделью по отношению к `Room`: одному помещению может соответствовать несколько бронирований.

Параметр `related_name="bookings"` задаёт имя обратной связи. Благодаря ему из объекта помещения можно получить связанные бронирования:

```
# Получаем все бронирования, связанные с конкретным помещением.
room.bookings.all()
```

Если `related_name` не указан, `Django` обычно создаёт имя обратного менеджера автоматически. В таком случае обращение выглядело бы примерно как `room.booking_set.all()`. В проекте следует использовать именно то имя, которое определено в модели.

Для отображения связанных записей на странице помещения Django предоставляет два основных класса:

- `TabularInline` – отображает дочерние объекты в виде таблицы;
- `StackedInline` – отображает поля каждого дочернего объекта отдельным вертикальным блоком.

Для бронирований удобнее начать с `TabularInline`: у каждого объекта сравнительно небольшой набор полей, поэтому табличное представление экономит место.

Шаг 2. Создайте класс `BookingInline`

Откройте файл rooms/admin.py.

Добавьте импорт модели Booking и определите класс BookingInline:

```
# rooms/admin.py

from django.contrib import admin

from .models import Room
from bookings.models import Booking
```

```
class BookingInline(admin.TabularInline):
    # Модель, записи которой будут показаны на странице Room.
    model = Booking

    # Поля бронирования, отображаемые в таблице.
    # Указывайте только реально существующие поля модели Booking.
    fields = ("user", "booking_date", "is_cancelled")

    # Запрещаем добавлять новые бронирования через встроенную таблицу.
    # Создание бронирований по-прежнему доступно в разделе Bookings.
    extra = 0

    # Скрываем возможность удаления бронирований из встроенной таблицы.
    # Это помогает сохранить историю бронирований при работе через
RoomAdmin.
    can_delete = False
```

Рассмотрим назначение новых настроек.

- `model` определяет модель, записи которой отображаются во встроенной таблице. В нашем случае это Booking.
- `fields` задаёт поля, которые необходимо показать. В примере выводятся пользователь, дата и признак отмены. Поле room не включено в список, поскольку на странице помещения оно уже известно: Django автоматически связывает создаваемые через inline формы записи с текущим объектом Room.
- `extra` задаёт количество дополнительных пустых форм для новых объектов. Значение 0 означает, что пустые строки для создания бронирований не отображаются. Существующие записи при этом продолжают показываться.
- `can_delete` управляет возможностью удалять объекты через `inline`. Значение False скрывает соответствующий механизм удаления в этой встроенной форме. Это не является универсальным запретом удаления бронирований: в другом административном интерфейсе или в коде приложения операция может оставаться доступной при наличии соответствующих прав и отсутствии ограничений модели.

Если в модели `Booking` поле пользователя называется не `user`, а иначе, скорректируйте `fields` в соответствии с фактической реализацией. Не создавайте поля только ради соответствия примеру.

Шаг 3. Подключите `BookingInline` к `RoomAdmin`

В том же файле rooms/admin.py найдите существующий класс `RoomAdmin` и добавьте в него настройку `inlines`.

```
# rooms/admin.py
```

```
from django.contrib import admin

from .models import Room
from bookings.models import Booking


class BookingInline(admin.TabularInline):
    # Показываем связанные бронирования в табличной форме.
    model = Booking
    fields = ("user", "booking_date", "is_cancelled")
    extra = 0
    can_delete = False


@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    # Настройки списка помещений, созданные в предыдущих пунктах.
    list_display = ("id", "name", "capacity")
    list_display_links = ("name",)
    list_editable = ("capacity",)
    search_fields = ("name",)
    list_filter = ("capacity",)
    ordering = ("name",)
    list_per_page = 20

    # Встраиваем связанные бронирования на страницу помещения.
    inlines = (BookingInline,)
```

Обратите внимание: это единый вариант файла, объединяющий ранее рассмотренные настройки `RoomAdmin` с новым механизмом `inline`. Если в вашем `RoomAdmin` уже есть другие параметры, сохраните их. Не создавайте второй класс `RoomAdmin` и не регистрируйте Room повторно.

Запись `inlines = (BookingInline,)` передаёт Django список классов встроенных форм. В данном случае используется кортеж из одного элемента. Запятая после элемента необходима для того, чтобы Python распознал выражение как кортеж.

Шаг 4. Проверьте результат в административной панели

Запустите сервер:

```
python manage.py runserver
```

Откройте административную панель, перейдите к списку помещений и откройте страницу редактирования одного из них.

В нижней части страницы должна появиться таблица связанных бронирований.

Проверьте следующие ситуации:

1. У помещения есть бронирования – они отображаются в таблице.
2. У помещения нет бронирований – таблица не содержит существующих записей, а дополнительные пустые формы не показываются.
3. У бронирования отображаются пользователь, дата и признак отмены.
4. Изменения в настройках `RoomAdmin` не исчезли после добавления `inlines`.
5. Через встроенную таблицу нельзя удалить бронирование, если используется `can_delete = False`.

Для проверки создайте несколько тестовых бронирований через отдельный раздел `Bookings`, а затем откройте соответствующие помещения.

Если у вас уже есть тестовые данные, повторно создавать их не требуется.

Шаг 5. Разберите ограничения и возможные ошибки

- Ошибка `admin.E202`.

Она может указывать на то, что Django не обнаружил подходящий внешний ключ от встроенной модели к родительской модели. Проверьте, что `Booking` действительно содержит `ForeignKey` на `Room`, а поле ссылается на правильную модель.

- Ошибка импорта `Booking`.

Проверьте путь импорта и наличие модели в bookings/models.py. Если модели организованы иначе, используйте фактическое расположение.

- Встроенная таблица не отображается.

Убедитесь, что `inlines = (BookingInline,)` находится внутри существующего класса `RoomAdmin`, а сам `RoomAdmin` зарегистрирован через `@admin.register(Room)` либо другим единственным способом.

- Поля не отображаются.

Проверьте значения `fields`: каждое имя должно соответствовать существующему полю модели `Booking` или допустимому элементу настройки `ModelAdmin`.

- Возникает ошибка доступа к связанным объектам.

Проверьте, что для каждого бронирования указаны существующие связанные объекты `Room` и `User`, а модель соответствует актуальным миграциям.

Шаг 6. Сравните `TabularInline` и `StackedInline`

Для понимания механизма полезно сравнить два варианта.

Класс	Представление	Когда удобно использовать
`TabularInline`	Таблица с отдельной строкой на объект	Когда у дочерней записи немного полей
`StackedInline`	Отдельный вертикальный блок для каждого объекта	Когда у дочерней записи много полей или сложная форма

В Roomly для бронирований обычно достаточно TabularInline, поскольку основные сведения можно компактно разместить в таблице. Если форма бронирования станет значительно сложнее, можно рассмотреть StackedInline.

Результат выполнения пункта

На странице редактирования помещения должна отображаться таблица связанных бронирований. Это позволит просматривать историю бронирований конкретного помещения непосредственно в административной панели.

5.2.5.5. Настройка полей только для чтения с помощью `readonly_fields`

В административной панели Django некоторые поля необходимо показывать пользователю, но не разрешать ему изменять. Например, идентификатор помещения присваивается автоматически при создании записи. Ручное изменение такого значения может нарушить связь помещения с бронированиями.

Для настройки таких полей используется параметр `readonly_fields` класса ModelAdmin.

Шаг 1. Добавление поля только для чтения

Откройте файл rooms/admin.py и найдите класс `RoomAdmin`, созданный в предыдущих пунктах. Добавьте в него параметр `readonly_fields`:

```
# rooms/admin.py

from django.contrib import admin

from .models import Room
from bookings.models import Booking


class BookingInline(admin.TabularInline):
    # Показываем бронирования, связанные с текущим помещением.
    model = Booking
    fields = ("user", "booking_date", "is_cancelled")
    extra = 0
    can_delete = False


@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    # Настройки списка помещений.
    list_display = ("id", "name", "capacity")
    list_display_links = ("name",)
    list_editable = ("capacity",)
    search_fields = ("name",)
    list_filter = ("capacity",)
```

```
    ordering = ("name",)
    list_per_page = 20

    # Поля, которые отображаются в форме, но недоступны для редактиро-
вания.
    readonly_fields = ("id",)

    # Встроенная таблица связанных бронирований.
    inlines = (BookingInline,)
```

В данном примере `id` – первичный ключ помещения, значение которого Django обычно назначает автоматически.

Параметр `readonly_fields` принимает кортеж имён полей или методов, которые необходимо отображать только для чтения. Если указано одно поле, запятая после его имени обязательна: `("id",)`.

Примечание: `readonly_fields` действует на форму редактирования объекта в административной панели. Этот параметр не запрещает изменять значение программно, через ORM, SQL или другие интерфейсы приложения.

Шаг 2. Проверка работы настройки

1. Запустите сервер разработки:

```
python manage.py runserver
```

2. Откройте административную панель Django и перейдите в раздел помещений.
3. Выберите существующее помещение, открыв его по ссылке в столбце названия.
4. Найдите поле id в форме редактирования.
5. Убедитесь, что идентификатор отображается, но изменить его через форму нельзя.
6. Проверьте, что остальные поля продолжают работать согласно настройкам: например, вместимость можно изменить, если у пользователя есть соответствующее разрешение.

Изменение `readonly_fields` не требует миграции базы данных: оно настраивает представление формы административной панели, а не структуру модели.

Шаг 3. Настройка поля только для чтения у бронирования

Аналогичную настройку можно использовать для модели Booking. Например, идентификатор бронирования также целесообразно показывать без возможности ручного редактирования.

Откройте bookings/admin.py и дополните существующий класс `BookingAdmin`:

```
# bookings/admin.py

from django.contrib import admin

from .models import Booking
```

```
@admin.register(Booking)
class BookingAdmin(admin.ModelAdmin):
    # Информация, отображаемая в списке бронирований.
    list_display = (
        "id",
        "room",
        "user",
        "booking_date",
        "is_cancelled",
    )

    # По нажатию на ID открывается карточка бронирования.
    list_display_links = ("id",)

    # Поиск по названиям помещений и именам пользователей.
    search_fields = ("room__name", "user__name")

    # Фильтры списка бронирований.
    list_filter = ("booking_date", "is_cancelled")

    # Сначала отображаются более поздние бронирования.
    ordering = ("-booking_date",)

    # Ограничение количества строк на одной странице.
    list_per_page = 20

    # Идентификатор можно просматривать, но нельзя менять через форму.
    readonly_fields = ("id",)
```

Если `BookingAdmin` уже содержит эти настройки, повторно создавать класс не нужно: достаточно добавить `readonly_fields = ("id",)` в существующий класс.

Названия полей в `list_display` и `search_fields` должны соответствовать фактической модели `Booking`. Приведённый пример предполагает наличие полей `room`, `user`, `booking_date` и `is_cancelled`.

Шаг 4. Проверка результата

После сохранения файлов:

1. Откройте существующее бронирование в административной панели.
2. Убедитесь, что поле `id` отображается только для чтения.
3. Проверьте, что остальные поля доступны в соответствии с настройками формы и правами пользователя.
4. Убедитесь, что список бронирований, поиск и фильтрация продолжают работать.

Что важно понимать

`readonly_fields` и разрешения Django решают разные задачи:

- `readonly_fields` определяет, какие поля нельзя редактировать через конкретную административную форму;
- разрешения `view`, `add`, `change` и `delete` определяют, какие действия с объектами доступны пользователю;
- ограничения модели и базы данных обеспечивают целостность данных независимо от интерфейса.

Таким образом, если поле должно быть недоступно для редактирования только в административной панели, можно использовать `readonly_fields`. Если ограничение должно действовать во всём приложении, необходимо предусмотреть соответствующие проверки в модели, бизнес-логике или на уровне базы данных – в зависимости от требования.

5.2.5.6. Отображение полей связанных таблиц в списках и оптимизация запросов

До сих пор в списках административной панели отображались поля самой модели: идентификатор, название, вместимость, дата бронирования. На практике для работы со списком бронирований полезно видеть рядом с помещением его вместимость, а в списке помещений – количество активных бронирований. Эти значения относятся к связанным таблицам, и Django позволяет выводить их прямо в строке списка.

При этом важно помнить о цене такого вывода. Каждое обращение к полю связанной модели в строке списка – это отдельный запрос к базе данных. Список из двадцати бронирований без оптимизации порождает один основной запрос и ещё по одному на каждое помещение и пользователя. Этот эффект называют проблемой N+1, а обнаружить его помогает Django Debug Toolbar, который рассматривается в пункте 5.3.6.

Шаг 1. Столбец на основе метода `ModelAdmin`

Добавим в `BookingAdmin` столбец «Вместимость», значение которого берётся из связанного помещения. Значение для такого столбца возвращает метод класса `ModelAdmin`.

```
# rooms/admin.py
class BookingAdmin(admin.ModelAdmin):
# ... существующие настройки ...
   @admin.display(description="Вместимость", ordering="room__capacity")
   def room_capacity(self, obj):
       return obj.room.capacity
```

Метод `room_capacity` принимает объект бронирования и возвращает вместимость связанного помещения. Декоратор `@admin.display` задаёт заголовок столбца

через параметр description и правило сортировки через параметр ordering. Без ordering столбец отображался бы, но упорядочить список по нему было бы нельзя. Выражение room__capacity означает: от бронирования перейти по связи room к полю capacity модели Room.

Чтобы столбец появился в списке, имя метода включается в кортеж list_display наряду с обычными полями. Существующие поля не удаляются – в кортеж добавляется только имя метода:

```
list_display = ("id", "room", "user", "room_capacity", "booking_date",
"is_cancelled")
```

Шаг 2. Количество активных бронирований у помещения

Решим обратную задачу: выведем в списке помещений число активных бронирований. Посчитать количество для каждой строки «в лоб» можно вызовом bookings.count(), но такой вызов выполнит отдельный запрос для каждого помещения. Правильный подход – добавить к запросу списка вычисляемое поле с помощью аннотации.

```
# rooms/admin.py
from django.db.models import Count, Q


class RoomAdmin(admin.ModelAdmin):
    # ... существующие настройки ...
    def get_queryset(self, request):
        qs = super().get_queryset(request)
        return qs.annotate(
    active_bookings=Count("bookings",        filter=Q(bookings__is_can-
celled=False)),)
    @admin.display(description="Активных  бронирований",  ordering="ac-
tive_bookings")
    def active_bookings_count(self, obj):
        return obj.active_bookings
```

Метод get_queryset() переопределяет набор записей, который административная панель использует для построения списка. Сначала вызывается родительская реализация, затем к набору добавляется аннотация active_bookings. Имя bookings взято из обратной связи related_name="bookings", рассмотренной в пункте 5.1.7, а объект Q, знакомый по пункту 4.3.4, ограничивает подсчёт только активными бронированиями, не учитывая отменённые. Теперь значение доступно в каждой строке без дополнительных запросов.

Метод `active_bookings_count` лишь читает готовое аннотированное значение, а параметр `ordering="active_bookings"` разрешает сортировку списка по вычисленному количеству.

Обратите внимание: если бы метод столбца вызывал `obj.bookings.count()` напрямую, административная панель выполняла бы по одному запросу на каждую строку списка. Аннотация в `get_queryset()` считает значения для всех строк одним запросом.

Шаг 3. Снижение числа запросов – `list_select_related`

Поля `room` и `user` в списке бронирований отображаются через свои методы `__str__()`, и для каждой строки Django по умолчанию запрашивает связанные объекты отдельно. Параметр `list_select_related` указывает панели получить все связанные записи одним запросом с объединением (JOIN).

```
class BookingAdmin(admin.ModelAdmin):
    # ... существующие настройки ...
    list_select_related = ("room", "user")
```

Список бронирований, который раньше выполнял один основной запрос и по одному на каждое помещение и пользователя, теперь обходится одним запросом с объединениями. Проверить это можно по количеству SQL-запросов в Django Debug Toolbar: откройте список бронирований до и после добавления параметра и сравните показатели на вкладке SQL.

Шаг 4. Проверка результата

1. Откройте список бронирований и убедитесь, что появился столбец «Вместимость» с корректными значениями.
2. Откройте список помещений и найдите столбец «Активных бронирований».
3. Щёлкните по заголовку каждого нового столбца и проверьте, что список упорядочивается.
4. Откройте список бронирований в Django Debug Toolbar и убедитесь, что количество SQL-запросов уменьшилось после добавления `list_select_related`.

Что важно понимать

Методы-столбцы и аннотации решают одну задачу разными способами:

- метод `ModelAdmin` возвращает значение для конкретной строки и удобен для преобразований на стороне Python;
- аннотация в `get_queryset()` вычисляет значение средствами базы данных и не создаёт лишних запросов;
- `list_select_related` устраняет проблему N+1 для полей, которые уже отображаются в списке.

Таким образом, вывод связанных данных допустимо начинать с простого метода, а по мере роста списков переносить вычисления в аннотации и включать `list_select_related`.

5.2.5.7. Массовые действия и быстрые выборы в административном интерфейсе

Над списком записей административной панели находится выпадающее меню массовых операций. По умолчанию в нём доступен только один пункт – «Удалить выбранные записи». Когда одну и ту же операцию требуется выполнить сразу для нескольких объектов, например отменить группу бронирований, открывать каждую запись по отдельности неудобно. Django позволяет добавить в это меню собственные пункты.

К быстрым элементам интерфейса относятся и другие настройки: навигация по датам над списком и поиск связанного объекта прямо в поле формы. Эти возможности не меняют данные, но заметно ускоряют повседневную работу администратора.

Шаг 1. Принцип работы массовых действий

Каждое массовое действие оформляется методом класса `ModelAdmin`, который принимает объект запроса `request` и набор записей `queryset`. В `queryset` попадают только записи, отмеченные флажками в списке. Чтобы действие появилось в меню, его имя включается в кортеж `actions`, а декоратор `@admin.action` задаёт понятную надпись пункта меню.

Шаг 2. Действие «Отменить выбранные бронирования»

Отмена бронирования в проекте выполняется предусмотренным методом cancel(), который рассматривался в пункте 5.1.4. Поэтому действие обходит выбранные записи циклом и вызывает этот метод для каждой из них.

```
# rooms/admin.py
from django.contrib import messages


class BookingAdmin(admin.ModelAdmin):
    # ... существующие настройки ...
    @admin.action(description="Отменить выбранные бронирования")
    def cancel_bookings(self, request, queryset):
        count = 0
        for booking in queryset:
            booking.cancel()
            count += 1
            self.message_user(
            request,
            f"Отменено бронирований: {count}.",
            messages.SUCCESS,
          )
```

Метод `message_user()` выводит администратору сообщение о результате: сколько записей обработано и с каким статусом. Без такого сообщения непонятно, сработало ли действие, поэтому его принято добавлять в каждое массовое действие.

Шаг 3. Действие «Вернуть в активные» и подключение меню

Второе действие возвращает ошибочно отменённые бронирования в активное состояние. Здесь изменение простое и не требует бизнес-проверок, поэтому достаточно одного запроса `queryset.update()`.

```
@admin.action(description="Вернуть в активные")
def restore_bookings(self, request, queryset):
    updated = queryset.update(is_cancelled=False)
    self.message_user(
    request,
    f"Возвращено бронирований: {updated}.",
    messages.SUCCESS,
    )
    actions = ("cancel_bookings", "restore_bookings")
```

Кортеж `actions` подключает оба действия к меню списка. Порядок пунктов в меню соответствует порядку имён в кортеже.

Шаг 4. Навигация по датам – `date_hierarchy`

Параметр `date_hierarchy` добавляет над списком быстрое меню выбора дат: сначала годы, затем месяцы и дни. Для списка бронирований это самый естественный способ перейти к записям конкретного месяца, не прибегая к фильтрам и поиску.

```
class BookingAdmin(admin.ModelAdmin):
# ... существующие настройки ...
date_hierarchy = "booking_date"
```

Поле должно быть датой или датой со временем. Если в текущей модели поле даты бронирования называется иначе, укажите фактическое имя.

Шаг 5. Быстрый выбор связанного объекта – `autocomplete_fields`

На странице бронирования поля `room` и `user` по умолчанию отображаются стандартными списками выбора, которые неудобны при большом количестве записей. Параметр `autocomplete_fields` превращает их в поля с поиском по мере ввода.

```
class BookingAdmin(admin.ModelAdmin):
# ... существующие настройки ...
autocomplete_fields = ("room", "user")
```

Для работы такого поля у связанной модели в её `ModelAdmin` обязательно должен быть настроен поиск `search_fields`. Для `Room` и `User` он уже настроен в пунктах 5.1.2 и 5.1.3, поэтому дополнительные изменения не требуются.

Шаг 6. Проверка результата

1. Выделите флажками два-три бронирования, выберите в меню «Отменить выбранные бронирования» и нажмите «Выполнить». Убедитесь, что появилось сообщение с количеством обработанных записей.
2. Выберите отменённые записи и верните их в активное состояние вторым действием.
3. Убедитесь, что над списком бронирований появилась навигация по датам, и откройте записи одного месяца.
4. Откройте форму бронирования и начните вводить название помещения: список вариантов должен сокращаться по мере ввода.
5. Откройте меню массовых операций и убедитесь, что в нём присутствуют оба добавленных пункта, а стандартный пункт удаления остался на месте.

Что важно понимать

При выборе между циклом с методом модели и запросом update() учитывайте следующее:

- метод `cancel()` может содержать бизнес-проверки, поэтому в действии отмены записи обрабатываются по одной;
- `update()` выполняется одним запросом и подходит для простых изменений, но не вызывает `save()` и методы модели;
- в меню массовых действий стоит оставлять только операции, которые безопасно применять к нескольким записям сразу.

После добавления этих настроек административный интерфейс Roomly покрывает повседневные сценарии администратора: быстрый поиск и фильтрация, массовая отмена бронирований, переход к записям конкретного месяца и удобный выбор помещения и пользователя при создании брони.

5.3. Работа с данными через Django ORM

5.3.1. Получение записей и основы работы с QuerySet

Django ORM (`Object-Relational Mapping`) позволяет работать с данными базы через Python-объекты и методы моделей. Вместо непосредственного написания SQL-запросов можно использовать методы `all()`, `filter()`, `exclude()`, `get()` и другие.

В Roomly ORM используется для получения помещений, пользователей и бронирований, а также для реализации поиска, фильтрации, проверки доступности помещений и формирования статистики.

В этом разделе рассматривается получение записей из базы данных и принцип работы QuerySet. Фильтрация, сортировка и поиск отдельных объектов будут разобраны далее.

Шаг 1. Получение всех помещений

Откройте терминал в каталоге Django-проекта, где находится файл manage.py, и запустите интерактивную оболочку:

```
python manage.py shell
```

В оболочке импортируйте модель Room:

```
# Импортируем модель помещения из приложения rooms.
from rooms.models import Room
```

Получите все помещения с помощью метода all():

```
# Формируем запрос для получения всех помещений.
rooms = Room.objects.all()
# Выводим полученный результат.
print(rooms)
```

Здесь используются два важных элемента:

- Room – модель, описывающая помещение;
- objects – менеджер модели, через который Django предоставляет операции с записями базы данных.

Метод all() возвращает не обычный список Python, а объект QuerySet.

QuerySet – это объект Django ORM, представляющий набор записей, получаемых по запросу к базе данных. Он позволяет строить запросы, объединять условия фильтрации и выполнять другие операции над набором записей.

Если в базе есть несколько помещений, запрос может вернуть результат следующего вида:

```
<QuerySet [<Room: Аудитория 301>, <Room: Конференц-зал>]>
```

Конкретный вывод зависит от данных в базе и метода __str__() модели Room. Если этот метод не определён, представление объектов может быть менее информативным.

Шаг 2. Перебор записей QuerySet

Объект QuerySet можно перебирать в цикле так же, как другие коллекции Python:

```
# Получаем набор всех помещений.
rooms = Room.objects.all()
# Перебираем помещения и выводим их характеристики.
for room in rooms:
```

```
    print(room.id, room.name, room.capacity)
```

В данном примере `room` на каждой итерации – отдельный экземпляр модели `Room`.

Поэтому для обращения к характеристикам помещения используется точечная запись:

```
room.id
room.name
room.capacity
```

а не обращение к элементам словаря через квадратные скобки.

Если в базе находятся три помещения, цикл выполнится три раза. Если записей нет, цикл не выполнится ни разу.

Для проверки можно вывести только названия помещений:

```
# Получаем все помещения и выводим их названия.
for room in Room.objects.all():
print(room.name)
```

Этот подход будет использоваться при подготовке данных для страниц Roomly.

Шаг 3. Преобразование `QuerySet` в список

При необходимости результат можно преобразовать в обычный список Python с помощью `list()`:

```
# Получаем набор помещений и материализуем его в список.
rooms_list = list(Room.objects.all())
# Проверяем тип полученного объекта.
print(type(rooms_list))
# Выводим количество элементов списка.
print(len(rooms_list))
```

После преобразования `rooms_list` является обычным списком, содержащим объекты `Room`.

Однако преобразовывать каждый `QuerySet` в список без необходимости не следует. Django ORM умеет самостоятельно перебирать набор записей и применять к нему дополнительные условия.

Например, если требуется только перебрать помещения, достаточно написать:

```
# Отдельный список создавать не требуется.
for room in Room.objects.all():
    print(room.name)
```

Примечание: преобразование в список приводит к выполнению запроса и загрузке полученных записей. Для больших таблиц это может потребовать значительных ресурсов.

Шаг 4. Понимание отложенного выполнения запросов

Одно из важных свойств `QuerySet` – отложенное выполнение запроса (`lazy evaluation`). Рассмотрим код:

```
# Создаём QuerySet для получения всех помещений.
rooms = Room.objects.all()
# Пока только выводим объект QuerySet.
print("Запрос подготовлен")
# Здесь выполняем перебор записей.
for room in rooms:
print(room.name)
```

При создании `QuerySet` Django обычно ещё не обращается к базе данных за самими записями. Он формирует объект, описывающий запрос.

Запрос выполняется, когда Django требуется получить результаты. Например, это происходит при переборе `QuerySet` в цикле или при преобразовании его в список. Это позволяет последовательно уточнять запрос до его выполнения. Например:

```
# Формируем запрос для получения помещений.
rooms = Room.objects.all()
# Добавляем сортировку по названию.
rooms = rooms.order_by("name")
# Получаем и выводим результаты.
for room in rooms:
    print(room.name)
```

Здесь `order_by()` будет подробно рассмотрен в одном из следующих пунктов. Пока важно понять принцип: Django позволяет строить запрос постепенно, а затем получать результат.

Вместо получения всех записей и сортировки средствами Python сортировка задаётся в ORM и выполняется базой данных.

Шаг 5. Подсчёт количества записей

Если требуется узнать, сколько помещений хранится в базе, не обязательно загружать все записи в Python.

Для этого используется метод `count()`:

```
# Получаем количество помещений в базе данных.
room_count = Room.objects.count()
# Выводим результат.
print(room_count)
```

Метод `count()` возвращает целое число – количество записей, соответствующих запросу. Например, если в базе хранится 15 помещений, результатом будет число 15. Сравним два подхода:

```
# Подсчитываем записи средствами базы данных.
room_count = Room.objects.count()
# Альтернативный вариант: загружаем записи и считаем элементы.
rooms = list(Room.objects.all())
room_count_from_list = len(rooms)
```

Для простого подсчёта обычно предпочтителен `count()`: база данных выполняет подсчёт, не передавая приложению все объекты помещений.

Если же записи уже были получены и загружены в память, повторное обращение к базе только ради подсчёта может оказаться излишним. Выбор зависит от того, какие данные уже доступны в программе.

Шаг 6. Проверка существования записей

Иногда необходимо выяснить, есть ли в базе хотя бы одно помещение. Для этого у `QuerySet` есть метод `exists()`:

```
# Проверяем, существует ли хотя бы одно помещение.
has_rooms = Room.objects.exists()
# Выводим True или False.
print(has_rooms)
```

Метод возвращает логическое значение:

- `True` – существует хотя бы одна запись;
- `False` – записей нет.

В отличие от загрузки всех помещений, `exists()` предназначен именно для проверки наличия записей.

Например, такая проверка может пригодиться при подготовке страницы со списком помещений:

```
# Проверяем, есть ли помещения в базе.
if Room.objects.exists():
    print("Помещения найдены")
else:
    print("Помещения пока не добавлены")
```

Это пример использования ORM в Python-коде. В реальном Django-представлении результат обычно передаётся в шаблон, где отображается соответствующее сообщение.

Шаг 7. Проверка через Django `shell`

Выполните в оболочке Django следующие команды:

```
# Импортируем модель.
from rooms.models import Room

# Получаем все помещения.
```

```
rooms = Room.objects.all()
print(rooms)

# Перебираем записи.
for room in rooms:
    print(room.id, room.name, room.capacity)

# Считаем количество помещений.
print(Room.objects.count())

# Проверяем наличие хотя бы одного помещения.
print(Room.objects.exists())
```

Сопоставьте результаты:

- `all()` возвращает `QuerySet`;
- перебор позволяет получить отдельные объекты модели;
- `count()` возвращает количество записей;
- `exists()` сообщает, есть ли записи.

Если база данных пуста, это не обязательно означает ошибку: возможно, помещения ещё не были добавлены. При необходимости создайте тестовую запись через административную панель и повторите проверку.

Чтобы выйти из интерактивной оболочки, выполните:

```
exit()
```

Типичные ошибки

- Ошибка 1. Попытка обращаться к `QuerySet` как к одному объекту

Неправильно:

```
# QuerySet представляет набор записей, а не одно помещение.
rooms = Room.objects.all()
print(rooms.name)
```

У набора записей нет атрибута `name`. Чтобы получить названия, нужно перебрать объекты:

```
# Обращаемся к атрибуту каждого отдельного помещения.
for room in Room.objects.all():
    print(room.name)
```

- Ошибка 2. Использование `len()` для подсчёта записей без необходимости

```
# Этот вариант загружает записи в список.
rooms = list(Room.objects.all())
print(len(rooms))
```

Если нужно только количество записей, используйте:

```
# Для простого подсчёта достаточно count().
print(Room.objects.count())
```

- Ошибка 3. Ожидание, что `all()` возвращает список

Метод `all()` возвращает `QuerySet`. Его можно перебирать, но это не обычный список. Если необходим именно список, используйте `list()`, учитывая, что это приводит к получению результатов запроса.

- Ошибка 4. Путаница между объектом модели и словарём

После перехода от JSON к Django ORM запись помещения представлена экземпляром `Room`, поэтому используются выражения `room.name` и `room.capacity`, а не `room["name"]` и `room["capacity"]`.

Соответствие методов Django ORM и операций SQL показано на рис. 4.

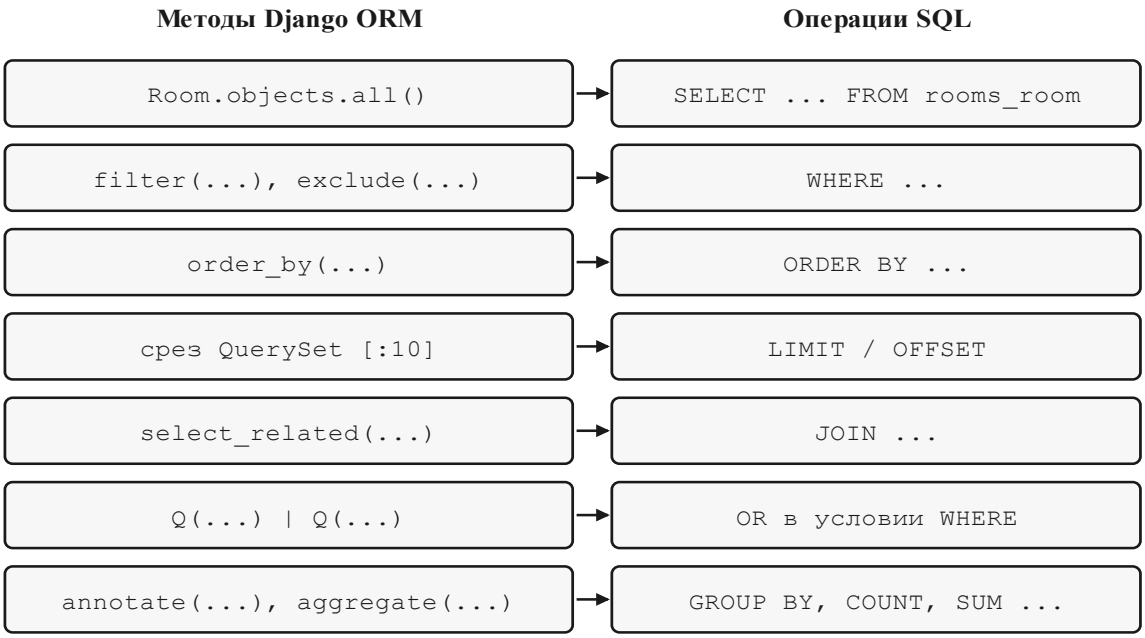


Рисунок 4 – Соответствие методов Django ORM и операций SQL

5.3.2. Фильтрация записей с помощью filter(), exclude() и условий поиска

При работе с Roomly обычно требуется получать не все записи из базы данных, а только те, которые соответствуют определённым условиям. Например, можно найти помещения вместимостью не менее 30 человек, выбрать помещения по части названия или получить бронирования за определённую дату.

Для этого в Django ORM используются методы `filter()` и `exclude()`, а также специальные выражения поиска – `field lookups`.

Шаг 1. Фильтрация помещений по одному условию

Откройте Django shell:

```
python manage.py shell
```

Импортируйте модель помещения:

```
# Импортируем модель Room.
```

```
from rooms.models import Room
```

Получите помещения вместимостью не менее 30 человек:

```
# Выбираем помещения, вместимость которых составляет 30 человек или больше.
rooms = Room.objects.filter(capacity__gte=30)
# Выводим характеристики найденных помещений.
for room in rooms:
    print(room.name, room.capacity)
```

В выражении `capacity__gte=30`:

- `capacity` – поле модели, по которому выполняется поиск;
- `__gte` – оператор сравнения «больше или равно»;
- 30 – значение, с которым сравнивается поле.

Двойное подчёркивание `__` используется в Django ORM для указания операции поиска или перехода по связи между моделями.

Метод `filter()` возвращает `QuerySet`, содержащий записи, соответствующие условию. Если подходящих помещений нет, будет получен пустой `QuerySet`. Это нормальная ситуация, а не ошибка.

Шаг 2. Использование основных операторов поиска

Django предоставляет набор стандартных операторов сравнения. Они указываются после имени поля через двойное подчёркивание.

Оператор	Значение	Пример
`exact`	Точное совпадение	`name__exact="Аудитория 301"`
`iexact`	Точное совпадение без учёта регистра	`name__iexact="аудитория 301"`
`contains`	Содержит указанную подстроку	`name__contains="301"`
`icontains`	Содержит подстроку без учёта регистра	`name__icontains="аудитория"`
`startswith`	Начинается с указанной строки	`name__startswith="Ауд"`
`istartswith`	Начинается с указанной строки без учёта регистра	`name__istartswith="ауд"`
`gt`	Больше	`capacity__gt=30`
`gte`	Больше или равно	`capacity__gte=30`
`lt`	Меньше	`capacity__lt=30`
`lte`	Меньше или равно	`capacity__lte=30`
`in`	Значение входит в заданную коллекцию	`capacity__in=[20, 30, 60]`

Рассмотрим несколько примеров для Roomly:

```
# Ищем помещения с названием, содержащим слово «зал».
rooms = Room.objects.filter(name__icontains="зал")
# Ищем помещения вместимостью строго больше 30 человек.
```

```
large_rooms = Room.objects.filter(capacity__gt=30)
# Ищем помещения вместимостью ровно 30 человек.
rooms_30 = Room.objects.filter(capacity=30)
# Ищем помещения с одной из заданных вместимостей.
selected_rooms = Room.objects.filter(capacity__in=[20, 30, 60])
```

В последнем примере запись `capacity=30` эквивалентна `capacity__exact=30`, поэтому оператор `exact` можно не указывать.

Результаты поиска зависят от фактических данных, а особенности сравнения строк могут зависеть от используемой СУБД и её настроек. Для поиска по части названия обычно удобно применять `icontains`.

Шаг 3. Использование нескольких условий

В одном вызове `filter()` можно указать несколько условий:

```
# Ищем помещения, которые:
# 1. имеют вместимость не менее 30 человек;
# 2. в названии содержат слово «зал».
rooms = Room.objects.filter(
    capacity__gte=30,
    name__icontains="зал",
)

# Выводим найденные помещения.
for room in rooms:
    print(room.name, room.capacity)
```

Условия, переданные в одном вызове `filter()`, объединяются логическим И. В результат попадут только записи, удовлетворяющие всем условиям одновременно.

Аналогичный запрос можно записать через последовательные вызовы:

```
# Сначала выбираем достаточно вместительные помещения.
rooms = Room.objects.filter(capacity__gte=30)
# Затем оставляем среди них помещения с нужным словом в названии.
rooms = rooms.filter(name__icontains="зал")
```

В данном случае результат будет эквивалентен предыдущему примеру. Django ORM строит уточняемый запрос, а не требует каждый раз вручную перебирать все записи в Python.

Шаг 4. Исключение записей с помощью `exclude()`

Метод `exclude()` позволяет получить записи, которые не соответствуют указанным условиям. Например, исключим помещения вместимостью менее 30 человек:

```
# Получаем помещения, не удовлетворяющие условию «вместимость меньше 30».
rooms = Room.objects.exclude(capacity__lt=30)
# Выводим результаты.
```

```
for room in rooms:
    print(room.name, room.capacity)
```

Для обычных числовых значений результат соответствует выборке помещений вместимостью не менее 30 человек.

Можно также исключить помещения с определённым названием:

```
# Исключаем помещения, в названии которых встречается слово «склад».
rooms = Room.objects.exclude(name__icontains="склад")
```

Если передать в `exclude()` несколько условий, Django исключит записи, для которых выполняются все указанные условия одновременно.

```
# Исключаем помещения, одновременно удовлетворяющие обоим условиям:
# вместимость не менее 30 человек и название содержит слово «зал».
rooms = Room.objects.exclude(
    capacity__gte=30,
    name__icontains="зал",
)
```

Это не то же самое, что исключить записи, удовлетворяющие хотя бы одному условию. Для более сложных сочетаний логических И, ИЛИ и НЕ используется объект `Q`, который будет рассмотрен отдельно.

Шаг 5. Фильтрация бронирований

Такие же методы применяются к модели `Booking`.

Импортируйте модель:

```
# Импортируем модель бронирования.
from bookings.models import Booking
```

Получим активные бронирования:

```
# Выбираем бронирования, которые не были отменены.
active_bookings = Booking.objects.filter(is_cancelled=False)

# Выводим информацию о найденных бронированиях.
for booking in active_bookings:
    print(
        booking.id,
        booking.room,
        booking.user,
        booking.booking_date,
    )
```

Здесь используется обычное сравнение логического поля с `False`.

Можно получить только отменённые бронирования:

```
# Выбираем записи с признаком отмены.
cancelled_bookings = Booking.objects.filter(is_cancelled=True)
```

Или исключить отменённые бронирования из общего набора:

```
# Исключаем отменённые бронирования.
active_bookings = Booking.objects.exclude(is_cancelled=True)
```

Для поля `is_cancelled` эти два подхода дают одинаковый результат, если поле имеет обычное логическое значение `True` или `False` и не допускает `NULL`. Это важно для предметной области Roomly: отменённые бронирования сохраняются в истории, но не должны считаться активными при проверке занятости помещения.

Шаг 6. Фильтрация по связанным моделям

В Roomly модель `Booking` связана с `Room` и `User`. Поэтому можно отбирать бронирования по характеристикам связанных объектов. Например, найдём бронирования помещений, вместимость которых не менее 30 человек:

```
# Фильтруем бронирования по полю capacity связанной модели Room.
bookings = Booking.objects.filter(room__capacity__gte=30)

# Выводим сведения о найденных бронированиях.
for booking in bookings:
    print(
        booking.id,
        booking.room.name,
        booking.booking_date,
    )
```

Разберём выражение `room__capacity__gte=30`:

1. `room` – поле связи с моделью `Room`;
2. `capacity` – поле модели `Room`;
3. `gte` – оператор сравнения «больше или равно»;
4. 30 – значение для сравнения.

Таким образом, двойное подчёркивание может использоваться не только для операторов сравнения, но и для перехода от одной модели к связанной модели.

Можно фильтровать бронирования и по данным пользователя:

```
# Ищем бронирования пользователей, чьё имя содержит указанную строку.
bookings = Booking.objects.filter(user__name__icontains="Иван")
# Выводим найденные бронирования.
for booking in bookings:
    print(booking.id, booking.user.name, booking.room.name)
```

Названия `room`, `user`, `capacity` и `name` в этих примерах должны соответствовать фактическим полям моделей проекта. Поскольку Roomly использует собственную модель `users.User`, здесь применяется связь с ней, а не с `django.contrib.auth.models.User`.

Шаг 7. Фильтрация по дате бронирования

Если `booking_date` в модели `Booking` является полем `DateField`, Django ORM позволяет сравнивать даты и искать записи в заданном диапазоне.

Например:

```
# Импортируем тип date для задания календарных дат.
from datetime import date

# Получаем бронирования с 1 по 30 сентября 2026 года включительно.
bookings = Booking.objects.filter(
    booking_date__gte=date(2026, 9, 1),
    booking_date__lte=date(2026, 9, 30),
)

# Выводим найденные записи.
for booking in bookings:
    print(booking.id, booking.booking_date)
```

Оба условия применяются одновременно: дата должна быть не раньше 1 сентября и не позже 30 сентября.

Если поле `booking_date` в текущей модели имеет тип `CharField`, а не `DateField`, прежде чем применять сравнение дат, следует учитывать фактический тип хранения. Для корректной работы с календарными датами предпочтителен `DateField`, но менять тип поля в рамках этого задания не требуется.

Шаг 8. Практическая проверка

В Django `shell` выполните несколько запросов:

```
# Импортируем модели.
from rooms.models import Room
from bookings.models import Booking
# 1. Помещения вместимостью не менее 30 человек.
rooms = Room.objects.filter(capacity__gte=30)
print(list(rooms))

# 2. Помещения, в названии которых есть слово «зал».
rooms = Room.objects.filter(name__icontains="зал")
print(list(rooms))

# 3. Помещения вместимостью не менее 30 человек,
#    название которых содержит слово «зал».
rooms = Room.objects.filter(
    capacity__gte=30,
    name__icontains="зал",
)
print(list(rooms))
```

```
# 4. Активные бронирования.
bookings = Booking.objects.filter(is_cancelled=False)
print(list(bookings))

# 5. Бронирования помещений вместимостью не менее 30 человек.
bookings = Booking.objects.filter(room__capacity__gte=30)
print(list(bookings))
```

Для проверки используйте данные, которые фактически находятся в базе. Если запрос возвращает пустой набор, проверьте исходные записи и условия поиска, а не считайте результат ошибкой автоматически.

В этих командах `list()` используется только для наглядного вывода результатов в оболочке. В прикладном коде обычно можно работать непосредственно с `QuerySet`.

Типичные ошибки

- Ошибка 1. Неверное имя поля или оператора

```
# Неверно: оператор gte записан без двойного подчёркивания.
Room.objects.filter(capacity_gte=30)
```

Правильный вариант:

```
# Оператор поиска отделяется от имени поля двойным подчёркиванием.
Room.objects.filter(capacity__gte=30)
```

- Ошибка 2. Использование имени связанного поля без перехода к нему

```
# Неверно, если capacity находится в модели Room, а запрос выполняется
по Booking.
Booking.objects.filter(capacity__gte=30)
```

Правильный вариант:

```
# Сначала переходим по связи room, затем обращаемся к capacity.
Booking.objects.filter(room__capacity__gte=30)
```

- Ошибка 3. Ожидание, что несколько условий автоматически объединятся через ИЛИ

```
# Здесь нужны одновременно оба условия, а не любое из них.
Room.objects.filter(
capacity__gte=30,
name__icontains="зал",
)
```

Если требуется выбрать помещения, которые соответствуют хотя бы одному из условий, необходимо использовать `Q` и оператор |. Этот сценарий будет рассмотрен в отдельном пункте.

5.3.3. Сортировка записей и получение отдельных объектов

При работе с Roomly недостаточно уметь находить записи по условиям. Необходимо также управлять порядком результатов, получать отдельные объекты и ограничивать количество возвращаемых записей.

Например, список помещений можно сортировать по названию или вместимости, а при открытии карточки помещения необходимо получить конкретный объект `Room`. Если пользователь укажет несуществующий идентификатор, приложение должно корректно обработать эту ситуацию.

В Django ORM для этих задач используются методы `order_by()`, `reverse()`, `first()`, `last()`, `get()`, срезы QuerySet и функция `get_object_or_404()`.

Шаг 1. Сортировка с помощью `order_by()`

Откройте Django shell:

```
python manage.py shell
```

Импортируйте модель помещения:

```
# Импортируем модель помещения.
from rooms.models import Room
```

Получите помещения в алфавитном порядке по названию:

```
# Сортируем помещения по возрастанию названия.
rooms = Room.objects.order_by("name")
# Выводим отсортированный результат.
for room in rooms:
    print(room.name, room.capacity)
```

Метод `order_by()` задаёт порядок записей в SQL-запросе. По умолчанию сортировка по строковому полю выполняется по возрастанию в соответствии с правилами используемой СУБД и её сортировки.

Чтобы расположить помещения по убыванию вместимости, перед именем поля укажите дефис:

```
# Сначала выводим помещения большей вместимости.
rooms = Room.objects.order_by("-capacity")
for room in rooms:
    print(room.name, room.capacity)
```

Таким образом:

- `order_by("name")` – сортировка по возрастанию названия;
- `order_by("-name")` – сортировка по убыванию названия;
- `order_by("capacity")` – сортировка по возрастанию вместимости;
- `order_by("-capacity")` – сортировка по убыванию вместимости.

Можно задать несколько полей сортировки:

```
# Сначала сортируем по вместимости по возрастанию,
# а при одинаковой вместимости – по названию.
rooms = Room.objects.order_by("capacity", "name")
```

Второе поле используется для упорядочивания записей, у которых значения первого поля совпадают.

Если необходимо обеспечить стабильный порядок записей с одинаковыми значениями, можно добавить идентификатор:

```
# При одинаковой вместимости и названии используем ID.
rooms = Room.objects.order_by("capacity", "name", "id")
```

Это полезно, в частности, при постраничном отображении списка.

Шаг 2. Изменение порядка через `reverse()`

Метод `reverse()` позволяет изменить направление текущего порядка сортировки:

```
# Сначала сортируем помещения по названию.
rooms = Room.objects.order_by("name")

# Меняем порядок на обратный.
rooms = rooms.reverse()
for room in rooms:
    print(room.name)
```

В данном случае помещения будут расположены в обратном порядке относительно сортировки по названию. Для предсказуемого результата желательно сначала явно задать порядок через `order_by()`, а затем применять `reverse()`. Если порядок не определён, результат может зависеть от настроек сортировки модели и СУБД.

Шаг 3. Ограничение количества записей

Иногда требуется получить не все записи, а только первые несколько. Для этого можно использовать срез `QuerySet`. Например, выведем первые пять помещений по названию:

```
# Сначала задаём порядок.
rooms = Room.objects.order_by("name")
# Выбираем не более пяти первых записей.
rooms = rooms[:5]
for room in rooms:
    print(room.name, room.capacity)
```

Срез [:5] позволяет ограничить количество результатов. В отличие от преобразования всего набора в список с последующим отбором элементов, такой срез обычно приводит к ограничению количества записей на стороне базы данных.

Можно пропустить первые пять записей и получить следующие пять:

```
# Сортируем помещения по названию.
rooms = Room.objects.order_by("name")
# Пропускаем первые пять записей и получаем следующие пять.
rooms = rooms[5:10]
for room in rooms:
    print(room.name)
```

Срезы особенно полезны при реализации пагинации, когда список делится на страницы. Нельзя использовать отрицательные индексы для `QuerySet` так же, как для обычного списка. Например, `rooms`[-1] не является допустимым способом получить последнюю запись. Для этого предусмотрены методы `last()` и другие операции ORM.

Также не следует без необходимости преобразовывать весь `QuerySet` в список, чтобы затем взять несколько элементов: при большом объёме данных это может привести к загрузке лишних записей в память.

Шаг 4. Получение первого и последнего объектов

Если необходимо получить только один объект из набора, можно использовать `first()` или `last()`.

```
# Получаем первое помещение по названию.
first_room = Room.objects.order_by("name").first()
# Получаем последнее помещение по названию.

last_room = Room.objects.order_by("name").last()
# Проверяем результаты.

print(first_room)
print(last_room)
```

Если подходящих записей нет, оба метода возвращают `None`. Поэтому перед обращением к атрибутам результата нужно проверить, что объект найден:

```
# Получаем первое помещение по названию.
room = Room.objects.order_by("name").first()
if room is not None:
# Обращаемся к атрибутам только при наличии объекта.
    print(room.name, room.capacity)
else:
    print("Помещения не найдены")
```

Такой подход подходит, когда отсутствие записей является допустимой ситуацией и не должно приводить к исключению.

Шаг 5. Получение одной записи с помощью `get()`

Метод `get()` применяется, когда требуется получить конкретный объект, соответствующий условию.

Например, если известен идентификатор помещения:

```
# Получаем помещение с идентификатором 1.
room = Room.objects.get(id=1)

# Выводим сведения о найденном помещении.
print(room.name, room.capacity)
```

В отличие от `filter()`, который возвращает `QuerySet`, метод `get()` возвращает один экземпляр модели. Однако у `get()` есть важная особенность: он ожидает, что условию соответствует ровно одна запись. Возможны три ситуации:

1. Найдена ровно одна запись – возвращается объект.
2. Не найдено ни одной записи – возникает исключение `Room.DoesNotExist`.
3. Найдено несколько записей – возникает исключение `Room.MultipleObjectsReturned`.

Для поля первичного ключа, например `id`, третья ситуация обычно исключена, поскольку идентификаторы уникальны. Но при поиске по обычному полю, например названию, несколько записей могут соответствовать одному условию.

Пример обработки отсутствующей записи:

```
# Импортируем модель.
from rooms.models import Room

try:
    # Пытаемся получить помещение с указанным ID.
    room = Room.objects.get(id=999999)

    print(room.name)

except Room.DoesNotExist:
    # Выполняется, если помещение с таким ID отсутствует.
    print("Помещение не найдено")
```

В прикладном Django-коде вручную обрабатывать это исключение требуется не всегда: для типичного сценария получения объекта по URL предусмотрена функция `get_object_or_404()`.

Шаг 6. Обработка отсутствующего объекта через `get_object_or_404()`

В Roomly пользователь может открыть страницу помещения по адресу вида `/rooms/1/`. Если помещение с указанным идентификатором отсутствует,

приложение должно вернуть страницу ошибки 404, а не завершиться с необработанным исключением. Для этого используется функция `get_object_or_404()` из `django.shortcuts`.

В файле `rooms/views.py` найдите существующее представление карточки помещения. Если оно уже использует `get_object_or_404()`, повторно переписывать его не нужно – проверьте, что механизм работает корректно.

Пример представления:

```
# rooms/views.py

from django.shortcuts import get_object_or_404, render

from .models import Room


def room_detail(request, room_id):
    # Получаем помещение по ID.
    # Если запись отсутствует, Django вернёт HTTP 404.
    room = get_object_or_404(Room, id=room_id)

    # Передаём объект помещения в шаблон.
    context = {
        "room": room,
    }

    # Укажите фактический путь к шаблону карточки помещения.
    return render(request, "rooms/room_detail.html", context)
```

В проекте Roomly маршрут карточки помещения уже предусматривает параметр идентификатора. Сохраните фактические имена представления, URL и шаблона, если они отличаются от примера.

Функция `get_object_or_404()` работает следующим образом:

- если объект найден, возвращает его;
- если объект не найден, вызывает исключение Http404, которое Django преобразует в `HTTP`-ответ со статусом 404.

Это удобнее, чем вручную перехватывать `DoesNotExist` в каждом представлении. При этом функция не заменяет проверку прав доступа. Если объект существует, но пользователь не должен его видеть, правила авторизации необходимо реализовать отдельно.

Шаг 7. Применение сортировки в списке помещений Roomly

Теперь добавим выбор порядка сортировки в пользовательский интерфейс. Пользователь сможет выбирать сортировку по названию или вместимости, а также направление сортировки.

В `rooms/views.py` найдите существующее представление списка помещений. В предыдущем пункте оно было дополнено поиском с использованием `Q`. Сохраните эту логику и добавьте обработку параметра сортировки.

Ниже приведён пример объединённого представления. Используйте его как образец интеграции: если в вашем текущем представлении уже реализованы пагинация или другие фильтры, сохраните их и внесите только необходимые изменения.

```
# rooms/views.py

from django.db.models import Q
from django.shortcuts import render

from .models import Room


def room_list(request):
    # Получаем строку поиска из параметра q.
    search_query = request.GET.get("q", "").strip()

    # Получаем параметр сортировки.
    # Если он не задан, используем сортировку по названию.
    sort_by = request.GET.get("sort", "name")

    # Получаем направление сортировки.
    # Значение desc означает сортировку по убыванию.
    direction = request.GET.get("direction", "asc")

    # Начинаем с полного набора помещений.
    rooms = Room.objects.all()

    # Сохраняем ранее реализованный поиск.
    if search_query:
        condition = Q(name__icontains=search_query)

        # Для числового запроса ищем также точное совпадение вместимо-
сти.
        if search_query.isdigit():
            condition |= Q(capacity=int(search_query))

        rooms = rooms.filter(condition)
```

```
    # Разрешаем сортировку только по заранее заданным полям.
    # Это не позволяет произвольно передавать имена полей из URL.
    allowed_sort_fields = {
        "name": "name",
        "capacity": "capacity",
    }

    # Если параметр некорректен, используем сортировку по названию.
    sort_field = allowed_sort_fields.get(sort_by, "name")

    # Добавляем дефис для сортировки по убыванию.
    if direction == "desc":
        sort_field = f"-{sort_field}"

    # Добавляем ID как дополнительный критерий,
    # чтобы порядок записей с одинаковыми значениями был стабильнее.
    rooms = rooms.order_by(sort_field, "id")

    # Передаём данные в шаблон.
    context = {
        "rooms": rooms,
        "search_query": search_query,
        "sort_by": sort_by,
        "direction": direction,
    }

    # Сохраните фактический путь к шаблону списка Roomly.
    return render(request, "rooms/room_list.html", context)
```

Здесь используется словарь `allowed_sort_fields`. Не следует напрямую передавать произвольное значение из URL в `order_by()` без проверки. Явный список разрешённых полей помогает ограничить поведение интерфейса и избежать неожиданных запросов. Если в существующем представлении уже есть импорт `Q`, `Room` или `render`, не добавляйте дублирующие импорты – объедините их.

Добавление элементов управления сортировкой

В шаблоне списка помещений добавьте элементы выбора сортировки рядом с существующей формой поиска:

```
<!-- Форма выбора порядка сортировки помещений. -->
<form method="get">
    <!-- Сохраняем поисковый запрос при смене сортировки. -->
    <input type="hidden" name="q" value="{{ search_query }}">

    <label for="room-sort">Сортировать по:</label>
```

```
    <select id="room-sort" name="sort">
        <option value="name" {% if sort_by == "name" %}selected{% endif
%}>
            Названию
        </option>

        <option value="capacity" {% if sort_by == "capacity" %}se-
lected{% endif %}>
            Вместимости
        </option>
    </select>

    <label for="room-direction">Порядок:</label>

    <select id="room-direction" name="direction">
        <option value="asc" {% if direction == "asc" %}selected{% endif
%}>
            По возрастанию
        </option>

        <option value="desc" {% if direction == "desc" %}selected{% en-
dif %}>
            По убыванию
        </option>
    </select>

    <button type="submit">Применить</button>
</form>
```

Если форма поиска уже присутствует на странице, не удаляйте её. Можно объединить поиск и сортировку в одну форму, чтобы пользователь мог задать оба параметра одновременно. Важно, чтобы при смене сортировки сохранялось значение поискового запроса, а при поиске не терялся выбранный порядок. Путь к шаблону и его структура должны соответствовать текущей реализации Roomly.

Шаг 8. Проверка в браузере

Запустите сервер разработки:

```
python manage.py runserver
```

Проверьте работу списка помещений:

1. Откройте страницу `/rooms/`.
2. Выберите сортировку по названию.
3. Выберите сортировку по вместимости.

4. Измените направление сортировки на убывание.
5. Выполните поиск по названию и проверьте, что сортировка применяется к найденным помещениям.
6. Откройте карточку существующего помещения.
7. Измените идентификатор в URL на заведомо несуществующий и убедитесь, что отображается страница 404.

Дополнительно проверьте в Django `shell`:

```
# Импортируем модель.
from rooms.models import Room

# Получаем первые пять помещений по названию.
rooms = Room.objects.order_by("name")[:5]

for room in rooms:
    print(room.id, room.name)

# Получаем первое помещение по названию.
first_room = Room.objects.order_by("name").first()
print(first_room)

# Получаем количество помещений.
print(Room.objects.count())
```

Типичные ошибки

- Ошибка 1. Ожидание, что `get()` всегда возвращает объект

Если запись отсутствует, `get()` вызывает `DoesNotExist`. Если условию соответствуют несколько записей, возникает `MultipleObjectsReturned`. Для получения объекта в Django-представлении с автоматическим ответом 404 удобно использовать `get_object_or_404()`.

- Ошибка 2. Использование отрицательного индекса у `QuerySet`

Для получения последнего объекта используйте `last()`, а не отрицательный индекс.

- Ошибка 3. Сортировка только по полю с повторяющимися значениями

Если несколько помещений имеют одинаковую вместимость, их взаимный порядок может быть нестабильным. Добавление `id` в `order_by()` задаёт дополнительный критерий сортировки.

- Ошибка 4. Потеря поискового запроса при смене сортировки

Если параметры поиска и сортировки отправляются разными формами, убедитесь, что нужные параметры сохраняются. Иначе пользователь может потерять результаты поиска после изменения порядка.

- Ошибка 5. Полная замена существующего представления

В Roomly уже могут быть реализованы поиск, пагинация и другие механизмы. Вносите изменения в текущую реализацию, сохраняя действующую маршрутизацию и структуру шаблонов.

5.3.4. Получение отдельных полей и агрегирование данных с помощью values(), values_list(), aggregate() и annotate()

При работе с ORM не всегда требуется получать полноценные объекты моделей. Иногда необходимо выбрать только несколько полей, подсчитать количество записей или рассчитать статистические показатели.

Например, в Roomly можно получить список названий помещений, узнать среднюю вместимость, подсчитать количество бронирований каждого помещения или сформировать сводную статистику.

Для этого используются методы `values()`, `values_list()`, `aggregate()` и `annotate()`.

Шаг 1. Получение отдельных полей с помощью `values()`

Метод `values()` позволяет выбрать только указанные поля и получить результат в виде словарей Python.

Откройте Django shell:

```
python manage.py shell
```

Импортируйте модель помещения:

```
# Импортируем модель помещения.
from rooms.models import Room
```

Получите только названия и вместимость помещений:

```
# Импортируем модель помещения.
from rooms.models import Room
Получите только названия и вместимость помещений:
# Выбираем только два поля, а не полноценные объекты Room.
rooms = Room.objects.values("name", "capacity")

# Перебираем результаты.
for room in rooms:
    print(room["name"], room["capacity"])
```

В отличие от `Room.objects.all()`, результат `values()` содержит не экземпляры модели `Room`, а словари с выбранными полями.

Например, отдельная запись может выглядеть так:

```
{
"name": "Аудитория 301",
"capacity": 30,
```

```
}
```

Это удобно, когда для операции не нужны остальные атрибуты модели и достаточно нескольких значений. Обратите внимание на способ доступа к данным:

```
# Для результата values() используется ключ словаря.
room["name"]
room["capacity"]
```

В обычном экземпляре модели используются атрибуты:

```
# Для экземпляра Room используется точечная запись.
room.name
room.capacity
```

Не следует путать эти два типа результатов.

Шаг 2. Получение значений с помощью `values_list()`

Метод `values_list()` также выбирает отдельные поля, но возвращает их в виде кортежей.

```
# Получаем названия и вместимость помещений в виде кортежей.
rooms = Room.objects.values_list("name", "capacity")
for room in rooms:
print(room)
```

Пример результата:

```
('Аудитория 301', 30)
('Конференц-зал', 60)
```

Если необходимо получить значения только одного поля, можно использовать параметр `flat=True`:

```
# Получаем названия и вместимость помещений в виде кортежей.
rooms = Room.objects.values_list("name", "capacity")

for room in rooms:
    print(room)
```

Параметр `flat=True` допустим, когда выбирается ровно одно поле. Если указать несколько полей, Django сообщит об ошибке.

Сравним три способа получения данных:

Выражение	Результат
`Room.objects.all()`	Объекты модели Room
`Room.objects.values("name", "capacity")`	Словари
`Room.objects.values_list("name", "capacity")`	Кортежи
`Room.objects.values_list("name", flat=True)`	Значения одного поля

Выбирайте способ в зависимости от задачи. Если необходимо вызывать методы модели или работать с её поведением, обычно нужны экземпляры модели. Если

требуются только отдельные значения, можно использовать `values()` или `values_list()`.

Шаг 3. Вычисление общей статистики с помощью `aggregate()`

Метод `aggregate()` позволяет вычислять итоговые показатели по набору записей. В отличие от `annotate()`, который добавляет вычисляемые значения к отдельным объектам или группам записей, `aggregate()` обычно возвращает один словарь с результатами вычислений по всей выбранной выборке.

Например, рассчитаем количество помещений, среднюю вместимость и максимальную вместимость.

```
# Импортируем агрегатные функции.
from django.db.models import Avg, Count, Max, Min

# Рассчитываем общую статистику по помещениям.
room_stats = Room.objects.aggregate(
    total_rooms=Count("id"),
    average_capacity=Avg("capacity"),
    maximum_capacity=Max("capacity"),
    minimum_capacity=Min("capacity"),
)

# Выводим результаты.
print(room_stats)
```

Здесь используются следующие функции:

- `Count()` – подсчёт количества записей;
- `Avg()` – вычисление среднего значения;
- `Max()` – поиск максимального значения;
- `Min()` – поиск минимального значения.

Аргументы `total_rooms`, `average_capacity`, `maximum_capacity` и `minimum_capacity` – это имена ключей, под которыми результаты будут доступны в итоговом словаре. Например:

```
# Получаем количество помещений из результата агрегирования.
total_rooms = room_stats["total_rooms"]

# Получаем среднюю вместимость.
average_capacity = room_stats["average_capacity"]
```

Если в базе нет помещений, количество записей будет равно нулю, а некоторые статистические показатели, например среднее значение, будут равны `None`.

Метод `aggregate()` полезен для формирования сводных показателей на страницах статистики и в административных отчётах.

Шаг 4. Агрегирование после фильтрации

Агрегатные функции можно применять к уже отфильтрованному набору записей. Например, найдём среднюю вместимость помещений, рассчитанных не менее чем на 30 человек:

```
# Сначала выбираем достаточно вместительные помещения,
# затем вычисляем статистику только по найденным записям.
stats = Room.objects.filter(
    capacity__gte=30,
).aggregate(
    total_rooms=Count("id"),
    average_capacity=Avg("capacity"),
    maximum_capacity=Max("capacity"),
)

print(stats)
```

Порядок действий имеет значение: статистика рассчитывается по помещениям, которые удовлетворяют условию фильтрации.

Аналогичный принцип можно использовать для бронирований – например, подсчитывать количество активных записей:

```
# Импортируем модель бронирования.
from bookings.models import Booking

# Считаем только неотменённые бронирования.
active_booking_count = Booking.objects.filter(
    is_cancelled=False,
).count()

print(active_booking_count)
```

Если требуется только количество записей, достаточно `count()`. Если нужно получить несколько показателей одним итоговым словарём, можно использовать `aggregate()`.

Шаг 5. Добавление вычисляемых показателей с помощью `annotate()`

Метод `annotate()` позволяет добавить к результатам запроса вычисляемые значения. Например, можно подсчитать количество бронирований для каждого помещения. В отличие от общего количества бронирований, здесь результат будет отдельным для каждого помещения.

Сначала выполним группировку по помещению через модель `Booking`:

```
# Импортируем модель бронирования и функцию подсчёта.
from bookings.models import Booking
from django.db.models import Count
```

```
# Группируем бронирования по идентификатору и названию помещения.
# Для каждой группы рассчитываем количество бронирований.
room_booking_stats = (
    Booking.objects
    .values("room_id", "room__name")
    .annotate(total_bookings=Count("id"))
    .order_by("room__name")
)

# Выводим статистику.
for item in room_booking_stats:
    print(
        item["room_id"],
        item["room__name"],
        item["total_bookings"],
    )
```

В этом запросе:

1. `values("room_id", "room__name")` задаёт поля, по которым формируются группы.
2. `annotate(total_bookings=Count("id"))` добавляет к каждой группе количество бронирований.
3. `order_by("room__name")` упорядочивает полученный результат по названию помещения.

Результат содержит только те помещения, для которых есть бронирования. Помещения без бронирований в эту выборку не попадут, поскольку запрос начинается с модели `Booking`. Это важно учитывать при разработке статистики: если требуется вывести все помещения, в том числе без бронирований, запрос нужно строить от модели `Room` с использованием обратной связи.

Шаг 6. Подсчёт бронирований для каждого помещения, включая помещения без бронирований

Для такого сценария удобно использовать `annotate()` непосредственно у `Room`. Сначала проверьте, как в модели `Booking` определена связь с `Room`:

```
# Фрагмент модели Booking — пример связи.
room = models.ForeignKey(
    Room,
    on_delete=models.PROTECT,
    related_name="bookings",
)
```

Если в фактической модели указан `related_name="bookings"`, обратная связь называется bookings. Если `related_name` не задан, стандартное имя обратной связи обычно имеет вид `booking_set`.

При `related_name="bookings"` запрос будет выглядеть так:

```
# Импортируем модель помещения и функцию подсчёта.
from rooms.models import Room
from django.db.models import Count

# Добавляем каждому помещению количество связанных бронирований.
rooms = Room.objects.annotate(
    total_bookings=Count("bookings"),
).order_by("name")

# Выводим название и рассчитанное количество.
for room in rooms:
    print(room.name, room.total_bookings)
```

Если в модели нет `related_name="bookings"`, замените `"bookings"` в `Count()` на фактическое имя обратной связи, например `"booking"` или другое имя, предусмотренное моделью. Не меняйте `related_name` только ради совпадения с примером без необходимости.

В данном запросе каждое помещение получает дополнительный атрибут `total_bookings`. Он вычисляется базой данных в рамках запроса и не является обычным полем модели, хранящимся в таблице.

Если помещение не имеет связанных бронирований, `Count()` вернёт для него 0. Поэтому этот вариант подходит для сводного списка всех помещений.

Шаг 7. Подсчёт активных бронирований через условную агрегацию

В Roomly отменённые бронирования сохраняются в истории, но не должны учитываться как активные. Поэтому полезно отдельно подсчитывать активные и отменённые записи.

Для этого можно использовать условный `Count()`:

```
# Импортируем инструменты условной агрегации.
from django.db.models import Count, Q
from rooms.models import Room

# Считаем активные и отменённые бронирования для каждого помещения.
rooms = Room.objects.annotate(
    active_bookings=Count(
        "bookings",
        filter=Q(bookings__is_cancelled=False),
    ),
```

```
    cancelled_bookings=Count(
        "bookings",
        filter=Q(bookings__is_cancelled=True),
    ),
)

# Выводим обе характеристики.
for room in rooms:
    print(
        room.name,
        room.active_bookings,
        room.cancelled_bookings,
    )
```

Этот пример предполагает, что обратная связь от `Room` к `Booking` называется `bookings`. Если в проекте используется другое имя, его необходимо заменить во всех соответствующих выражениях. Здесь `Q` применяется не для обычной фильтрации всего набора помещений, а для задания условия внутри агрегатной функции `Count()`.

Это принципиально отличается от такого запроса:

```
# Этот запрос исключает помещения без активных бронирований.
rooms = Room.objects.filter(
    bookings__is_cancelled=False,
).annotate(
    active_bookings=Count("bookings"),
)
```

Во втором случае фильтрация применяется к выборке до группировки, поэтому помещения без активных бронирований могут исчезнуть из результата. Условная агрегация позволяет сохранить все помещения и подсчитать только записи, удовлетворяющие нужному условию.

Если в ПР9 уже выполнялась условная агрегация с `Q`, данный пример следует рассматривать как перенос знакомого механизма на статистику помещений, а не как повторное введение темы с нуля.

Шаг 8. Применение статистики в Roomly

Теперь используем агрегирование в приложении. В Roomly уже предусмотрена страница статистики бронирований, поэтому сначала найдите соответствующее представление в `bookings/views.py`.

Если в нём уже рассчитываются показатели через ORM, не создавайте вторую страницу с дублирующим назначением. Дополните существующую реализацию нужными вычислениями.

Ниже приведён пример отдельного представления статистики по бронированиям:

```
# bookings/views.py

from django.db.models import Count
from django.shortcuts import render

from .models import Booking


def booking_statistics(request):
    # Рассчитываем общие показатели по всем бронированиям.
    stats = Booking.objects.aggregate(
        total_bookings=Count("id"),
        active_bookings=Count(
            "id",
            filter=Q(is_cancelled=False),
        ),
        cancelled_bookings=Count(
            "id",
            filter=Q(is_cancelled=True),
        ),
    )

    # Передаём статистику в шаблон.
    context = {
        "stats": stats,
    }

    # Укажите фактический путь к шаблону статистики Roomly.
    return render(request, "bookings/statistics.html", context)
```

В этом фрагменте используется `Q`, поэтому его необходимо импортировать:

```
# Импортируем функции подсчёта и объект для условий.
from django.db.models import Count, Q
```

Если `booking_statistics` уже существует, не создавайте ещё одну функцию с таким же именем: добавьте вычисления в существующее представление, сохранив его URL и другие действия. В шаблоне можно вывести полученные значения:

```
<h2>Статистика бронирований</h2>

<ul>
    <li>Всего бронирований: {{ stats.total_bookings }}</li>
    <li>Активных бронирований: {{ stats.active_bookings }}</li>
    <li>Отменённых бронирований: {{ stats.cancelled_bookings }}</li>
```

```
</ul>
```

Путь к шаблону и его структуру необходимо согласовать с фактической реализацией проекта. Если статистика уже отображается в существующем шаблоне, достаточно добавить соответствующие строки.

Поскольку отменённые бронирования не удаляются, общее количество записей включает и активные, и отменённые бронирования. Это позволяет сохранять историю и одновременно получать актуальные показатели.

Шаг 9. Проверка результата

В Django `shell` выполните:

```
# Импортируем модели и агрегатные функции.
from rooms.models import Room
from bookings.models import Booking
from django.db.models import Avg, Count, Max, Min, Q

# 1. Получаем отдельные поля помещений.
print(list(Room.objects.values("name", "capacity")))

# 2. Получаем только названия помещений.
print(list(Room.objects.values_list("name", flat=True)))

# 3. Рассчитываем общую статистику по помещениям.
print(
    Room.objects.aggregate(
        total_rooms=Count("id"),
        average_capacity=Avg("capacity"),
        maximum_capacity=Max("capacity"),
        minimum_capacity=Min("capacity"),
    )
)

# 4. Подсчитываем общее количество бронирований.
print(Booking.objects.count())

# 5. Подсчитываем активные и отменённые бронирования.
print(
    Booking.objects.aggregate(
        active=Count("id", filter=Q(is_cancelled=False)),
        cancelled=Count("id", filter=Q(is_cancelled=True)),
    )
)
```

Затем откройте страницу статистики Roomly в браузере и убедитесь, что отображаемые показатели соответствуют данным в базе.

Если в базе нет записей, часть агрегатных показателей может быть равна `None` или 0 в зависимости от функции. Проверяйте это поведение при формировании интерфейса.

Типичные ошибки

- Ошибка 1. Обращение к результату `values()` как к объекту модели

Результат `values()` содержит словари. Для него используется `item["name"]`, а не `item.name`.

- Ошибка 2. Использование `flat=True` с несколькими полями

`flat=True` применим только при выборе одного поля в `values_list()`.

- Ошибка 3. Путаница между `aggregate()` и `annotate()`
- `aggregate()` возвращает общий результат вычислений по выбранному набору записей.
- `annotate()` добавляет вычисляемые показатели к объектам или группам записей.
- Ошибка 4. Неверное имя обратной связи

В выражениях вроде `Count("bookings")` имя должно соответствовать фактическому `related_name` связи `Booking.room`. Если `related_name` отсутствует, используйте стандартное имя обратной связи.

- Ошибка 5. Исключение помещений без бронирований из статистики

Если запрос начинается с `Booking`, помещения без бронирований не появятся в результате. Для списка всех помещений используйте запрос от `Room` и условную агрегацию по обратной связи.

- Ошибка 6. Повторное создание страницы статистики

Если в Roomly уже реализовано представление статистики, расширяйте его, а не создавайте дублирующую страницу. Сохраните существующие URL, шаблон и ранее реализованные показатели.

5.3.5. Оптимизация запросов к связанным объектам

При работе с ORM важно не только получить правильные данные, но и не выполнять лишние обращения к базе данных. Особенно это заметно при выводе списков бронирований, в которых для каждой записи необходимо показать название помещения и имя пользователя.

В Roomly бронирование связано с двумя объектами:

- `Room` – помещение;
- User – пользователь, представленный моделью приложения `users`, а не встроенной моделью Django.

При обращении к связанным объектам Django может выполнять дополнительные SQL-запросы. Если в списке много бронирований, количество обращений к базе данных может существенно увеличиться.

В этом пункте необходимо сравнить обычный запрос и запрос с `select_related()`, научиться оценивать количество SQL-запросов и применить полученные знания к существующему приложению Roomly.

Шаг 1. Подготовка данных в Django `Shell`

Откройте терминал в каталоге проекта Roomly и активируйте виртуальное окружение. Запустите оболочку Django:

```
python manage.py shell
```

Импортируйте модели:

```
# Модель бронирования.
from bookings.models import Booking

# Модель помещения.
from rooms.models import Room

# Доменная модель пользователя приложения Roomly.
from users.models import User
```

Проверьте, что в базе данных есть бронирования:

```
# Получаем несколько бронирований для эксперимента.
bookings = Booking.objects.all()[:10]

# Выводим количество полученных записей.
print(bookings.count())
```

Если бронирований нет, сначала создайте несколько записей через административную панель Django или используйте существующие данные проекта. Для сравнения полезно иметь несколько бронирований, связанных с разными помещениями и пользователями.

В примерах предполагается, что модели находятся в указанных приложениях, а в `Booking` поля внешних ключей называются `room` и `user`. Если фактическая структура проекта отличается, используйте реальные имена моделей и полей.

Шаг 2. Обращение к связанным объектам без оптимизации

Выполните запрос:

```
# Получаем бронирования без дополнительной оптимизации.
bookings = Booking.objects.all()[:10]

# Выводим данные бронирований и связанных объектов.
for booking in bookings:
```

```
    print(
        booking.id,
        booking.booking_date,
        booking.room.name,
        booking.user.name,
    )
```

Здесь используются поля самого бронирования и атрибуты связанных объектов:

- `booking.id` – идентификатор бронирования;
- `booking.booking_date` – дата бронирования;
- `booking.room.name` – название помещения;
- `booking.user.name` – имя пользователя.

При чтении `booking.room.name` и `booking.user.name` Django должен получить связанные объекты, если они ещё не загружены.

В простом варианте это может привести к выполнению дополнительных запросов. Например, при выводе десяти бронирований ORM может выполнить один запрос для получения списка и отдельные запросы для получения помещений и пользователей. Точное количество зависит от данных, кэша связанных объектов и структуры запросов.

Такое поведение называют проблемой `N+1` запросов: один запрос получает основной набор записей, после чего выполняются дополнительные запросы для связанных данных каждой записи.

Шаг 3. Использование `select_related()`

Метод `select_related()` позволяет получить объект и связанные с ним объекты по внешним ключам в рамках одного SQL-запроса с использованием соединений таблиц. Выполните:

```
# Получаем бронирования и связанные помещения и пользователей
# в рамках одного запроса с JOIN.
bookings = (
    Booking.objects
    .select_related("room", "user")
    .all()[:10]
)

# Обращение к связанным объектам не требует отдельного
# запроса для каждого помещения и пользователя.
for booking in bookings:
    print(
        booking.id,
        booking.booking_date,
        booking.room.name,
```

```
        booking.user.name,
    )
```

Вызов:

```
.select_related("room", "user")
```

указывает Django загрузить вместе с бронированием связанные объекты `room` и `user`.

При этом имена "`room`" и "`user`" – названия полей модели `Booking`, а не названия таблиц и не имена приложений.

Метод `select_related()` подходит прежде всего для связей типа `ForeignKey` и `OneToOneField`. Он не предназначен для непосредственной загрузки коллекций связанных объектов через `ManyToManyField` или обратную связь «один ко многим» – для таких случаев применяется `prefetch_related()`.

Шаг 4. Сравнение количества SQL-запросов

Для проверки оптимизации можно использовать инструмент `CaptureQueriesContext` из Django. Он позволяет временно перехватить SQL-запросы, выполняемые внутри определённого блока кода.

В Django `Shell` выполните:

```
# Инструменты для перехвата выполненных SQL-запросов.
from django.db import connection
from django.test.utils import CaptureQueriesContext

# Вариант 1. Запрос без select_related().
with CaptureQueriesContext(connection) as queries_without_join:
    bookings = list(Booking.objects.all()[:10])

    # Обращаемся к связанным объектам, чтобы Django
    # действительно выполнил необходимые запросы.
    for booking in bookings:
        _ = booking.room.name
        _ = booking.user.name

print("Без select_related:", len(queries_without_join))
```

Затем сравните результат с оптимизированным запросом:

```
# Вариант 2. Запрос с select_related().
with CaptureQueriesContext(connection) as queries_with_join:
    bookings = list(
        Booking.objects
        .select_related("room", "user")
        .all()[:10]
    )
```

```
    # Связанные объекты уже загружены вместе с бронированиями.
    for booking in bookings:
        _ = booking.room.name
        _ = booking.user.name

print("С select_related:", len(queries_with_join))
```

Обратите внимание на несколько деталей:

1. В обоих вариантах используется одинаковый предел – десять бронирований.
2. Вызов `list()` выполняет ленивый запрос и загружает записи.
3. В первом варианте чтение связанных объектов может вызывать дополнительные запросы.
4. Во втором варианте связанные объекты загружаются через `JOIN`, поэтому отдельные запросы для их получения не нужны.

Точное количество запросов зависит от состояния базы данных и кэша, поэтому следует сравнивать фактические результаты, а не ожидать определённого числа во всех случаях.

Шаг 5. Просмотр SQL-запроса

Чтобы изучить SQL, сформированный ORM, выполните:

```
# Формируем QuerySet с загрузкой связанных объектов.
queryset = (
    Booking.objects
    .select_related("room", "user")
    .all()
)

# Выводим SQL, который соответствует QuerySet.
print(queryset.query)
```

В запросе должны присутствовать соединения таблицы бронирований с таблицами помещений и пользователей. Точный SQL зависит от названий таблиц и полей в проекте. Сравните его с запросом:

```
# Тот же набор бронирований без явной загрузки связанных объектов.
queryset = Booking.objects.all()

print(queryset.query)
```

Второй запрос обычно выбирает данные только из таблицы бронирований. Связанные объекты будут загружаться отдельно при обращении к ним.

Обращение к `queryset.query` показывает SQL для построения запроса, но само по себе не выполняет его. Для измерения количества реально выполненных запросов используйте предыдущий пример с `CaptureQueriesContext`.

Шаг 6. Применение в приложении Roomly

Теперь примените оптимизацию к существующему представлению списка бронирований. Откройте файл:

```
bookings/views.py
```

Найдите представление, которое формирует список бронирований и передаёт его в шаблон. Не создавайте второе представление с той же функцией и не заменяйте весь существующий код: необходимо изменить формирование `QuerySet`, сохранив текущие фильтры, сортировку, пагинацию и остальные параметры страницы.

Например, если сейчас используется:

```
# Получаем список бронирований.
bookings = Booking.objects.all()
```

и далее выполняется фильтрация, добавьте `select_related()` к исходному `QuerySet`:

```
# Получаем бронирования вместе со связанными объектами.
# Остальные фильтры и сортировку можно применять к этому QuerySet.
bookings = Booking.objects.select_related("room", "user")
```

Если в существующем представлении уже есть фильтрация, например:

```
# Пример структуры запроса: сохраните реальные условия проекта.
bookings = (
    Booking.objects
    .filter(is_cancelled=False)
    .select_related("room", "user")
    .order_by("booking_date")
)
```

то `select_related()` можно добавить к существующей цепочке, не меняя условия отбора. В примере выше фильтр исключает отменённые бронирования; используйте его только в том случае, если именно такое поведение требуется текущей странице.

Если в представлении уже применяются `select_related("room", "user")`, не добавляйте повторный вызов. Вместо этого проверьте, действительно ли все необходимые связанные объекты загружаются эффективно.

В шаблоне списка бронирований могут использоваться выражения вида:

```
{# Название помещения, связанного с бронированием. #}
{{ booking.room.name }}
{# Имя пользователя, создавшего бронирование. #}
{{ booking.user.name }}
```

Сам шаблон менять только ради `select_related()` не требуется: оптимизация выполняется на уровне `QuerySet` в представлении, а способ отображения данных остаётся прежним.

Шаг 7. Проверка результата в браузере

Запустите сервер разработки:

```
python manage.py runserver
```

Откройте существующую страницу списка бронирований Roomly и проверьте:

- список бронирований отображается корректно;
- для каждой записи показываются правильные помещение и пользователь;
- фильтрация, сортировка и пагинация продолжают работать;
- сведения об отменённых бронированиях отображаются в соответствии с правилами текущей страницы;
- переход к деталям бронирования работает по-прежнему.

Если в проекте уже установлена Django Debug Toolbar, используйте её для просмотра SQL-запросов, выполненных при открытии страницы. Сравните количество запросов до и после изменения. Если Debug Toolbar ещё не настроена, её установку и настройку рассмотрим отдельно – для выполнения этого пункта она не обязательна.

Шаг 8. Дополнительная проверка обратной связи

`select_related()` используется для загрузки связанных с бронированием помещения и пользователя. Но если необходимо получить бронирования для конкретного помещения, можно использовать обратную связь.

Например, при наличии в модели `Booking` поля:

```
# Пример поля внешнего ключа, если оно соответствует модели проекта.
room = models.ForeignKey(
    Room,
    on_delete=models.PROTECT,
    related_name="bookings",
)
```

обратная связь будет доступна так:

```
# Получаем помещение по идентификатору.
room = Room.objects.get(id=1)

# Получаем бронирования этого помещения.
room_bookings = room.bookings.all()
```

Если у поля не задан `related_name="bookings"`, стандартное имя обратного менеджера в Python – `room.booking_set`. Не изменяйте модель только ради этого примера: используйте фактическую настройку связи в проекте.

Для загрузки коллекций связанных объектов при выводе списка помещений с их бронированиями подходит `prefetch_related()`. Например, при наличии `related_name="bookings"`:

```
# Загружаем помещения и их бронирования отдельными запросами,
# после чего Django связывает результаты в памяти.
rooms = Room.objects.prefetch_related("bookings")

for room in rooms:
    print(room.name)

    for booking in room.bookings.all():
        print(booking.booking_date)
```

Этот пример нужен для понимания различия между двумя механизмами. Не внедряйте его в приложение без конкретной задачи по отображению бронирований у помещений.

Типичные ошибки

1. Указано несуществующее имя поля в `select_related()`.

Проверьте, что `room` и `user` – реальные поля внешних ключей в модели `Booking`. Названия приложений `rooms` и `users` не заменяют имена полей.

2. Используется `select_related()` для коллекции бронирований помещения.

Для обратной связи «одно помещение – много бронирований» нужен `prefetch_related()`, а не `select_related()`.

3. Изменение запроса нарушило фильтрацию или пагинацию.

Не заменяйте всё представление примером из методички. Добавьте оптимизацию к существующему `QuerySet`, сохранив прежние условия и порядок операций.

4. Количество запросов не уменьшилось.

Убедитесь, что сравниваются одинаковые наборы данных, связанные объекты действительно читаются внутри проверяемого блока, а запросы не выполняются до начала перехвата.

5. Оптимизация добавлена, но страница не использует соответствующий `QuerySet`.

Проверьте, что именно оптимизированный `QuerySet` передаётся в контекст шаблона и используется при формировании списка.

5.3.6. Использование F(), annotate() и values() для формирования списка бронирований

При работе с Django ORM важно не только отбирать записи, но и получать данные в форме, необходимой для конкретной страницы приложения. Если для отображения списка нужны лишь отдельные поля бронирования, помещения и пользователя, необязательно загружать полные объекты всех связанных моделей.

В Roomly этот подход уже используется в представлении `booking_list()`. В данном пункте необходимо разобраться в его устройстве, проверить

сформированный SQL-запрос и убедиться, что фильтрация, сортировка и пагинация работают корректно.

Шаг 1. Анализ существующего представления

Откройте файл `bookings/views.py` и найдите функцию `booking_list()`.

В начале представления выбираются только активные бронирования:

```
# Выбираем только активные бронирования.
bookings_data = Booking.objects.filter(
    is_cancelled=False
)
```

Далее к `QuerySet` последовательно применяются фильтры по дате, помещению и пользователю. Условия добавляются только при наличии соответствующих параметров запроса.

Например:

```
# Фильтруем бронирования по выбранному помещению.
if room_id.isdigit():
    bookings_data = bookings_data.filter(
        room_id=int(room_id)
    )
```

Такой подход позволяет формировать запрос постепенно, не загружая записи в Python после каждого условия.

После применения фильтров выполняется аннотирование:

```
# Добавляем сведения о связанных объектах.
bookings_data = bookings_data.annotate(
    room_name=F("room__name"),
    capacity=F("room__capacity"),
    user_name=F("user__name"),
)
```

Здесь используются `F()` и `annotate()`. Они позволяют включить значения полей связанных объектов в результирующий набор данных.

Шаг 2. Изучение выражений `F()`

Выражение `F()` ссылается на поле модели на стороне базы данных. Оно не извлекает значение поля в Python заранее, а позволяет использовать его в выражении ORM.

В Roomly применяются следующие выражения:

Выражение	Получаемое значение
`F("room__name")`	Название помещения, связанного с бронированием
`F("room__capacity")`	Вместимость связанного помещения
`F("user__name")`	Имя пользователя, связанного с бронированием

Двойное подчёркивание `__` обозначает переход по связи модели. Например, `room__name` означает поле `name` модели помещения, связанной с бронированием через поле `room`.

Само по себе использование `F()` не гарантирует уменьшения количества SQL-запросов. В данном случае оно позволяет сослаться на нужные поля в выражении ORM. Выигрыш от отсутствия дополнительных запросов появляется благодаря тому, что необходимые значения выбираются в составе одного SQL-запроса, а не извлекаются последовательными обращениями к связанным объектам.

Шаг 3. Изучение `annotate()`

Метод `annotate()` добавляет к каждой записи `QuerySet` дополнительные значения. В данном случае это `room_name`, `capacity` и `user_name`.

Важно различать два варианта:

```
# Получаем объекты модели Booking.
bookings = Booking.objects.all()
```

и:

```
# Добавляем к бронированиям дополнительные значения.
bookings = Booking.objects.annotate(
    room_name=F("room__name"),
)
```

Во втором случае результат по-прежнему основан на записях `Booking`, но каждая запись также получает дополнительное значение `room_name`. До вызова `values()` это не словарь, а объект модели с аннотированным атрибутом.

В Roomly аннотации позволяют включить в результат поля связанных объектов без необходимости обращаться к ним в шаблоне через `booking.room.name` и `booking.user.name`.

Шаг 4. Формирование набора полей с помощью `values()`

После аннотирования в представлении выполняется:

```
# Формируем набор полей для шаблона.
bookings_data = bookings_data.values(
    "id",
    "booking_date",
    "room_name",
    "capacity",
    "user_name",
```

```
)
```

Метод `values()` ограничивает результат перечисленными полями и возвращает записи в виде словарей. Таким образом, вместо полных объектов `Booking` представление получает данные, необходимые для отображения списка:

```
{
    "id": 1,
    "booking_date": ...,
    "room_name": "Аудитория 301",
    "capacity": 30,
    "user_name": "Иван Петров",
}
```

Это иллюстративный пример структуры результата; фактические значения зависят от данных базы.

В шаблоне `bookings/booking_list.html` такие записи используются через ключи словаря:

```
{# Идентификатор бронирования. #}
{{ booking.id }}

{# Дата бронирования. #}
{{ booking.booking_date }}

{# Название связанного помещения. #}
{{ booking.room_name }}

{# Вместимость помещения. #}
{{ booking.capacity }}

{# Имя пользователя. #}
{{ booking.user_name }}
```

Не заменяйте эти обращения на `booking.room.name` или `booking.user.name`: после вызова `values()` элементы результата являются словарями, а не экземплярами модели `Booking`.

Шаг 5. Проверка сформированного SQL-запроса

Чтобы увидеть SQL, соответствующий `QuerySet`, можно выполнить проверку в Django `Shell`. Откройте терминал в каталоге проекта и запустите:

```
python manage.py shell
```

Выполните:

```
# Импортируем модели и выражение F.
from bookings.models import Booking
from django.db.models import F
```

```
# Формируем QuerySet по аналогии с представлением Roomly.
bookings_data = (
    Booking.objects
    .filter(is_cancelled=False)
    .annotate(
        room_name=F("room__name"),
        capacity=F("room__capacity"),
        user_name=F("user__name"),
    )
    .values(
        "id",
        "booking_date",
        "room_name",
        "capacity",
        "user_name",
    )
)

# Выводим SQL-запрос, сформированный ORM.
print(bookings_data.query)
```

В сформированном SQL должны быть видны обращения к таблице бронирований и соединения с таблицами, из которых берутся поля помещения и пользователя. Конкретные имена таблиц и синтаксис зависят от конфигурации проекта и используемой СУБД.

Обратите внимание: обращение к `bookings_data.query` показывает построенный SQL, но само по себе не выполняет запрос. Для выполнения запроса и просмотра результата используйте:

```
# Выполняем запрос и получаем первые пять словарей.
for booking in bookings_data[:5]:
    print(booking)
```

Шаг 6. Проверка количества SQL-запросов

Для измерения числа запросов можно воспользоваться инструментом Django `CaptureQueriesContext`.

```
# Импортируем инструменты для подсчёта запросов.
from django.db import connection
from django.test.utils import CaptureQueriesContext

# Перехватываем SQL-запросы при выполнении QuerySet.
with CaptureQueriesContext(connection) as queries:
    result = list(bookings_data[:5])
```

```
# Выводим количество выполненных запросов.
print("Количество SQL-запросов:", len(queries))

# При необходимости изучаем сами запросы.
for query in queries.captured_queries:
    print(query["sql"])
```

Здесь `list()` заставляет Django выполнить запрос и загрузить результат в память. Срез ограничивает выборку пятью записями.

Этот пример показывает количество запросов, необходимое для выполнения конкретной выборки. Он не измеряет все запросы страницы Roomly: при открытии списка отдельно могут запрашиваться помещения и пользователи для фильтров, а механизм пагинации может выполнять запрос для подсчёта общего числа записей.

Если в проекте установлена Django Debug Toolbar, можно дополнительно проверить SQL-запросы при открытии страницы списка бронирований. Установку панели рассмотрим отдельно.

Шаг 7. Проверка сортировки и пагинации

В представлении сортировка выполняется до вызова `values()`:

```
# Сопоставляем параметр URL с разрешёнными полями.
sort_fields = {
    "date": "booking_date",
    "room": "room__name",
    "user": "user__name",
}

# Выбираем поле сортировки.
sort_field = sort_fields.get(sort, "booking_date")

# Сортируем по выбранному полю и идентификатору.
bookings_data = bookings_data.order_by(
    sort_field,
    "id",
)
```

Такой словарь ограничивает набор полей, по которым пользователь может сортировать список. Не следует напрямую передавать произвольное значение из URL в `order_by()`.

После выбора нужных полей выполняется пагинация:

```
# Разбиваем список на страницы по пять записей.
paginator = Paginator(bookings_data, 5)

# Получаем номер страницы из GET-параметра.
page_number = request.GET.get("page")
```

```
# Выбираем страницу.
page_obj = paginator.get_page(page_number)
```

Поскольку `QuerySet` вычисляется лениво, Django может выполнить запрос только тогда, когда понадобятся данные для отображения страницы. При пагинации также может понадобиться отдельный запрос для определения общего количества записей.

Не меняйте порядок операций без необходимости: сначала должны применяться фильтры и сортировка, затем формироваться набор полей и выполняться пагинация.

Шаг 8. Проверка в приложении Roomly

Запустите сервер разработки:

```
python manage.py runserver
```

Откройте существующую страницу списка бронирований и проверьте:

- отображаются только активные бронирования;
- фильтр по дате работает корректно;
- фильтры по помещению и пользователю применяются;
- сортировка по дате, помещению и пользователю работает;
- на странице отображается не более пяти записей;
- переход между страницами не сбрасывает остальные параметры фильтрации и сортировки;
- название помещения, вместимость и имя пользователя соответствуют конкретному бронированию;
- сообщение об ошибке даты отображается в предусмотренном проектом виде.

Если обнаружена проблема, исправляйте соответствующий участок существующего представления или шаблона, не заменяя всю функцию `booking_list()` учебным примером.

Типичные ошибки

1. Использование `booking.room.name` после вызова `values()`.

После `values()` результат содержит словари. Используйте ключ `booking.room_name`, который создаётся в аннотации.

2. Ожидание, что `F()` автоматически уменьшает число запросов.

`F()` позволяет ссылаться на поле в выражении ORM. Количество запросов зависит от того, как сформирован и используется `QuerySet`.

3. Отсутствие нужных полей в `values()`.

Если шаблону требуется поле, которого нет в выбранном наборе, добавьте его в `values()` и убедитесь, что оно существует в модели или было создано через `annotate()`.

4. Потеря фильтров при добавлении аннотаций.

`annotate()` и `values()` должны применяться к `QuerySet`, в котором уже сформированы необходимые условия. Не заменяйте его новым запросом без переноса фильтров.

5.3.7. Создание пользовательского менеджера записей (Manager)

В Django ORM запросы к модели обычно начинаются с менеджера `objects`:

```
# Получаем все бронирования.
Booking.objects.all()

# Получаем только активные бронирования.
Booking.objects.filter(is_cancelled=False)
```

Если одни и те же условия используются в нескольких представлениях, их приходится повторять. Например, в Roomly активные бронирования выбираются при формировании списка бронирований, а в дальнейшем такое условие может понадобиться при проверке доступности помещения или построении статистики.

Чтобы не дублировать запросы, можно создать пользовательский менеджер и набор методов `QuerySet`. Это позволит обращаться к повторяющимся условиям через понятные методы:

```
# Получаем активные бронирования.
Booking.objects.active()

# Получаем отменённые бронирования.
Booking.objects.cancelled()

# Получаем бронирования за определённую дату.
Booking.objects.for_date(selected_date)
```

В этом пункте необходимо реализовать пользовательский менеджер для модели `Booking` и применить его в существующем представлении Roomly.

Шаг 1. Разберитесь, за что отвечают `Manager` и `QuerySet`

Менеджер (`Manager`) – объект, через который приложение начинает получать записи модели. По умолчанию Django предоставляет менеджер `objects`.

`QuerySet` – набор записей, к которому можно последовательно применять фильтрацию, сортировку, аннотации и другие операции ORM. Например:

```
# objects - менеджер модели Booking.
# filter() возвращает QuerySet с подходящими записями.
```

```
bookings = Booking.objects.filter(
is_cancelled=False
)
# Дополнительно фильтруем полученный QuerySet.
bookings = bookings.filter(
room_id=1
)
```

Пользовательский менеджер позволяет добавлять методы для часто используемых запросов. Однако если требуется, чтобы новые методы можно было также применять к уже сформированному `QuerySet`, удобно использовать собственный класс `QuerySet` и связанный с ним менеджер.

Шаг 2. Создайте пользовательский `QuerySet`

Откройте файл:

```
bookings/models.py
```

Перед классом `Booking` добавьте класс `BookingQuerySet`:

```
from django.db import models


class BookingQuerySet(models.QuerySet):
    """Набор запросов для работы с бронированиями."""

    def active(self):
        """Возвращает только активные бронирования."""
        return self.filter(is_cancelled=False)

    def cancelled(self):
        """Возвращает только отменённые бронирования."""
        return self.filter(is_cancelled=True)

    def for_date(self, selected_date):
        """Возвращает бронирования за указанную дату."""
        return self.filter(booking_date=selected_date)
```

Если в начале файла уже есть импорт `models` из `django.db`, не добавляйте его повторно.

Методы используют существующие поля модели `Booking`:

- `is_cancelled` – признак отмены;
- `booking_date` – дата бронирования.

Метод `for_date()` получает дату параметром. В зависимости от фактического типа поля `booking_date` передаваемое значение должно соответствовать типу,

который ожидает модель: например, для `DateField` обычно используется объект `datetime.date`.

Каждый метод возвращает `QuerySet`, а не список и не отдельный объект. Поэтому запрос можно продолжить:

```
# Активные бронирования для конкретного помещения.
bookings = (
    Booking.objects
    .active()
    .filter(room_id=1)
)
```

`QuerySet` по-прежнему вычисляется лениво: обращение к базе данных произойдёт, когда результат понадобится.

Шаг 3. Создайте менеджер на основе `QuerySet`

После класса BookingQuerySet добавьте:

```
class BookingManager(
    models.Manager.from_queryset(BookingQuerySet)
):
    """Менеджер модели Booking с дополнительными методами."""
    pass
```

Конструкция `models.Manager.from_queryset(BookingQuerySet)` создаёт менеджер, который предоставляет методы пользовательского `QuerySet`.

Это позволяет вызывать `active()`, `cancelled()` и `for_date()` непосредственно через менеджер `Booking.objects`.

Шаг 4. Подключите менеджер к модели `Booking`

Найдите существующий класс `Booking` в `bookings/models.py`.

Внутри класса добавьте атрибут:

```
class Booking(models.Model):
    # Здесь остаются существующие поля модели.

 # Подключение пользовательского менеджера записей к модели Booking
    objects = BookingManager()

    # Здесь остаются существующие методы модели.
```

Это фрагмент для интеграции, а не замена всей модели. Сохраните все существующие поля, ограничения, методы, связи и настройки `Meta`.

Если у модели уже объявлен менеджер `objects`, замените именно его объявление на `objects = BookingManager()`, предварительно проверив, не содержит ли

существующий менеджер важную дополнительную логику. Не создавайте второе объявление `objects` в том же классе.

После подключения пользовательского менеджера стандартные запросы по-прежнему доступны:

```
# Все бронирования, включая отменённые.
Booking.objects.all()

# Только активные бронирования.
Booking.objects.active()

# Только отменённые бронирования.
Booking.objects.cancelled()
```

Важно, что мы не делаем активные бронирования единственным набором записей по умолчанию. Обычный вызов `Booking.objects.all()` должен сохранять доступ к истории, включая отменённые бронирования.

Шаг 5. Проверьте работу менеджера в Django `Shell`

Запустите оболочку Django:

```
python manage.py shell
```

Импортируйте модель:

```
from bookings.models import Booking
```

Проверьте общее количество бронирований:

```
# Все записи, включая отменённые.
print(Booking.objects.count())
```

Затем получите количество активных и отменённых бронирований:

```
# Количество активных бронирований.
print(Booking.objects.active().count())

# Количество отменённых бронирований.
print(Booking.objects.cancelled().count())
```

Сравните результаты. Сумма количества активных и отменённых записей должна совпадать с общим количеством бронирований, если поле `is_cancelled` заполнено корректно и используется как булев признак.

Проверьте фильтрацию по дате:

```
# Получаем дату из стандартного модуля Python.
from datetime import date

# Укажите дату, для которой в базе есть бронирования.
selected_date = date(2026, 10, 25)
```

```
# Получаем бронирования за указанную дату.
bookings = Booking.objects.for_date(selected_date)

# Выводим найденные записи.
for booking in bookings:
    print(
        booking.id,
        booking.booking_date,
        booking.is_cancelled,
    )
```

Дата в примере приведена для иллюстрации. При проверке используйте дату, которая действительно присутствует в вашей базе данных. Если `booking_date` имеет тип `DateField`, объект `date` подходит для такого фильтра.

Проверьте совместное применение методов:

```
# Получаем активные бронирования за выбранную дату.
bookings = (
    Booking.objects
    .active()
    .for_date(selected_date)
)

for booking in bookings:
    print(booking.id, booking.booking_date)
```

Этот запрос показывает преимущество собственного `QuerySet`: методы можно комбинировать с обычными фильтрами Django ORM.

Шаг 6. Примените менеджер в представлении Roomly

Теперь перенесите новый механизм в приложение.

1. Откройте bookings/views.py и найдите функцию `booking_list()`.

В начале представления сейчас используется:

```
# Выбираем только активные бронирования.
bookings_data = Booking.objects.filter(
    is_cancelled=False
)
```

2. Замените только этот участок:

```
# Выбираем только активные бронирования через пользовательский QuerySet.
bookings_data = Booking.objects.active()
```

Остальную часть представления оставьте без изменений:

- фильтрацию по дате;

- фильтрацию по помещению;
- фильтрацию по пользователю;
- `annotate()` с выражениями `F()`;
- сортировку;

```
values();
```

- пагинацию;
- формирование контекста и вывод шаблона.

Такой переход показывает, как ORM-операция, ранее записанная непосредственно в представлении, переносится в модельный слой без изменения поведения страницы.

В результате представление явно указывает, какие записи ему нужны, а правило отбора активных бронирований хранится в одном месте.

Шаг 7. Проверьте приложение

Запустите сервер:

```
python manage.py runserver
```

Откройте страницу списка бронирований Roomly.

Проверьте, что:

- отображаются активные бронирования;
- фильтры по дате, помещению и пользователю работают;
- сортировка и пагинация не изменились;
- отменённые бронирования по-прежнему сохраняются в базе данных;
- другие представления и административная панель продолжают работать.

Отдельно проверьте, что вызов `Booking.objects.all()` по-прежнему возвращает и активные, и отменённые записи. Это важно для сохранения истории бронирований и построения статистики.

Шаг 8. Нужна ли миграция?

Изменение менеджера обычно не меняет структуру таблиц базы данных. Добавление `BookingQuerySet` и `BookingManager` не требует миграции, если при этом не изменялись поля или другие параметры схемы.

Для проверки можно выполнить:

```
python manage.py makemigrations --check
```

Если Django сообщает, что изменений моделей нет, создавать миграцию для этого изменения не требуется. Если команда сообщает об обнаруженных изменениях, проверьте их причину, прежде чем создавать миграцию.

Типичные ошибки

1. Метод `active()` не найден.

Проверьте, что `BookingManager` построен на `BookingQuerySet`, а в модели подключён именно `objects = BookingManager()`.

2. Исчезли отменённые бронирования из других частей приложения.

Проверьте, что фильтр активности не перенесён в глобальный `get_queryset()` менеджера. В этой реализации `objects.all()` должен возвращать все записи, а фильтрация выполняется только при явном вызове `.active()`.

3. После изменения модели перестала работать часть существующих запросов.

Проверьте, не содержал ли старый менеджер дополнительные методы или переопределённый `get_queryset()`. Сохраните необходимую функциональность при объединении реализаций.

4. Метод `for_date()` не возвращает ожидаемые записи.

Проверьте фактический тип поля `booking_date` и тип передаваемого значения. Для `DateField` используйте объект `datetime.date`, а не произвольную строку.

5.3.8. Выражения F() для вычислений на стороне базы данных

Выражения `F()` позволяют обращаться к значениям полей непосредственно в запросе к базе данных. В предыдущих заданиях Roomly они использовались для получения названия помещения, его вместимости и имени пользователя. Теперь рассмотрим другой сценарий: вычисления и обновление значений без предварительной загрузки объектов в Python.

Это особенно полезно, когда операция должна выполняться непосредственно в базе данных, например при увеличении числового значения на определённую величину или сравнении двух полей одной модели.

Шаг 1. Сравните обычное обновление и выражение `F()`

Предположим, что требуется увеличить числовое значение поля на 5.

При обычном подходе значение сначала читается из базы данных, затем изменяется в Python и записывается обратно:

```
# Получаем объект помещения.
room = Room.objects.get(id=1)

# Изменяем значение в Python.
room.capacity += 5

# Сохраняем новое значение в базе данных.
room.save()
```

В этом примере выполняются получение объекта и последующее сохранение изменений.

Для числового поля можно использовать выражение `F()`:

```
from django.db.models import F
# Увеличиваем вместимость помещения непосредственно в базе данных.
Room.objects.filter(id=1).update(
    capacity=F("capacity") + 5
)
```

Вместо чтения текущего значения в Python Django формирует SQL-операцию, которая использует существующее значение поля в базе данных.

Условно операция соответствует выражению:

```
UPDATE rooms_room
SET capacity = capacity + 5
WHERE id = 1;
```

Это иллюстративная запись SQL: реальное имя таблицы зависит от конфигурации модели. Преимущество такого подхода состоит в том, что вычисление выполняется на стороне базы данных. Кроме того, при конкурентном изменении записи выражение использует актуальное значение поля на момент выполнения SQL-операции, что помогает избежать типичной проблемы обновлений по схеме «прочитать – изменить – записать».

Примечание: этот пример демонстрирует механизм, но не является указанием увеличивать вместимость помещений на 5. В предметной области вместимость должна задаваться на основании реальных данных, а не изменяться произвольно. Поэтому выполнять приведённый `update()` на рабочих данных проекта не нужно.

Шаг 2. Изучите использование `F()` в вычисляемых выражениях

Выражение `F()` можно использовать не только в `update()`, но и в `annotate()`.

Например:

```
from django.db.models import F

# Получаем помещения и добавляем вычисляемое значение.
rooms = Room.objects.annotate(
    capacity_plus_five=F("capacity") + 5
)

# Выводим исходную вместимость и вычисленное значение.
for room in rooms:
    print(
        room.name,
        room.capacity,
        room.capacity_plus_five,
    )
```

Для каждого помещения база данных вычислит дополнительное значение, равное исходной вместимости плюс пять. При этом поле `capacity` в самой базе данных не изменится.

Результат аннотации доступен как дополнительный атрибут объекта модели. Однако `capacity_plus_five` не становится постоянным полем таблицы: оно вычисляется в рамках запроса.

Это различие принципиально:

- `annotate()` добавляет вычисляемое значение в результат запроса;
- `update()` изменяет данные в базе;
- `F()` позволяет использовать значение поля непосредственно в выражении ORM.

Не добавляйте `capacity_plus_five` в модель как обычное поле: для данного примера это не требуется.

Шаг 3. Используйте `F()` для сравнения полей

Ещё одна возможность – сравнивать значения разных полей одной модели.

Например, если модель содержит поля `capacity` и `minimum_capacity`, можно выбрать записи, у которых вместимость больше минимального значения:

```
# Пример только для модели, в которой существуют оба поля.
from django.db.models import F

rooms = Room.objects.filter(
capacity__gt=F("minimum_capacity")
)
```

Здесь `F("minimum_capacity")` означает, что Django должен сравнить `capacity` со значением другого поля, а не с фиксированным числом.

В текущей модели Roomly такого сравнения применять не нужно, если поля `minimum_capacity` нет. Не добавляйте поле только ради демонстрации выражения. Этот пример показывает принцип, который пригодится при появлении реального требования предметной области.

Шаг 4. Проверьте выражения `F()` в Django `Shell`

Откройте оболочку Django:

```
python manage.py shell
```

Импортируйте модель и необходимые инструменты:

```
# Импортируем модель помещения.
from rooms.models import Room

# Импортируем выражение F.
from django.db.models import F
```

Выполните безопасный пример с вычисляемым значением:

```
# Получаем помещения с дополнительным вычисляемым значением.
rooms = Room.objects.annotate(
    capacity_plus_five=F("capacity") + 5
)

# Выводим исходную вместимость и результат вычисления.
for room in rooms:
    print(
        room.name,
        room.capacity,
        room.capacity_plus_five,
    )
```

Затем изучите SQL:

```
# Показываем SQL, сформированный Django ORM.
print(rooms.query)
```

Убедитесь, что в SQL присутствует выражение, использующее значение поля `capacity` и прибавляющее к нему 5.

Поскольку использован `annotate()`, а не `update()`, данные в таблице помещений не изменяются.

Для дополнительной проверки можно выполнить:

```
# Получаем помещение и его исходную вместимость.
room = Room.objects.first()

if room is not None:
    original_capacity = room.capacity

    # Получаем вычисленное значение отдельным запросом.
    calculated_capacity = (
        Room.objects
        .filter(pk=room.pk)
        .annotate(
            capacity_plus_five=F("capacity") + 5
        )
        .values_list("capacity_plus_five", flat=True)
        .first()
    )

    print("Исходная вместимость:", original_capacity)
    print("Вычисленное значение:", calculated_capacity)

    # Проверяем, что исходное значение не изменилось.
```

```
    room.refresh_from_db()
    print("Вместимость в базе:", room.capacity)
```

Если в базе есть хотя бы одно помещение, вычисленное значение должно быть на 5 больше исходного, а сохранённая вместимость должна остаться прежней.

Шаг 5. Определите, где `F()` пригодится в Roomly

В текущей модели Roomly выражения `F()` уже используются в `booking_list()` для формирования набора данных:

```
# Добавляем значения полей связанных объектов.
bookings_data = bookings_data.annotate(
    room_name=F("room__name"),
    capacity=F("room__capacity"),
    user_name=F("user__name"),
)
```

Это корректное применение `F()` в составе ORM-запроса. Оно позволяет выбрать нужные значения связанных объектов и затем сформировать словари с помощью `values()`.

А вот для арифметических обновлений в текущей модели пока нет очевидной необходимости. В частности:

- вместимость помещения не следует увеличивать автоматически;
- отмена бронирования должна выполняться предусмотренным методом, например `booking.cancel()`, а не переключением `is_cancelled` с помощью `~F("is_cancelled")`;
- новые поля не нужно добавлять только для демонстрации сравнения значений.

Таким образом, в этом пункте мы изучаем дополнительные возможности `F()`, но не меняем существующее представление `booking_list()`. Уже реализованное использование выражений остаётся без изменений.

Типичные ошибки

1. Ожидание, что `F()` возвращает обычное значение Python.

`F("capacity")` – выражение ORM, а не само числовое значение. Оно используется для построения запроса к базе данных.

2. Использование `F()` без существующего поля.

В выражении `F("minimum_capacity")` должно указываться реальное поле модели. Несуществующие поля приведут к ошибке.

3. Случайное изменение данных через `update()`.

Метод `update()` непосредственно изменяет записи в базе данных. Перед его использованием необходимо понимать, какие объекты будут затронуты и соответствует ли операция бизнес-правилам.

5.3.9. Работа со связанными объектами и оптимизация запросов

Шаг 1. Подготовка Django `Shell`

Откройте терминал в каталоге проекта Roomly, активируйте виртуальное окружение, если оно ещё не активировано, и выполните:

```
python manage.py shell
```

Импортируйте модели и инструменты для анализа запросов:

```
from django.db import connection
from django.test.utils import CaptureQueriesContext

from rooms.models import Room
from bookings.models import Booking
```

Если в проекте используется другая структура каталогов приложений, скорректируйте импорты в соответствии с фактическим расположением моделей.

Шаг 2. Загрузка связанных объектов с помощью `select_related()`

В Roomly объект `Booking` связан с помещением `Room` и пользователем `User`. При получении бронирований может потребоваться вывести название помещения и имя пользователя.

Сначала выполните запрос без предварительной загрузки связанных объектов:

```
with CaptureQueriesContext(connection) as queries:
    bookings = list(Booking.objects.all()[:5])

    for booking in bookings:
        # Обращение к связанным объектам может вызвать дополнительные
запросы.
        room_name = booking.room.name
        user_name = booking.user.name

print("Количество SQL-запросов:", len(queries))
```

Запишите полученное количество запросов.

Теперь выполните аналогичную операцию с `select_related()`:

```
with CaptureQueriesContext(connection) as queries:
    bookings = list(
        Booking.objects.select_related(
            "room",
            "user",
        )[:5]
    )

    for booking in bookings:
        # Связанные объекты уже загружены вместе с бронированиями.
```

```
        room_name = booking.room.name
        user_name = booking.user.name

print("Количество SQL-запросов:", len(queries))
```

Сравните результаты.

select_related() позволяет получить бронирования и связанные с ними объекты в одном SQL-запросе. Для этого Django использует SQL JOIN.

Метод подходит для связей ForeignKey и OneToOneField, когда требуется загрузить один связанный объект для каждой основной записи.

Количество запросов в конкретном эксперименте зависит от данных и выполняемых операций. Основная задача – сравнить результаты двух вариантов, а не получить заранее заданное число.

Шаг 3. Определение обратной связи между помещением и бронированиями

Теперь рассмотрите обратную связь: от помещения Room к связанным с ним бронированиям Booking.

В модели Booking поле room ссылается на Room. Обратное имя связи определяется параметром related_name этого поля. Если related_name не задан, Django обычно использует имя вида booking_set.

Чтобы не угадывать имя связи, выполните:

```
# Находим обратную связь от Room к Booking.
room_booking_relation = next(
    field
    for field in Room._meta.get_fields()
    if field.one_to_many
    and field.related_model == Booking
)

# Получаем фактическое имя обратной связи.
booking_accessor = room_booking_relation.get_accessor_name()

print("Имя обратной связи:", booking_accessor)
```

Полученное имя понадобится в следующем запросе.

Шаг 4. Загрузка связанных коллекций с помощью prefetch_related()

Получим несколько помещений и обратимся к бронированиям каждого из них.

Сначала выполним запрос без предварительной загрузки обратной связи:

```
with CaptureQueriesContext(connection) as queries:
    rooms = list(Room.objects.all()[:5])

    for room in rooms:
```

```
        # Получаем бронирования конкретного помещения.
        room_bookings = list(
            getattr(room, booking_accessor).all()
        )

print("Количество SQL-запросов:", len(queries))
```

Теперь используем `prefetch_related()`:

```
with CaptureQueriesContext(connection) as queries:
    rooms = list(
        Room.objects.prefetch_related(
            booking_accessor
        )[:5]
    )

    for room in rooms:
        # Бронирования берутся из предварительно загруженных данных.
        room_bookings = list(
            getattr(room, booking_accessor).all()
        )

print("Количество SQL-запросов:", len(queries))
```

Сравните результаты: `prefetch_related()` предназначен для предварительной загрузки связанных коллекций, например бронирований каждого помещения. Обычно Django выполняет отдельный запрос для основных объектов и дополнительный запрос для связанных записей, а затем связывает результаты в Python.

Это позволяет избежать ситуации, когда для каждого помещения выполняется отдельный SQL-запрос к таблице бронирований.

Шаг 5. Сравнение методов

Зафиксируйте различия между двумя способами загрузки связанных данных.

Критерий	**select_related()**	**prefetch_related()**
Типичный случай применения	Загрузка одного связанного объекта	Загрузка коллекции связанных объектов
Основные типы связей	ForeignKey, OneToOneField	Обратные связи ForeignKey, ManyToManyField
Способ получения данных	SQL JOIN	Отдельные запросы с последующим объединением результатов в Python
Пример в Roomly	Бронирование вместе с помещением и пользователем	Помещения вместе с их бронированиями

Оба метода позволяют уменьшить количество запросов, но применяются к разным типам связей.

Шаг 6. Проверка существующего кода Roomly

Откройте `bookings/views.py` и найдите представление `booking_list()`.

В нём сведения о связанных объектах уже выбираются с помощью `annotate()` и `values()`:

```
bookings_data = bookings_data.annotate(
    room_name=F("room__name"),
    capacity=F("room__capacity"),
    user_name=F("user__name"),
)

bookings_data = bookings_data.values(
    "id",
    "booking_date",
    "room_name",
    "capacity",
    "user_name",
)
```

Этот код не нужно заменять на `select_related()`: ORM формирует запрос, который получает нужные значения связанных полей через SQL `JOIN`.

Не добавляйте `select_related()` в это представление только ради выполнения упражнения. Оптимизация должна учитывать фактическую структуру запроса и способ использования полученных данных.

Если в другом представлении Roomly сначала загружается коллекция помещений, а затем для каждого помещения отдельно запрашиваются бронирования, там может быть уместен `prefetch_related()`. Перед внесением изменений необходимо проверить реальный код этого представления и убедиться, что предварительная загрузка действительно соответствует выполняемым операциям.

5.3.10. Анализ SQL-запросов Django ORM

Шаг 1. Запуск Django `Shell`

Откройте терминал в каталоге проекта Roomly и выполните:

```
python manage.py shell
```

Импортируйте необходимые модели и инструменты:

```
from django.db import connection
from django.test.utils import CaptureQueriesContext

from rooms.models import Room
from bookings.models import Booking
```

Шаг 2. Просмотр SQL-запроса без его выполнения

Сформируйте запрос для получения помещений, отсортированных по вместимости:

```
rooms_query = Room.objects.order_by("-capacity", "name")
```

Посмотрите SQL, который сформировал Django:

```
print(rooms_query.query)
```

Обратите внимание на следующие элементы:

- `SELECT` – какие поля выбираются;
- `FROM` – из какой таблицы извлекаются данные;
- `ORDER BY` – по каким полям выполняется сортировка.

Само создание `QuerySet` обычно не приводит к обращению к базе данных. Обращение происходит при вычислении результата, например при переборе `QuerySet` или преобразовании его в список.

Шаг 3. Сравнение фильтрации и сортировки

Создайте два запроса:

```
# Выбираем помещения вместимостью не менее 30 человек.
query_filtered = Room.objects.filter(
    capacity__gte=30
)

# Выбираем те же помещения и сортируем по названию.
query_sorted = Room.objects.filter(
    capacity__gte=30
).order_by("name")
```

Выведите SQL обоих запросов:

```
print("Запрос с фильтрацией:")
print(query_filtered.query)

print("\nЗапрос с фильтрацией и сортировкой:")
print(query_sorted.query)
```

Сравните запросы.

В первом случае условие фильтрации отражается в SQL через `WHERE`. Во втором дополнительно появляется `ORDER BY`.

Фактические имена таблиц и полей зависят от настроек проекта и определений моделей.

Шаг 4. Анализ запроса с аннотацией

Ранее в Roomly использовался `F()` для получения дополнительных вычисляемых значений. Рассмотрим аналогичный пример:

```
from django.db.models import F

rooms_with_extra_capacity = Room.objects.annotate(
    capacity_plus_five=F("capacity") + 5
)
```

Посмотрите SQL:

```
print(rooms_with_extra_capacity.query)
```

В запросе должно быть видно выражение, вычисляющее значение на основе `capacity`.

Получите несколько результатов:

```
for room in rooms_with_extra_capacity[:5]:
    print(
        room.name,
        room.capacity,
        room.capacity_plus_five,
    )
```

Вычисляемое значение `capacity_plus_five` используется для текущего запроса. Оно не изменяет сохранённую вместимость помещения.

Шаг 5. Просмотр фактически выполненных запросов

Свойство `.query` показывает сформированный SQL для `QuerySet`, но само по себе не позволяет узнать, сколько запросов фактически выполнилось при обработке результата. Для этого используйте `CaptureQueriesContext`:

```
with CaptureQueriesContext(connection) as queries:
    rooms = list(
        Room.objects.filter(
            capacity__gte=30
        ).order_by("name")[:5]
    )

print("Количество SQL-запросов:", len(queries))

for query in queries:
    print(query["sql"])
```

Преобразование `QuerySet` в список приводит к выполнению запроса. После этого можно посмотреть фактически выполненный SQL.

При повторном обращении к уже вычисленному `QuerySet` Django может использовать его кэш результатов. Поэтому при сравнении разных вариантов создавайте отдельные `QuerySet` и выполняйте каждый эксперимент независимо.

Шаг 6. Получение плана выполнения запроса

Django ORM позволяет запросить план выполнения через `explain()`:

```
queryset = Room.objects.filter(
    capacity__gte=30
).order_by("name")
```

```
print(queryset.explain())
```

План выполнения показывает, как СУБД собирается выполнять запрос. В зависимости от базы данных в нём могут присутствовать сведения о способе доступа к таблице, использовании индексов, сортировке и других операциях.

Если СУБД поддерживает дополнительные параметры анализа, их применение зависит от используемой базы данных и версии Django. Не используйте параметры вроде `analyze=True` без предварительной проверки совместимости.

При интерпретации плана учитывайте размер таблиц и фактические данные. Само наличие полного сканирования таблицы не означает, что запрос обязательно нужно оптимизировать: для небольшой таблицы это может быть нормальным решением.

Шаг 7. Анализ существующего представления Roomly

Откройте `bookings/views.py` и найдите представление `booking_list()`.

Ранее в нём уже использовались:

- `filter()` для отбора бронирований;
- `annotate()` для добавления значений связанных полей;
- `order_by()` для сортировки;
- `values()` для выбора необходимых полей;
- пагинация для ограничения количества записей на странице.

Определите, какие SQL-операции должны соответствовать основным действиям представления:

1. Отбор активных бронирований.
2. Применение фильтров по дате, помещению и пользователю.
3. Сортировка.
4. Получение значений связанных объектов.
5. Ограничение количества результатов для пагинации.

При необходимости можно отдельно изучить SQL конкретного `QuerySet` в `Shell`. Если для анализа требуется увидеть запросы, которые выполняются при открытии страницы в браузере, это будет удобнее сделать в следующем тематическом блоке с помощью Django Debug Toolbar.

5.4. Дополнительные возможности Django в индивидуальном проекте

5.4.1. Подключение Django Debug Toolbar и анализ SQL-запросов

Шаг 1. Установка Django Debug Toolbar

Откройте терминал в каталоге проекта Roomly и активируйте виртуальное окружение.

Выполните команду:

```
python -m pip install django-debug-toolbar
```

Проверьте установку:

```
python -m pip show django-debug-toolbar
```

Если в проекте используется файл `requirements.txt`, добавьте в него зависимость `django-debug-toolbar` с установленной версией. Остальные зависимости удалять или заменять не нужно.

Шаг 2. Подключение приложения в настройках Django

Откройте файл roomly/settings.py.

Найдите список `INSTALLED_APPS` и добавьте `debug_toolbar`:

```
INSTALLED_APPS = [
    # Стандартные приложения Django.
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Приложения проекта.
    "homepage",
    "rooms",
    "users",
    "bookings",

    # Инструмент анализа запросов и отладки.
    "debug_toolbar",
]
```

Если в вашем файле список уже содержит другие приложения, сохраните их. Не заменяйте весь список приведённым примером: необходимо только добавить `debug_toolbar`.

Шаг 3. Подключение `middleware`

В том же файле `roomly/settings.py` найдите `MIDDLEWARE`.

Добавьте `debug_toolbar.middleware.DebugToolbarMiddleware` после `SecurityMiddleware` и перед остальными `middleware`:

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",

    # Панель отладки должна обрабатывать запросы и ответы.
    "debug_toolbar.middleware.DebugToolbarMiddleware",
```

```
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

Если в проекте уже настроены дополнительные `middleware`, сохраните их. Важно добавить новый компонент в правильное место, а не заменять весь список.

Шаг 4. Настройка доступа для локальной разработки

В roomly/settings.py найдите параметр `INTERNAL_IPS`. Если его нет, добавьте:

```
# Разрешаем отображение панели для локальной разработки.
INTERNAL_IPS = [
    "127.0.0.1",
]
```

Проверьте также значение `DEBUG`. Для локальной разработки оно обычно должно быть установлено в `True`.

Примечание: Django Debug Toolbar предназначен для разработки и отладки. Не включайте его в публичном производственном окружении: панель может показывать технические сведения о запросах и работе приложения.

Шаг 5. Подключение маршрутов панели отладки

Откройте корневой файл маршрутизации проекта `roomly/urls.py`.

Добавьте импорты:

```
from django.conf import settings
from django.urls import include, path
```

Сохраните существующие маршруты Roomly. В конец файла добавьте условное подключение панели:

```
# Подключаем маршруты панели только в режиме отладки.
if settings.DEBUG:
    urlpatterns += [
        path(
            "__debug__/",
            include("debug_toolbar.urls"),
        ),
    ]
```

В результате панель будет доступна только при включённом режиме отладки.

Схема подключения Django Debug Toolbar показана на рис. 5.

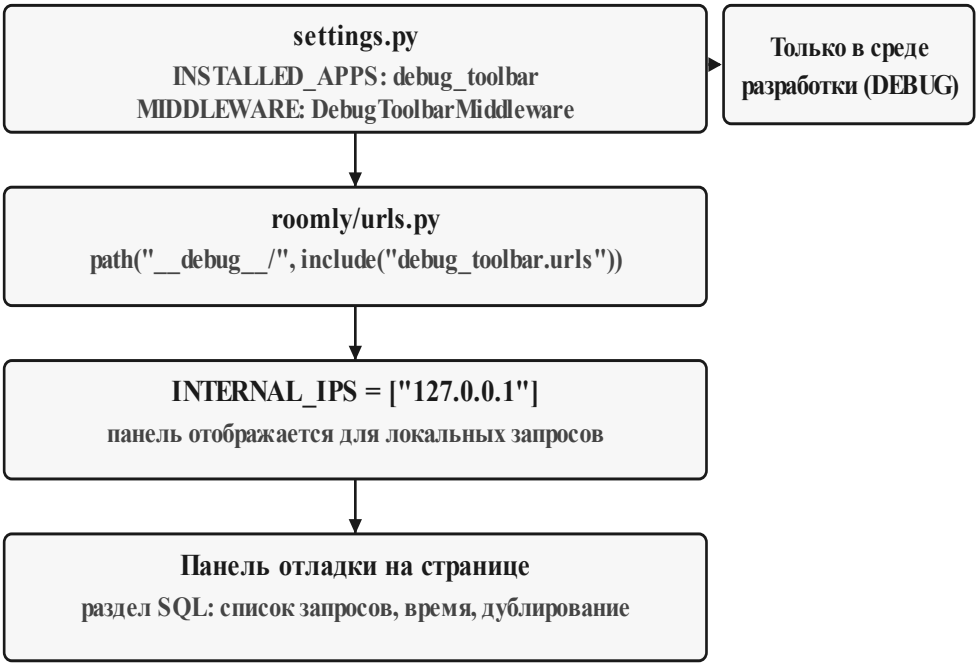


Рисунок 5 – Подключение Django Debug Toolbar

Если `urlpatterns` в вашем файле определён иначе, адаптируйте этот фрагмент к существующей структуре. Не удаляйте маршруты `homepage`, `rooms`, `bookings` и другие действующие маршруты проекта.

Шаг 6. Запуск и проверка приложения

В терминале выполните:

```
python manage.py check
```

Если проверка завершилась без ошибок, запустите сервер:

```
python manage.py runserver
```

Откройте в браузере страницу списка бронирований Roomly, например:

```
http://127.0.0.1:8000/bookings/
```

Если в проекте используется другой маршрут списка бронирований, откройте соответствующую страницу.

При успешном подключении Django Debug Toolbar на странице должна появиться панель отладки. Если панель не отображается, проверьте:

- установлен ли пакет `django-debug-toolbar`;
- добавлено ли приложение в `INSTALLED_APPS`;
- подключён ли `middleware`;
- добавлен ли маршрут `__debug__/`;
- включён ли `DEBUG`;
- открыта ли страница через локальный адрес, соответствующий `INTERNAL_IPS`.

Шаг 7. Анализ SQL-запросов

Откройте страницу списка бронирований и найдите в панели раздел SQL.

Изучите информацию, которую показывает панель:

- количество выполненных SQL-запросов;
- текст запросов;
- время выполнения, если оно доступно;
- повторяющиеся запросы;
- сведения о запросах, связанных с загрузкой объектов и обработкой страницы.

Особое внимание обратите на запросы к таблицам бронирований, помещений и пользователей. В bookings/views.py представление `booking_list()` уже использует `filter()`, `annotate()`, `order_by()` и `values()`. Поэтому на этом этапе необходимо не переписывать его, а посмотреть, какие запросы реально выполняются при открытии страницы.

Шаг 8. Проверка поведения при фильтрации

На странице списка бронирований выполните несколько действий:

1. Откройте страницу без дополнительных фильтров.
2. Примените фильтр по дате.
3. Примените фильтр по помещению.
4. Измените сортировку, если соответствующий элемент предусмотрен интерфейсом.
5. Перейдите на другую страницу результатов, если доступна пагинация.

После каждого действия изучите раздел SQL.

Сравните, какие запросы выполняются в разных случаях и как изменяются условия выборки. Обратите внимание, не появляются ли повторяющиеся запросы при отображении связанных объектов.

Не стремитесь обязательно уменьшить число запросов после каждого действия: применение фильтра или переход на другую страницу закономерно может приводить к выполнению нового запроса.

5.4.2. Оптимизация запросов Django ORM с использованием Django Debug Toolbar

При разработке Roomly необходимо учитывать не только корректность получаемых данных, но и количество SQL-запросов, которые Django выполняет при формировании страницы.

Например, список бронирований содержит сведения о помещении и пользователе. Эти данные хранятся в связанных таблицах, поэтому при работе с ORM важно правильно организовать получение связанных объектов.

В Roomly уже используются два подхода:

- в `booking_list()` связанные поля помещения и пользователя выбираются с помощью `annotate()` и `F()`;
- в `booking_detail()` применяется `select_related("room", "user")`.

В этом пункте необходимо изучить, как эти решения отражаются на SQL-запросах, проверить работу пагинации и справочных списков, а затем сравнить результаты через Django Debug Toolbar.

Шаг 1. Подготовка к исследованию

Убедитесь, что Django Debug Toolbar установлен и подключён к проекту в соответствии с подразделом «Подключение Django Debug Toolbar и анализ SQL-запросов».

Запустите сервер разработки:

```
python manage.py runserver
```

Откройте страницу списка бронирований Roomly. В зависимости от настроенных маршрутов это может быть адрес `/bookings/`.

В представлении `booking_list()` уже реализованы:

- выбор активных бронирований через `Booking.objects.active()`;
- фильтрация по дате, помещению и пользователю;
- сортировка по дате, помещению или пользователю;
- получение связанных полей через `annotate()` и `F()`;
- разбиение результата на страницы по пять записей;
- получение списков помещений и пользователей для фильтров.

Все эти операции необходимо сохранить. Задача исследования – выяснить, какие SQL-запросы выполняются при загрузке страницы и какие из них можно оптимизировать.

Шаг 2. Анализ запросов страницы списка бронирований

Откройте страницу списка бронирований и выберите раздел SQL в Django Debug Toolbar.

Изучите запросы, которые выполняются при формировании страницы. Особое внимание уделите следующим операциям:

1. Получение активных бронирований из таблицы `bookings`.
2. Соединение таблиц `bookings`, `rooms` и `users`.
3. Получение списка помещений для фильтра.
4. Получение списка пользователей для фильтра.
5. Подсчёт количества записей, необходимый для пагинации.

В вашем представлении связанные поля выбираются следующим образом:

```
# Добавляем сведения о связанных объектах.
bookings_data = bookings_data.annotate(
    room_name=F("room__name"),
    capacity=F("room__capacity"),
    user_name=F("user__name"),
)
```

Здесь `F()` обращается к полям связанных моделей через связи `room` и `user`. Django ORM формирует SQL-запрос, который получает необходимые значения из связанных таблиц. При обычной работе с такими полями отдельный запрос для каждого помещения или пользователя не требуется.

Поэтому в данном представлении проблема `N+1` уже предотвращена способом, отличным от `select_related()`. Не следует считать, что любой запрос к связанному полю автоматически приводит к дополнительному SQL-запросу: результат зависит от способа построения ORM-запроса.

При использовании `annotate()` вместе с `F()` значения связанных полей включаются в результат SQL-запроса. Кроме того, последующий вызов `values()` возвращает словари с выбранными полями, а не полноценные объекты `Booking`.

Проанализируйте количество SQL-запросов, которое отображает Debug Toolbar при открытии страницы. Отдельно отметьте запрос основного списка, запросы справочников помещений и пользователей, а также запрос подсчёта записей, если он выполняется.

Количество запросов зависит от фактической реализации проекта и не должно заранее задаваться как обязательное значение.

Шаг 3. Проверка `annotate()` и `F()`

В `bookings/views.py` найдите блок, в котором добавляются сведения о связанных объектах:

```
# Добавляем сведения о связанных объектах.
bookings_data = bookings_data.annotate(
    room_name=F("room__name"),
    capacity=F("room__capacity"),
    user_name=F("user__name"),
)
```

Этот код не требуется заменять на `select_related()`: в текущей реализации он решает задачу получения конкретных полей, а ниже результат преобразуется в словари с помощью `values()`.

Проверьте, что после выполнения запроса шаблон `bookings/booking_list.html` обращается именно к выбранным полям, например:

```
{{ booking.room_name }}
{{ booking.capacity }}
{{ booking.user_name }}
```

Если шаблон использует именно эти ключи, представление и шаблон согласованы между собой.

Теперь откройте раздел SQL в Debug Toolbar и найдите запрос основного списка. Убедитесь, что в нём присутствуют необходимые поля связанных таблиц и что Django не выполняет отдельный запрос к помещению и пользователю для каждой строки списка.

Для дополнительной проверки измените количество бронирований в базе данных и повторно откройте страницу. Если количество строк увеличивается, но отдельные запросы к помещениям и пользователям для каждой записи не появляются, это подтверждает корректную организацию выборки.

Шаг 4. Анализ `select_related()` на странице отдельного бронирования

Откройте bookings/views.py и найдите функцию `booking_detail()`:

```
def booking_detail(request, booking_id):
    # Получаем бронирование по первичному ключу.
    # Одновременно загружаем связанные объекты помещения
    # и пользователя, чтобы избежать лишних запросов.
    booking = get_object_or_404(
        Booking.objects.select_related("room", "user"),
        pk=booking_id,
    )

    # Передаём бронирование в шаблон.
    context = {
        "booking": booking,
    }

    return render(
        request,
        "bookings/booking_detail.html",
        context,
    )
```

В этом представлении уже используется `select_related("room", "user")`. Метод `select_related()` предназначен для связей типа `ForeignKey` и `OneToOneField`. Он позволяет получить основной объект и связанные объекты в одном SQL-запросе, используя соединения таблиц.

В данном случае Django получает бронирование, помещение и пользователя. Благодаря этому шаблон может обращаться к связанным объектам:

```
{{ booking.room.name }}
{{ booking.room.capacity }}
{{ booking.user.name }}
```

без необходимости выполнять отдельный запрос при каждом таком обращении.

Откройте страницу отдельного бронирования и изучите SQL-запросы в Debug Toolbar. Найдите запрос, который получает бронирование по первичному ключу.

Проверьте, что в запросе используются соединения с таблицами помещений и пользователей. При этом не должно возникать дополнительных запросов к этим таблицам только из-за обращения к `booking.room` и `booking.user` в шаблоне.

Для сравнения можно временно убрать `select_related("room", "user")`, перезапустить запрос страницы и посмотреть, как изменится список SQL-запросов. Затем обязательно восстановите исходный вариант с `select_related()`.

Если в шаблоне выводятся оба связанных объекта, при удалении `select_related()` Django может выполнить дополнительные запросы к базе данных. Фактический результат необходимо проверить через Debug Toolbar.

Шаг 5. Проверка справочных списков и пагинации

В `booking_list()` также выполняются запросы:

```
# Получаем помещения для фильтра.
"rooms": Room.objects.order_by("name"),

# Получаем пользователей для фильтра.
"users": User.objects.order_by("name"),
```

Эти запросы нужны для формирования списков выбора в форме фильтрации. Они не являются ошибкой N+1: это отдельные выборки справочных данных, необходимые интерфейсу.

При этом пагинация:

```
paginator = Paginator(bookings_data, 5)
page_number = request.GET.get("page")
page_obj = paginator.get_page(page_number)
```

может приводить к выполнению дополнительного SQL-запроса для определения общего количества записей, поскольку Django должен знать, сколько страниц доступно пользователю.

В разделе SQL проверьте:

- выполняется ли отдельный запрос подсчёта записей;
- ограничивает ли запрос списка объём получаемых данных для текущей страницы;
- сохраняется ли фильтрация при переходе между страницами;
- не загружаются ли все бронирования в Python до применения пагинации.

В вашем случае `Paginator` получает объект `QuerySet`, сформированный цепочкой ORM-операций. Поэтому Django может выполнить запрос с ограничением количества записей для выбранной страницы, а не загружать весь список целиком.

Не следует удалять запрос подсчёта только ради уменьшения числа запросов: он нужен для корректной работы пагинации. Оптимизация должна сохранять функциональность страницы.

Шаг 6. Проверка фильтрации и сортировки

В `booking_list()` фильтрация и сортировка формируются последовательно:

```
# Выбираем поле сортировки.
sort_fields = {
    "date": "booking_date",
    "room": "room__name",
    "user": "user__name",
}

sort_field = sort_fields.get(sort, "booking_date")

# Сортируем по выбранному полю.
bookings_data = bookings_data.order_by(sort_field, "id")
```

Проверьте страницу с разными параметрами URL, например:

```
/bookings/
/bookings/?sort=room
/bookings/?sort=user
/bookings/?date=2026-10-10
/bookings/?room=1
/bookings/?user=1
```

Используйте существующие в базе идентификаторы и даты. Если конкретной даты нет, укажите дату, для которой существуют бронирования.

Для каждого варианта откройте раздел SQL и проверьте, что:

- условия фильтрации отражаются в запросе;
- сортировка по выбранному полю выполняется в SQL;
- при смене параметров не появляются отдельные запросы к связанным объектам для каждой строки;
- пагинация сохраняет выбранные параметры фильтрации, если это предусмотрено шаблоном.

Не заменяйте текущую логику фильтрации ради демонстрации оптимизации: она уже формирует `QuerySet` и позволяет Django выполнять условия на стороне базы данных.

Шаг 7. Нужно ли добавлять `prefetch_related()`?

В текущем коде `booking_list()` выводит конкретные поля помещения и пользователя. Для этих задач уже применяются `F()` и `annotate()`, а в `booking_detail()` — `select_related()`.

Поэтому добавлять `prefetch_related()` в эти представления только ради демонстрации не требуется.

Метод `prefetch_related()` полезен, когда необходимо загрузить набор связанных объектов, например, несколько бронирований для каждого помещения при использовании обратной связи или объекты через `ManyToManyField`. Он выполняет дополнительные запросы и связывает результаты в Python.

Если в другом разделе Roomly появится сценарий отображения списка помещений вместе со всеми связанными бронированиями, можно отдельно исследовать `prefetch_related()`. При этом имя обратной связи должно соответствовать фактическому `related_name` в модели.

В рамках текущего пункта достаточно исследовать уже используемые решения и подтвердить их эффективность через SQL-панель.

Шаг 8. Итоговая проверка

После исследования убедитесь, что код `bookings/views.py` сохраняет исходную функциональность.

Выполните команды:

```
python manage.py check
pytest -v
```

Затем проверьте:

- список активных бронирований;
- фильтрацию по дате, помещению и пользователю;
- сортировку по дате, помещению и пользователю;
- переход между страницами;
- страницу отдельного бронирования;
- отображение связанных данных помещения и пользователя.

Повторно откройте Django Debug Toolbar и сравните SQL-запросы на страницах списка и отдельного бронирования.

В отчёте или рабочих заметках зафиксируйте, какие запросы выполняются, какие из них относятся к основному списку и справочникам, как работает пагинация и почему в текущем коде используются разные способы получения связанных данных.

Результат выполнения: проведён анализ SQL-запросов Roomly через Django Debug Toolbar. Подтверждено, что `annotate()` с `F()` позволяет получить выбранные

поля связанных объектов в запросе списка бронирований, а `select_related()` объединяет получение бронирования с данными помещения и пользователя на странице деталей. Фильтрация, сортировка и пагинация сохранены.

5.4.3. Использование TextChoices для статусов бронирования и развитие менеджера

На этом этапе необходимо заменить булево поле `is_cancelled` в модели `Booking` на поле `status`, поддерживающее три состояния бронирования:

- `ACTIVE` – активно;
- `CANCELLED` – отменено;
- `COMPLETED` – завершено.

Перечисление `TextChoices` позволяет хранить в базе данных текстовый код статуса и отображать пользователю его человекочитаемое название. Менеджер бронирований будет использоваться для получения активных, отменённых и завершённых бронирований.

Основное правило предметной области сохраняется: **помещение считается занятым только при наличии активного бронирования на выбранную дату**. Отмена или завершение бронирования не должны удалять запись из базы данных.

Шаг 1. Изменение модели Booking

Откройте файл `bookings/models.py` и найдите класс `Booking`.

Внутри класса, рядом с другими атрибутами модели, добавьте перечисление статусов:

```
class Status(models.TextChoices):
    # Бронирование действует.
    ACTIVE = "active", "Активно"
    # Бронирование отменено.
    CANCELLED = "cancelled", "Отменено"
    # Бронирование завершено.
    COMPLETED = "completed", "Завершено"
Затем добавьте поле status:
status = models.CharField(
    max_length=12,
    choices=Status.choices,
    default=Status.ACTIVE,
    verbose_name="Статус бронирования",
)
```

На этом этапе не удаляйте поле is_cancelled. Оно понадобится для переноса существующих данных. Не заменяйте весь класс `Booking` приведёнными фрагментами: сохраните остальные поля, связи, методы и настройки модели.

Шаг 2. Создание и применение миграции нового поля

В терминале, в каталоге проекта с файлом `manage.py`, выполните:

```
python manage.py makemigrations bookings
python manage.py migrate
```

Первая команда создаст миграцию для добавления поля `status`, вторая применит её к базе данных.

Поскольку у поля задано значение по умолчанию `Status.ACTIVE`, существующие записи получат начальный статус «Активно». На следующем шаге необходимо перенести в новое поле сведения об отменённых бронированиях.

Шаг 3. Перенос существующих статусов

Создайте пустую миграцию данных:

```
python manage.py makemigrations bookings --empty --name migrate_booking_statuses
```

В каталоге `bookings/migrations/` появится новый файл миграции. Откройте его и добавьте функцию переноса данных и операцию `RunPython`.

```
from django.db import migrations
def migrate_booking_statuses(apps, schema_editor):
    # Получаем историческую модель для выполнения миграции.
    Booking = apps.get_model("bookings", "Booking")
    # Сохраняем сведения об отменённых бронированиях.
    Booking.objects.filter(
        is_cancelled=True,
    ).update(
        status="cancelled",
    )
    # Все остальные существующие бронирования считаем активными.
    Booking.objects.filter(
        is_cancelled=False,
    ).update(
        status="active",
    )
def reverse_booking_statuses(apps, schema_editor):
    # Получаем историческую модель.
    Booking = apps.get_model("bookings", "Booking")
    # Восстанавливаем булево поле при откате миграции.
    Booking.objects.filter(
        status="cancelled",
    ).update(
        is_cancelled=True,
    )
    Booking.objects.exclude(
        status="cancelled",
    ).update(
        is_cancelled=False,
    )
class Migration(migrations.Migration):
```

```
    dependencies = [
        # Замените имя на фактическую предыдущую миграцию bookings.
        ("bookings", "000X_previous_migration"),
    ]
    operations = [
        migrations.RunPython(
            migrate_booking_statuses,
            reverse_booking_statuses,
        ),
    ]
```

В `dependencies` укажите имя миграции, которая непосредственно предшествует созданной миграции данных. Не оставляйте `000X_previous_migration` в готовом файле.

Примените миграцию:

```
python manage.py migrate
```

После переноса данных старые отменённые бронирования получат статус `cancelled`, а остальные – `active`.

Автоматически присваивать статус `completed` по прошедшей дате не следует: такое поведение требует отдельного бизнес-правила. Если в существующих данных уже есть завершённые бронирования, предусмотрите их отдельный перенос.

Шаг 4. Адаптация существующего BookingQuerySet

Откройте `bookings/models.py` и найдите уже реализованный класс `BookingQuerySet`. Не создавайте новый менеджер и не удаляйте существующие методы. Замените реализацию методов `active()` и `cancelled()`, а также добавьте метод `completed()`. Метод `for_date()` оставьте, если он уже реализован так, как показано ниже.

```
class BookingQuerySet(models.QuerySet):
    # Возвращает активные бронирования.
    def active(self):
        return self.filter(
            status=self.model.Status.ACTIVE,
        )
    # Возвращает отменённые бронирования.
    def cancelled(self):
        return self.filter(
            status=self.model.Status.CANCELLED,
        )
    # Возвращает завершённые бронирования.
    def completed(self):
        return self.filter(
            status=self.model.Status.COMPLETED,
        )
```

```
    # Возвращает бронирования за указанную дату.
    def for_date(self, selected_date):
        return self.filter(
            booking_date=selected_date,
        )
```

Использование `self.model.Status` позволяет обращаться к перечислению модели `Booking` из методов набора запросов.

Существующий класс `BookingManager` оставьте без изменений:

```
class BookingManager(
    models.Manager.from_queryset(BookingQuerySet)
):
    pass
```

Проверьте также, что в модели `Booking` подключён этот менеджер:

```
objects = BookingManager()
```

Если он уже подключён, повторно добавлять его не нужно.

После этого можно использовать единообразные вызовы:

```
Booking.objects.active()
Booking.objects.cancelled()
Booking.objects.completed()
Booking.objects.for_date(selected_date)
```

Шаг 5. Замена условий в функции поиска свободных помещений

Найдите функцию `find_available_rooms()` в файле, где она сейчас реализована. Если она находится в `bookings/utils.py`, измените именно этот файл.

Было:

```
booked_room_ids = Booking.objects.filter(
    booking_date=selected_date,
    is_cancelled=False,
).values_list(
    "room_id",
    flat=True,
)
```

Замените на:

```
booked_room_ids = (
    Booking.objects
    .active()
    .filter(booking_date=selected_date)
    .values_list("room_id", flat=True)
)
```

Если функция содержит последующее исключение занятых помещений, оставьте его без изменений:

```
return (
    Room.objects
    .exclude(pk__in=booked_room_ids)
    .order_by("name")
)
```

Теперь функция использует существующий менеджер и учитывает только активные бронирования. Отменённые и завершённые записи не блокируют помещение.

Шаг 6. Изменение статистики бронирований

Найдите в файле представлений, например `bookings/views.py`, функцию статистики бронирований. В ней найдите вызов `Booking.objects.aggregate()`.

Замените фильтры по `is_cancelled` на фильтры по статусу:

```
stats = Booking.objects.aggregate(
    total_bookings=Count("id"),
    active_bookings=Count(
        "id",
        filter=Q(status=Booking.Status.ACTIVE),
    ),
    cancelled_bookings=Count(
        "id",
        filter=Q(status=Booking.Status.CANCELLED),
    ),
    completed_bookings=Count(
        "id",
        filter=Q(status=Booking.Status.COMPLETED),
    ),
)
```

Проверьте импорты в начале файла:

```
from django.db.models import Count, Q
```

Если `Count` и `Q` уже импортированы, повторять импорт не нужно.

В шаблоне `bookings/statistics.html` найдите вывод показателей статистики. Если требуется показывать все три состояния, добавьте отображение `completed_bookings`. Если в интерфейсе пока нужны только активные и отменённые бронирования, новый показатель можно не выводить.

Шаг 7. Адаптация Django Admin

Откройте файл `rooms/admin.py`, если в нём объявлен `BookingInline`. Было:

```
fields = ("user", "booking_date", "is_cancelled")
```

Замените на:

```
fields = ("user", "booking_date", "status")
```

Затем откройте bookings/admin.py и найдите класс `BookingAdmin`.

В `list_display` замените `is_cancelled` на `status`:

```
list_display = (
    "id",
    "room",
    "user",
    "booking_date",
    "status",
)
```

В `list_filter` выполните такую же замену:

```
list_filter = ("booking_date", "status")
```

Остальные настройки, включая `list_display_links` и `search_fields`, оставьте без изменений.

В Django Admin теперь будет отображаться название статуса, а фильтрация позволит выбирать активные, отменённые и завершённые бронирования.

Шаг 8. Поиск оставшихся обращений к `is_cancelled`

До удаления старого поля найдите по всему проекту все вхождения `is_cancelled`. В первую очередь проверьте:

- bookings/models.py – модель и `BookingQuerySet`;
- файлы представлений, включая список бронирований и статистику;
- функцию `find_available_rooms()`;
- bookings/admin.py и rooms/admin.py;
- шаблоны списка, карточки и статистики бронирований;
- автоматизированные тесты;
- импорт и экспорт данных, если они используют это поле.

Заменяйте только те обращения, которые относятся к текущей модели `Booking`. Не изменяйте историческую миграцию данных: она должна продолжать обращаться к `is_cancelled`, поскольку переносит сведения из старого поля.

Если шаблоны выводят булево значение напрямую, замените его на отображение статуса. Например:

```
{{ booking.get_status_display }}
```

Для проверки статуса в шаблонах можно использовать:

```
{% if booking.status == booking.Status.ACTIVE %}
    Активно
{% elif booking.status == booking.Status.CANCELLED %}
    Отменено
```

```
{% elif booking.status == booking.Status.COMPLETED %}
    Завершено
{% endif %}
```

Если проверка в шаблоне не требуется, достаточно `get_status_display`.

Шаг 9. Адаптация автоматизированных тестов

В тестах, где создаются бронирования, замените передачу `is_cancelled` на `status`. Например, активное бронирование:

```
booking = Booking.objects.create(
    room=room,
    user=user,
    booking_date=selected_date,
    status=Booking.Status.ACTIVE,
)
```

Отменённое бронирование:

```
booking = Booking.objects.create(
    room=room,
    user=user,
    booking_date=selected_date,
    status=Booking.Status.CANCELLED,
)
```

Завершённое бронирование:

```
booking = Booking.objects.create(
    room=room,
    user=user,
    booking_date=selected_date,
    status=Booking.Status.COMPLETED,
)
```

Проверьте как минимум следующие правила:

- активное бронирование исключает помещение из списка свободных на соответствующую дату;
- отменённое бронирование не блокирует помещение;
- завершённое бронирование не блокирует помещение;
- методы `active()`, `cancelled()`, `completed()` возвращают только соответствующие записи;
- статистика правильно считает каждое состояние.

На этом этапе старое поле ещё может оставаться в модели, но прикладной код и тесты уже не должны от него зависеть.

Запустите проверки:

```
python manage.py check
```

```
pytest -v
```

Устраните ошибки до удаления старого поля.

Шаг 10. Удаление `is_cancelled`

Только после успешного прохождения проверок откройте bookings/models.py и удалите объявление поля `is_cancelled` из класса `Booking`.

Затем выполните:

```
python manage.py makemigrations bookings
python manage.py migrate
```

Django создаст миграцию удаления поля. Перед её применением проверьте, что она удаляет только `is_cancelled` и не содержит непредусмотренных изменений.

После применения миграции повторите проверки:

```
python manage.py check
pytest -v
```

Результат выполнения

После завершения работы модель `Booking` использует три статуса вместо булева признака отмены. Существующий менеджер предоставляет методы для выборки бронирований по статусу и дате. Поиск свободных помещений, статистика, административный интерфейс и тесты работают с новым полем.

Старые бронирования сохраняются, а их статус переносится из `is_cancelled`. Записи не удаляются при отмене или завершении бронирования.

Далее рассмотрим три возможности Django в последовательности от модели к пользовательскому интерфейсу:

1. `GeneratedField` – вычисляемое поле модели, значение которого формируется на уровне базы данных.
2. `slug` и `get_absolute_url()` – формирование удобных адресов страниц помещений.
3. `Context processor` – передача общих данных в шаблоны без дублирования кода в представлениях.

5.4.4. Использование GeneratedField в модели Room

`GeneratedField` позволяет объявить поле, значение которого вычисляет сама база данных на основе других полей той же записи. Такое значение автоматически пересчитывается при изменении исходных данных; при этом поддержка выражений зависит от используемой СУБД.

Для Roomly можно рассмотреть вычисляемый признак вместимости помещения. Например, будем считать помещение большим, если его вместимость составляет не

менее 50 человек. Порог в 50 человек – учебный пример; при необходимости замените его на значение, соответствующее требованиям проекта.

Шаг 1. Изменение модели `Room`

Откройте `rooms/models.py` и найдите класс `Room`. Внутри класса, рядом с полем `capacity`, добавьте поле `is_large`:

```
from django.db import modelsfrom django.db.models import Case, When, Value
```

Если `models` уже импортирован, повторять его импорт не нужно. Остальные импорты добавьте только при необходимости.

```
is_large = models.GeneratedField(
    # Вычисляем признак по вместимости помещения.
    expression=Case(
        When(capacity__gte=50, then=Value(True)),
        default=Value(False),
        output_field=models.BooleanField(),
    ),
    # Тип вычисляемого значения.
    output_field=models.BooleanField(),
    # Сохраняем вычисленное значение в базе данных.
    db_persist=True,
    verbose_name="Большое помещение",
)
```

Примечание: это поле не заменяет `capacity`. Вместимость остаётся исходным значением, а `is_large` вычисляется на его основе. Не добавляйте `is_large` в формы для ручного редактирования: значение должно определяться базой данных.

Шаг 2. Создание и применение миграции

В терминале, в каталоге проекта с файлом `manage.py`, выполните:

```
python manage.py makemigrations rooms
python manage.py migrate
```

Если миграция не создаётся или не применяется, проверьте версию SQLite и поддержку генерируемых столбцов. Также убедитесь, что выражение поддерживается используемой СУБД.

Шаг 3. Проверка вычисляемого значения

Откройте Django shell:

```
python manage.py shell
```

Выполните проверку, используя существующие помещения:

```
from rooms.models import Room

for room in Room.objects.all():
```

```
print(room.name, room.capacity, room.is_large)
```

Для помещения вместимостью 30 человек значение `is_large` должно быть `False`, а для помещения вместимостью 60 человек – `True`.

Проверьте и автоматический пересчёт. Выберите существующее помещение, измените его вместимость и сохраните:

```
room = Room.objects.first()

room.capacity = 60
room.save()

room.refresh_from_db()

print(room.capacity, room.is_large)
```

Здесь `refresh_from_db()` позволяет перечитать вычисленное значение из базы данных. В версиях и на СУБД, где Django автоматически обновляет такие поля после сохранения, явное обновление может быть избыточным, но для учебной проверки оно наглядно показывает источник значения.

Что проверить перед продолжением:

- миграция `rooms` применена успешно;
- значение `is_large` нельзя задавать как обычное редактируемое поле;
- после изменения `capacity` признак пересчитывается;
- существующие страницы списка и подробной информации о помещении продолжают работать.

5.4.5. Добавление англоязычного slug, реализация get_absolute_url() и использование в Django Admin

В этом пункте необходимо изменить способ формирования URL подробной страницы помещения. Вместо числового идентификатора в адресе будет использоваться англоязычный текстовый идентификатор `slug`.

Например, помещение «Аудитория 301» будет доступно по адресу:

```
/rooms/auditoriya-301-1/
```

Для этого потребуется:

добавить в модель Room поле slug;

- настроить транслитерацию русских названий в латиницу;
- заполнить `slug` для уже существующих помещений;
- обеспечить автоматическое создание `slug` для новых помещений;
- использовать `slug` при открытии подробной страницы;
- реализовать метод `get_absolute_url()`;

- использовать метод в шаблонах и Django Admin.

Шаг 1. Установка библиотеки для транслитерации

Стандартная функция Django `slugify()` не переводит русские буквы в латиницу. Если передать ей русское название с `allow_unicode=True`, кириллица сохранится в результате. Если не указывать этот параметр, кириллические символы могут быть удалены, а не транслитерированы.

Для преобразования русских названий в латиницу установите библиотеку `Unidecode`.

В терминале проекта выполните:

```
pip install Unidecode
```

Добавьте библиотеку в файл `requirements.txt`:

```
Django
Unidecode
```

Если в requirements.txt уже перечислены другие зависимости, сохраните их и добавьте только строку `Unidecode`. Версию Django укажите в соответствии с фактически используемой в проекте.

Теперь можно применять `unidecode()` для транслитерации, а `slugify()` – для формирования строки, подходящей для URL.

Например:

```
from django.utils.text import slugify
from unidecode import unidecode

name = "Аудитория 301"

# Преобразуем русское название в латиницу.
transliterated_name = unidecode(name)

# Убираем лишние символы и формируем slug.
base_slug = slugify(transliterated_name)

print(base_slug)
```

Результат:

```
auditoriya-301
```

Таким образом, транслитерация и формирование slug – два отдельных действия: сначала выполняется преобразование текста, затем нормализация строки для URL.

Шаг 2. Добавление поля `slug` в модель `Room`

Откройте `rooms/models.py` и найдите модель `Room`.

Добавьте в неё поле:

```
slug = models.SlugField(
    max_length=180,
    blank=True,
    editable=False,
    verbose_name="URL-идентификатор",
)
```

На этом этапе не указывайте `unique=True`: сначала нужно заполнить поле у всех существующих помещений, а затем добавить ограничение уникальности.

Параметры поля:

- `max_length=180` – максимальная длина текстового идентификатора;
- `blank=True` – поле может быть пустым при проверке формы;
- `editable=False` – поле не редактируется через обычные формы модели;
- `verbose_name` – название поля в административном интерфейсе.

Параметр `blank=True` нужен на промежуточном этапе, пока данные ещё не заполнены. После успешного заполнения и проверки уникальности поле можно сделать обязательным, убрав `blank=True`.

После изменения модели выполните:

```
python manage.py makemigrations rooms
python manage.py migrate
```

Эта миграция только добавляет поле. Она ещё не формирует slug для существующих помещений.

Шаг 3. Заполнение `slug` для существующих помещений

После добавления поля необходимо заполнить его для уже сохранённых объектов `Room`.

Для этого создайте отдельную миграцию данных. Например, файл может называться `0006_populate_room_slugs.py`, но номер зависит от фактической последовательности миграций проекта.

Создайте файл в каталоге `rooms/migrations/` со следующим содержимым:

```
from django.db import migrations
from django.utils.text import slugify
from unidecode import unidecode


def populate_room_slugs(apps, schema_editor):
    # Получаем историческую модель Room,
    # соответствующую состоянию БД на момент миграции.
    Room = apps.get_model("rooms", "Room")
```

```
    # Обрабатываем помещения в стабильном порядке.
    for room in Room.objects.all().order_by("pk"):
        # Переводим название в латиницу.
        transliterated_name = unidecode(room.name)

        # Формируем базовую часть slug.
        # Если название не содержит подходящих символов,
        # используем значение room.
        base_slug = slugify(transliterated_name) or "room"

        # Добавляем первичный ключ, чтобы названия
        # одинаковых помещений не приводили к совпадению slug.
        room.slug = f"{base_slug}-{room.pk}"

        # Сохраняем только изменяемое поле.
        room.save(update_fields=["slug"])


class Migration(migrations.Migration):

    dependencies = [
        # Замените имя на фактическую миграцию,
        # в которой добавлено поле slug.
        ("rooms", "0005_room_slug"),
    ]

    operations = [
        migrations.RunPython(
            populate_room_slugs,
            migrations.RunPython.noop,
        ),
    ]
```

Примечание: 0005_room_slug в примере – условное имя. Укажите реальное имя существующей миграции, добавившей поле slug. Не оставляйте ссылку на отсутствующий файл.

Почему используется `apps.get_model()`, а не обычный импорт `Room` из `rooms.models`? Миграция должна работать с историческим состоянием модели на момент своего выполнения. Это позволяет корректно выполнять миграции даже после дальнейших изменений `models.py`.

Обратите внимание: в миграции импортируется `unidecode`. Поэтому библиотека должна быть установлена в окружении, в котором выполняются миграции.

После создания файла проверьте последовательность миграций:

```
python manage.py showmigrations rooms
```

Затем выполните:

```
python manage.py migrate
```

Если возникает `NodeNotFoundError`, проверьте имена файлов и dependencies. Если возникает `CircularDependencyError`, проверьте, не образуют ли миграции циклическую зависимость. Миграция заполнения данных должна зависеть от миграции добавления поля, а не наоборот.

Шаг 4. Автоматическое формирование `slug` для новых помещений

Теперь необходимо настроить автоматическое создание slug при сохранении нового объекта `Room`.

В начале `rooms/models.py` добавьте импорты, если их ещё нет:

```
from django.utils.text import slugify
from unidecode import unidecode
```

Внутри модели Room реализуйте метод save():

```
def save(self, *args, **kwargs):
    # Формируем slug только тогда, когда он ещё не задан.
    if not self.slug:
        # Сначала транслитерируем русское название.
        transliterated_name = unidecode(self.name)

        # Затем приводим строку к формату URL.
        base_slug = slugify(transliterated_name) or "room"

        if self.pk:
            # У существующего объекта первичный ключ уже есть.
            # Поэтому можно сразу сформировать slug с ID.
            self.slug = f"{base_slug}-{self.pk}"
        else:
            # Новый объект ещё не имеет первичного ключа.
            # Сначала сохраняем его, чтобы база данных
            # присвоила ему ID.
            super().save(*args, **kwargs)

            # После первого сохранения добавляем ID к slug.
            self.slug = f"{base_slug}-{self.pk}"

            # Сохраняем сформированный slug.
            super().save(update_fields=["slug"])

            # Выходим, чтобы не выполнять сохранение повторно.
            return
```

```
    # Сохраняем объект обычным способом.
    super().save(*args, **kwargs)
```

Если в модели уже есть собственный метод `save()`, не создавайте второй метод с тем же именем. Объедините новую логику с существующей реализацией.

Разберём последовательность на примере нового помещения:

1. Создаётся объект с названием «Аудитория 301».
2. Метод `save()` транслитерирует название.
3. `slugify()` формирует базовую строку `auditoriya`-301.
4. Объект сохраняется, и база данных присваивает ему `ID`.
5. К базовой строке добавляется `ID`, например `auditoriya`-301-15.
6. Новое значение slug сохраняется в базе данных.

Если `slug` уже заполнен, метод не пересоздаёт его автоматически. Это позволяет не менять адрес объекта при каждом обычном сохранении.

Шаг 5. Обеспечение уникальности slug

После заполнения всех существующих записей и настройки автоматического формирования `slug` добавьте ограничение уникальности.

В `rooms/models.py` измените поле:

```
slug = models.SlugField(
    max_length=180,
    unique=True,
    editable=False,
    verbose_name="URL-идентификатор",
)
```

Здесь `blank=True` уже не требуется, поскольку все существующие помещения должны иметь значение, а новые объекты формируют slug автоматически.

Выполните:

```
python manage.py makemigrations rooms
python manage.py migrate
```

Если миграция завершается ошибкой, проверьте, что у каждого помещения заполнено поле `slug` и значения не повторяются.

Добавление `unique=True` важно не только для корректной работы URL. Оно обеспечивает ограничение на уровне базы данных, поэтому два помещения не смогут иметь одинаковый slug.

Шаг 6. Реализация метода `get_absolute_url()`

Теперь добавьте в модель `Room` метод `get_absolute_url()`.

В начале `rooms/models.py` должен быть импорт:

```
from django.urls import reverse
```

Внутри класса `Room` разместите метод:

```
def get_absolute_url(self):
    """Возвращает URL подробной страницы помещения."""

    # reverse() находит маршрут по его имени
    # и подставляет значение slug текущего объекта.
    return reverse(
        "rooms:detail",
        kwargs={"slug": self.slug},
    )
```

Если импорт `reverse` уже присутствует, повторно добавлять его не нужно.

Для чего нужен `get_absolute_url()`

Этот метод определяет, по какому адресу можно открыть конкретный объект модели. В нашем проекте он возвращает URL подробной страницы помещения.

Например, если объект имеет:

```
room.name == "Аудитория 301"
room.slug == "auditoriya-301-1"
```

вызов:

```
room.get_absolute_url()
```

вернёт:

```
/rooms/auditoriya-301-1/
```

Метод использует `reverse()`, а не собирает адрес вручную. Это означает, что вызывающему коду не нужно знать, что подробная страница располагается по пути `rooms/<slug>/`.

Вызов состоит из трёх основных элементов:

- `"rooms:detail"` – имя маршрута с пространством имён приложения;
- `kwargs={"slug": self.slug}` – значение параметра `slug` текущего объекта;
- `reverse()` – функция Django, которая формирует URL по имени маршрута.

get_absolute_`url()` не создаёт страницу и не выполняет поиск помещения в базе данных. Метод формирует адрес на основании уже существующего объекта. Само представление по этому адресу затем получает объект из базы данных.

Шаг 7. Настройка URL-маршрута

Откройте `rooms/urls.py`.

Маршрут подробной страницы должен использовать стандартный конвертер slug, поскольку теперь адреса содержат латинские буквы, цифры и дефисы.

Пример:

```
from django.urls import path

from . import views

app_name = "rooms"

urlpatterns = [
    # Страница со списком помещений.
    path("", views.room_list, name="list"),

    # Подробная страница помещения по англоязычному slug.
    path(
        "<slug:slug>/",
        views.room_detail,
        name="detail",
    ),
]
```

Если представление списка у вас называется иначе, оставьте фактическое имя. Важно, чтобы маршрут подробной страницы имел имя `detail`, а пространство имён приложения было `rooms`, как указано в `get_absolute_url()`.

Проверьте также подключение приложения в `roomly/urls.py`:

```
path("rooms/", include("rooms.urls")),
```

В результате маршрут будет соответствовать адресу:

```
/rooms/auditoriya-301-1/
```

Конвертер `slug` принимает латинские буквы, цифры, дефисы и подчёркивания. Кириллица в таких URL не требуется, поскольку названия транслитерируются при формировании `slug`.

Шаг 8. Изменение представления подробной страницы

Откройте `rooms/views.py` и найдите представление подробной страницы помещения.

Если раньше объект искался по числовому идентификатору, например:

```
room = get_object_or_404(Room, pk=room_id)
```

измените параметр представления и способ поиска:

```
def room_detail(request, slug):
    # Находим помещение по текстовому идентификатору.
    # Если запись не найдена, Django вернёт страницу 404.
    room = get_object_or_404(Room, slug=slug)
```

```
    # Остальную существующую логику представления
    # и формирования context сохраните.
```

Это фрагмент для изменения существующего представления, а не полный код `views.py`. Не удаляйте импорты и остальную логику, которая нужна для формирования страницы.

Теперь Django получает `slug` из URL и использует его для поиска соответствующего помещения.

Шаг 9. Использование `get_absolute_url()` в шаблонах

Откройте шаблон списка помещений, например `rooms/templates/rooms/room_list.html`.

Найдите ссылку на подробную страницу помещения. Если URL формируется через `id` или через {% `url` %}, замените эту ссылку вызовом `get_absolute_url()`.

Например, вместо:

```
<a href="{% url 'rooms:detail' room.id %}">
    {{ room.name }}
</a>
```

используйте:

```
<a href="{{ room.get_absolute_url }}">
    {{ room.name }}
</a>
```

В шаблонах Django метод без аргументов вызывается без круглых скобок. Поэтому правильная запись – `room.get_absolute_url`, а не `room.get_absolute_url()`.

Преимущество такого подхода состоит в том, что шаблон не должен знать, как именно устроен URL. Он обращается к методу модели, а метод самостоятельно формирует адрес.

Если ссылки на подробные страницы помещений есть в других шаблонах, используйте тот же способ.

Шаг 10. Использование `get_absolute_url()` в Django Admin

У метода есть ещё одно важное применение – формирование ссылки на публичную страницу объекта в административной панели Django.

Django Admin может использовать `get_absolute_url()` для ссылки «Посмотреть на сайте» (`View on site`) на странице просмотра или редактирования сохранённого объекта.

Чтобы это работало, необходимо:

1. Реализовать `get_absolute_url()` в модели `Room`.

2. Убедиться, что маршрут `rooms:detail` существует.

3. Убедиться, что метод возвращает действительный URL.

4. Не отключать ссылку на сайт в настройках `ModelAdmin`.

В существующем классе `RoomAdmin` проверьте параметр `view_on_site`. По умолчанию он включён. Если он установлен в `False`, измените его на `True`:

```
view_on_site = True
```

Например, если в `rooms/admin.py` уже зарегистрирована модель:

```
@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "name",
        "capacity",
    )

    # Разрешаем Django Admin показывать ссылку
    # на публичную страницу сохранённого помещения.
    view_on_site = True
```

Это пример настройки, а не требование создавать второй класс `RoomAdmin`. Если модель уже зарегистрирована, измените существующий класс. Повторная регистрация одной модели приведёт к ошибке.

Как проверить ссылку «Посмотреть на сайте»

1. Запустите сервер:

```
python manage.py runserver
```

2. Откройте `/admin/`.

3. Перейдите к списку помещений.

4. Откройте сохранённое помещение.

5. Найдите ссылку «Посмотреть на сайте».

6. Перейдите по ней и убедитесь, что открылась подробная страница этого помещения.

Если ссылка не отображается, проверьте `view_on_site` и наличие `get_absolute_url()` в модели. Если ссылка отображается, но ведёт к ошибке `NoReverseMatch` или 404, проверьте соответствие имени маршрута, параметра `slug` и фактического значения в базе данных.

Дополнительный вариант: ссылка непосредственно в списке Admin

При необходимости можно добавить в список помещений отдельный столбец «Страница на сайте». Это удобно, если требуется открывать публичную страницу без перехода в карточку редактирования.

В файле `rooms/admin.py` добавьте импорт:

```
from django.utils.html import format_html
```

Внутри существующего `RoomAdmin` добавьте метод:

```
@admin.display(description="Страница на сайте")
def site_link(self, obj):
    # Получаем URL через метод модели,
    # не формируя адрес вручную.
    return format_html(
        '<a href="{}" target="_blank">Открыть</a>',
        obj.get_absolute_url(),
    )
```

Затем добавьте "site_link" в существующий list_display:

```
list_display = (
    "id",
    "name",
    "capacity",
    "site_link",
)
```

Сохраните другие столбцы, если они уже используются в проекте.

Здесь:

- `@admin.display(description="Страница на сайте")` задаёт заголовок столбца;
- `obj` – конкретное помещение из списка;
- `obj.get_absolute_url()` возвращает адрес его публичной страницы;
- `format_html()` безопасно формирует `HTML`-ссылку;
- `target="_blank"` открывает страницу в новой вкладке.

Таким образом, метод `get_absolute_url()` используется и Django Admin, и обычными шаблонами. При этом логика формирования URL находится в одном месте – в модели.

Шаг 11. Обновление `slug` у существующих помещений

Изменение метода `save()` не изменяет автоматически slug у объектов, у которых поле уже заполнено. Поэтому после перехода на транслитерацию нужно обновить существующие записи.

Для небольшого учебного проекта это можно выполнить через Django `shell`.

В терминале запустите:

```
python manage.py shell
```

В открывшейся консоли выполните:

```
from django.utils.text import slugify
from unidecode import unidecode
from rooms.models import Room

for room in Room.objects.all():
    # Преобразуем русское название в латиницу.
    transliterated_name = unidecode(room.name)

    # Формируем базовую часть slug.
    base_slug = slugify(transliterated_name) or "room"

    # Добавляем первичный ключ для уникальности.
    room.slug = f"{base_slug}-{room.pk}"

    # Обновляем только поле slug.
    room.save(update_fields=["slug"])
```

В этом коде `save(update_fields=["slug"])` обновляет только поле `slug`. Поэтому метод `save()` модели, который создаёт slug только при его отсутствии, не будет перезаписывать уже заданное значение.

Перед выполнением убедитесь, что схема базы данных уже содержит поле `slug` и оно допускает сохранение новых значений.

Если проект содержит много данных или старые URL уже используются другими системами, предпочтительнее выполнить обновление через отдельную миграцию данных, а не вручную через `shell`.

Примечание: обновление `slug` меняет адреса существующих помещений. Если требуется сохранить старые ссылки, необходимо отдельно реализовать перенаправление со старых URL на новые.

Шаг 12. Проверка результата

Выйдите из Django `shell` командой `exit()` и выполните:

```
python manage.py check
python manage.py showmigrations rooms
pytest -v
```

Затем проверьте значения slug через Django `shell`:

```
python manage.py shell
from rooms.models import Room

for room in Room.objects.all():
    print(
```

```
        room.name,
        room.slug,
        room.get_absolute_url(),
    )
```

Для помещения «Аудитория 301» ожидается результат примерно следующего вида:

```
Аудитория 301 auditoriya-301-1 /rooms/auditoriya-301-1/
```

После этого проверьте:

- все помещения имеют заполненный `slug`;
- `slug` содержит латинские буквы, цифры и дефисы;
- значения `slug` уникальны;
- у новых помещений `slug` формируется автоматически;
- подробная страница открывается по адресу с `slug`;
- ссылки в шаблонах используют `get_absolute_url()`;
- Django Admin предоставляет ссылку на публичную страницу помещения;
- все существующие тесты проходят успешно.

В результате модель `Room` получает англоязычный `slug`, который формируется на основе транслитерации названия и идентификатора записи. Подробная страница помещения открывается по человекочитаемому адресу, шаблоны используют единый метод для формирования ссылок, а Django Admin позволяет переходить к публичной странице объекта.

5.4.6. Использование контекстного процессора Django (context processor)

Контекстный процессор Django – это функция, которая возвращает словарь данных, автоматически добавляемых в контекст шаблонов. Он позволяет передавать общие данные в шаблоны без необходимости повторять один и тот же код в нескольких представлениях.

Например, в проекте Roomly разные страницы могут использовать общую навигацию, название приложения и количество помещений. Если получать эти данные отдельно в каждой `view`-функции, код будет дублироваться.

Контекстный процессор позволяет централизовать подготовку таких данных и использовать их в общем шаблоне `base.html`.

В этом пункте необходимо:

- создать модуль контекстных процессоров;
- реализовать функцию, возвращающую общие данные Roomly;
- зарегистрировать функцию в настройках Django;
- вывести полученные данные в общем шаблоне;
- проверить доступность данных на нескольких страницах.

Шаг 1. Создание модуля контекстных процессоров

В приложении `rooms` создайте файл `context_processors.py`.

Структура соответствующей части проекта будет выглядеть так:

```
rooms/
├── __init__.py
├── admin.py
├── apps.py
├── context_processors.py  # Новый файл
├── models.py
├── urls.py
├── views.py
└── ...
```

В файле rooms/context_processors.py разместите следующий код:

```
from .models import Room


def room_context(request):
    """
    Передаёт общие данные о помещениях в шаблоны Roomly.

    request — текущий HTTP-запрос.
    Возвращаемый словарь становится частью контекста шаблона.
    """

    # Получаем актуальное количество помещений из базы данных.
    rooms_count = Room.objects.count()

    # Возвращаем словарь с данными,
    # которые будут доступны в шаблонах.
    return {
        "site_name": "Roomly",
        "rooms_count": rooms_count,
    }
```

Разберём реализацию.

Импорт модели `Room`.

```
from .models import Room
```

Контекстный процессор использует модель `Room`, чтобы получить количество помещений из базы данных. Это значение не задаётся константой и будет меняться вместе с данными.

Объявление функции.

```
def room_context(request):
```

Контекстный процессор представляет собой обычную Python-функцию, принимающую объект HTTP-запроса request. Django передаёт его функции автоматически.

В данном примере сам объект request не используется для вычислений, однако он предусмотрен интерфейсом контекстного процессора и может потребоваться в других реализациях.

Получение количества помещений.

```
rooms_count = Room.objects.count()
```

Метод count() выполняет подсчёт записей средствами базы данных. В отличие от получения всех объектов с последующим вызовом len(), такой подход не загружает всю коллекцию помещений в память.

Формирование результата.

```
return {
"site_name": "Roomly",
"rooms_count": rooms_count,
}
```

Функция возвращает словарь, в котором ключи становятся именами переменных шаблона. Поэтому в шаблонах будут доступны site_name и rooms_count.

Обратите внимание: контекстный процессор не возвращает HTML и не формирует готовую страницу. Его задача – подготовить данные, которые затем используются при обработке шаблона.

Шаг 2. Регистрация контекстного процессора в настройках Django

Создать файл недостаточно: Django должен знать, что эту функцию необходимо вызывать при формировании контекста шаблонов.

Откройте файл roomly/settings.py и найдите настройку TEMPLATES.

В ней должен присутствовать параметр OPTIONS, содержащий список context_processors.

Добавьте в этот список путь к созданной функции:

```
"rooms.context_processors.room_context",
```

Например, соответствующая часть настройки может выглядеть так:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
```

```
                # Стандартный контекстный процессор Django.
                "django.template.context_processors.debug",

                # Передаёт объект request в шаблон.
                "django.template.context_processors.request",

                # Данные пользователя и прав доступа.
                "django.contrib.auth.context_processors.auth",

                # Сообщения Django.
                "django.contrib.messages.context_processors.messages",

                # Общие данные приложения Roomly.
                "rooms.context_processors.room_context",
            ],
        },
    },
]
```

Это пример полного списка основных контекстных процессоров. Если в проекте уже есть дополнительные процессоры, сохраните их. Не заменяйте весь существующий список без необходимости: достаточно добавить строку с путём к `room_context`.

Путь записывается в формате `имя_приложения.модуль.функция`:

- `rooms` – приложение;
- `context_processors` – созданный модуль;
- `room_context` – функция, которая возвращает словарь данных.

После регистрации Django сможет вызывать эту функцию при формировании контекста шаблонов через стандартный механизм `RequestContext`.

Шаг 3. Использование данных в общем шаблоне `base.html`

Откройте общий шаблон `templates/base.html`.

Найдите элемент, в котором отображается название приложения. Если название сейчас записано непосредственно в `HTML`, замените его переменной `site_name`.

Например:

```
<title>{{ site_name }}</title>
```

В шапке сайта также можно вывести название приложения:

```
<header>
    <h1>{{ site_name }}</h1>
</header>
```

Добавьте в навигацию количество помещений:

```
<nav>
    <a href="{% url 'homepage:index' %}">Главная</a>
    <a href="{% url 'rooms:list' %}">
        Помещения ({{ rooms_count }})
    </a>
    <a href="{% url 'bookings:list' %}">Бронирования</a>
</nav>
```

В примере используются имена маршрутов `homepage:index`, `rooms:list` и `bookings:list`. Если в вашем проекте имена отличаются, укажите фактические имена маршрутов из URL-конфигураций приложений.

Переменная `rooms_count` будет заменена на число, полученное из базы данных. Например, при наличии четырёх помещений навигация отобразит «Помещения (4)». Не нужно добавлять `rooms_count` в контекст каждой `view`-функции: после регистрации контекстного процессора значение будет доступно шаблонам, использующим стандартный механизм формирования контекста Django.

Шаг 4. Проверка работы контекстного процессора на разных страницах

Убедитесь, что основные шаблоны приложения используют общий шаблон через `{% extends %}`.

Например, в `rooms/templates/rooms/room_list.html` в начале файла должно быть:

```
{% extends "base.html" %}

{% block content %}
    <h2>Помещения</h2>

    {# Здесь располагается существующий список помещений. #}
{% endblock %}
```

Не заменяйте содержимое страницы этим коротким примером: сохраните существующий цикл вывода помещений и остальные элементы. Важно, чтобы шаблон наследовал `base.html`.

Проверьте также наследование общего шаблона в страницах списка бронирований и главной странице, если они входят в структуру проекта.

Запустите сервер:

```
python manage.py runserver
```

Откройте несколько страниц Roomly, например:

- главную страницу;
- список помещений;
- список бронирований.

На каждой странице, использующей `base.html`, должно отображаться название Roomly и количество помещений в навигации.

Добавьте или удалите помещение предусмотренным в проекте способом, затем обновите страницу. Количество должно соответствовать текущим данным базы данных.

Шаг 5. Проверка контекстного процессора через тест

При необходимости добавьте автоматизированную проверку в `rooms/tests.py` или в отдельный файл тестов приложения.

Пример для проекта, в котором настроены стандартные URL-маршруты:

```
from django.test import TestCase
from django.urls import reverse

from rooms.models import Room


class RoomContextProcessorTests(TestCase):
    def test_room_count_is_available_in_template(self):
        # Создаём два помещения для проверки подсчёта.
        Room.objects.create(
            name="Аудитория 301",
            capacity=30,
        )
        Room.objects.create(
            name="Конференц-зал",
            capacity=60,
        )

        # Открываем страницу списка помещений.
        response = self.client.get(
            reverse("rooms:list")
        )

        # Проверяем успешный ответ сервера.
        self.assertEqual(response.status_code, 200)

        # Проверяем данные, переданные в контекст шаблона.
        self.assertEqual(
            response.context["site_name"],
            "Roomly",
        )
        self.assertEqual(
            response.context["rooms_count"],
            2,
```

```
        )
```

Перед использованием примера проверьте обязательные поля модели `Room` и при необходимости добавьте их в создание тестовых объектов. Если для создания помещения требуются дополнительные значения, укажите их в соответствии с моделью проекта.

Этот тест проверяет не только наличие функции в коде, но и доступность её данных в контексте страницы. Для проверки полного пользовательского сценария дополнительно убедитесь, что переменная `rooms_count` действительно используется в `base.html`.

Шаг 6. Учитывайте назначение контекстного процессора

Контекстный процессор следует использовать для данных, которые действительно нужны многим шаблонам. В данном примере количество помещений отображается в общей навигации, поэтому его получение централизовано.

При этом контекстный процессор вызывается при формировании контекста шаблона. Запрос `Room.objects.count()` может выполняться при каждом таком обращении, поэтому не следует переносить сюда все данные приложения и тяжёлые вычисления без необходимости.

Если количество помещений требуется только на одной странице, рациональнее получить его непосредственно в соответствующей `view`-функции. Если данные нужны повсеместно, контекстный процессор упрощает код и обеспечивает единый способ их передачи.

Также не следует создавать в контекстном процессоре переменные, имена которых совпадают с важными переменными отдельных представлений: значения из контекстных процессоров могут пересекаться с контекстом страницы. Используйте понятные и специфичные имена.

Результат выполнения

В результате в Roomly создан и зарегистрирован контекстный процессор `room_context`. Он передаёт название приложения и актуальное количество помещений в шаблоны, благодаря чему общие элементы интерфейса используют единый источник данных. Реализация сокращает дублирование кода в представлениях и позволяет централизованно управлять данными общей части сайта.

6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

После завершения практической работы необходимо проверить административную панель Roomly, корректность работы Django ORM, пользовательские сценарии, автоматизированные тесты и состояние репозитория. Проверка должна подтвердить не только наличие отдельных конструкций в коде, но и их работу в приложении.

6.1. Проверка административной панели Django

Откройте административную панель Roomly и проверьте работу моделей `Room`, `User` и `Booking`.

Убедитесь, что:

- модели `Room`, `User` и `Booking` зарегистрированы в административной панели;
- списки записей содержат предусмотренные столбцы;
- поиск помещений выполняется по названию;
- поиск пользователей выполняется по имени и адресу электронной почты;
- поиск бронирований позволяет находить записи по связанным помещениям и пользователям, если соответствующие поля включены в `search_fields`;
- для бронирований доступны предусмотренные фильтры, включая фильтрацию по статусу и дате;
- в списках настроены сортировка и количество записей на странице;
- связанные бронирования отображаются в карточках помещений и пользователей, если для этих моделей настроены `InlineModelAdmin`;
- русскоязычные названия моделей и полей отображаются корректно.

Проверьте также настройки, связанные с переходом к объекту на сайте. Если для модели `Room` реализован метод `get_absolute_url()`, убедитесь, что он возвращает адрес подробной страницы помещения. Если в административной панели используется стандартная ссылка «Посмотреть на сайте», проверьте, что она открывает соответствующую страницу.

Если названия приложений в административной панели должны отображаться на русском языке, проверьте настройки `verbose_name` в классах `AppConfig` и убедитесь, что соответствующие конфигурации приложений используются в `INSTALLED_APPS`.

Не регистрируйте модели повторно, если они уже зарегистрированы в административной панели.

6.2. Проверка работы Django ORM

6.2.1. Проверка функциональности

Проверьте, что основные операции чтения и обработки данных в приложении выполняются средствами Django ORM.

Убедитесь, что:

- список помещений формируется на основе данных модели `Room`;
- подробная страница помещения открывается по его slug, а поиск объекта выполняется с корректной обработкой отсутствующей записи;
- список бронирований формируется с учётом выбранного статуса;

- при отображении активных бронирований отменённые и завершённые записи не включаются в выборку;
- фильтрация бронирований по дате, помещению и пользователю изменяет результат выборки;
- сортировка и пагинация работают с `QuerySet` и не требуют предварительной загрузки всех записей в память приложения;
- для определения свободных помещений учитываются дата бронирования и статус `ACTIVE`;
- при определении доступности помещения отменённые и завершённые бронирования не блокируют его;
- для получения связанных данных применяются подходящие механизмы ORM, например `select_related()` при загрузке бронирования вместе с помещением и пользователем;
- статистические показатели рассчитываются по актуальным данным базы данных с учётом статусов бронирований.

После перехода на `TextChoices` состояние бронирования должно определяться полем `status`. Если старое поле `is_cancelled` уже удалено, оно не должно использоваться в представлениях, шаблонах, административной панели и тестах.

Если в проекте остались прямые SQL-запросы, проверьте их назначение. Не возвращайте прямые SQL-запросы в представления, уже переведённые на ORM в рамках предыдущего этапа.

6.2.2. Проверка оптимизации запросов

Откройте Django Debug Toolbar и проверьте SQL-запросы при выполнении следующих действий:

- открытие списка бронирований;
- открытие подробной страницы бронирования;
- переход между страницами списка;
- применение фильтров.

Для подробной страницы бронирования проверьте, что использование `select_related("room", "user")` позволяет получать данные бронирования, помещения и пользователя без дополнительных запросов при обращении к этим связанным объектам.

Для списка бронирований учитывайте, что использование `annotate()` с `F()` и последующего `values()` может позволять получить данные связанных объектов одним SQL-запросом. В таком случае дополнительный `select_related()` для этого списка не требуется.

Не считайте ошибкой сами по себе запрос подсчёта записей для пагинации или отдельные запросы для заполнения списков выбора помещений и пользователей. Оценивайте назначение запросов и наличие лишних повторных обращений к базе данных.

6.3. Проверка пользовательских сценариев

Выполните функциональную проверку приложения по приведённой таблице.

Сценарий	Ожидаемый результат
Открытие списка помещений	Отображаются помещения из базы данных
Открытие подробной страницы помещения	Отображаются данные выбранного помещения
Открытие помещения по slug	Адрес формируется корректно, страница открывается
Обращение к несуществующему slug	Возвращается страница 404
Поиск свободных помещений на дату, когда имеются только отменённые или завершённые бронирования	Такие бронирования не блокируют помещения
Поиск свободных помещений на дату с активным бронированием	Помещения с активными бронированиями исключаются из списка свободных
Открытие списка бронирований	Отображаются записи, предусмотренные логикой списка, например только активные бронирования
Фильтрация бронирований по дате	Отображаются записи, соответствующие выбранной дате
Фильтрация по помещению	Отображаются бронирования выбранного помещения
Фильтрация по пользователю	Отображаются бронирования выбранного пользователя
Сортировка списка бронирований	Записи располагаются в соответствии с выбранным критерием
Переход между страницами списка	Отображается нужная страница, параметры фильтрации сохраняются
Открытие подробной страницы бронирования	Отображаются данные бронирования, помещения и пользователя
Просмотр статистики	Показатели соответствуют данным базы и учитывают статусы бронирований
Изменение статуса бронирования в административной панели	После сохранения изменения отражаются в списке, проверке доступности и статистике в соответствии с логикой приложения

При проверке доступности помещения используйте одну и ту же дату для сравнения сценариев с активным и отменённым бронированием. Это позволит убедиться, что помещение блокируется именно активной записью, а не самим фактом существования бронирования.

Если в приложении реализовано отображение только активных бронирований, не следует ожидать появления отменённых или завершённых записей в этом списке. При этом соответствующие записи могут учитываться в статистике по своим статусам.

6.4. Проверка качества кода и автоматизированных тестов

Перед сдачей работы выполните в терминале из корневого каталога проекта:

```
python manage.py check
```

Затем запустите автоматизированные тесты:

```
pytest -v
```

Если `pytest` не настроен для запуска тестов проекта, используйте встроенный механизм Django:

```
python manage.py test
```

Все проверки должны завершаться без ошибок, связанных с выполненными изменениями.

Дополнительно проверьте:

- отсутствие повторной регистрации моделей в Django Admin;
- отсутствие неиспользуемых импортов и случайно оставленного отладочного кода;
- соответствие настроек административной панели фактическим полям моделей;
- согласованность статусов бронирования в моделях, представлениях, шаблонах и тестах;
- корректность работы `get_absolute_url()` и маршрута подробной страницы помещения;
- сохранение существующих маршрутов и шаблонов;
- корректность работы проекта после перезапуска сервера;
- соответствие README.md текущей структуре и функциональности приложения.

Если часть проверок не пройдена, исправьте обнаруженные ошибки и повторите соответствующие проверки. Если устранить проблему не удалось, укажите её в отчёте и не представляйте незавершённую функциональность как реализованную.

6.5. Контрольная точка Git

После проверки приложения зафиксируйте изменения в системе контроля версий. Сначала выполните:

```
git status
```

Проверьте изменённые и новые файлы. Убедитесь, что в репозитории не появились случайно:

- виртуальное окружение venv;
- локальные файлы IDE;
- временные файлы и кэш;
- секреты и локальные настройки;

- база данных или другие локальные данные, которые не должны храниться в репозитории согласно правилам проекта.

Проверьте содержимое .gitignore. Не удаляйте базу данных и тестовые данные, не убедившись, что они не нужны для работы проекта.

После проверки добавьте необходимые изменения в индекс Git. Можно использовать команду:

```
git add .
```

Однако перед её выполнением необходимо убедиться, что все ненужные локальные файлы исключены из репозитория. Если в рабочем каталоге есть посторонние изменения, добавляйте только необходимые файлы.

Создайте коммит:

```
git commit -m "Завершена настройка Django ORM и административной панели"
```

Проверьте результат:

```
git status
```

Если все необходимые изменения включены в коммит, рабочее дерево должно быть чистым. Если остались незакоммиченные изменения, проверьте их назначение и при необходимости создайте дополнительный коммит.

6.6. Сводная таблица проверки

Ниже приведена сводная таблица работ по практической работе. Главным столбцом является «Номер пункта из методички» – в нём перечислены все пункты методички в порядке их следования. Для каждого пункта в остальных столбцах отмечена суть работы, которую выполняет студент: действия в терминале, работа с моделями в оболочке Python (Django Shell), изменения в коде проекта, работа с отладочной панелью Django Debug Toolbar и миграции базы данных.

№	Пункт МУ	Терминале	Django Shell	Изменения в коде	Debug Toolbar	Миграции БД
1.	**5.2**	Создание суперпользователя; запуск сервера разработки	—	Регистрация моделей Room, User, Booking в admin.py; настройка ModelAdmin (отображение столбцов, поиск, фильтрация, сортировка, inline-формы); локализация названий приложений, моделей и полей	—	—

№	Пункт МУ	Терминале	Django Shell	Изменения в коде	Debug Toolbar	Миграции БД
2.	**5.3**	—	Выполнение запросов к моделям: фильтрация, исключение, сортировка, аннотирование, получение связанных данных, агрегация	Реализация представлений списка и деталей бронирований; пагинация; статистика по статусам	Анализ SQL-запросов при открытии страниц списка и деталей	—
3.	**5.4.1**	Установка пакета django-debug-toolbar	—	Подключение DDT в settings.py (INSTALLED_APPS, MIDDLEWARE, INTERNAL_IPS, DEBUG) и urls.py (условное подключение маршрутов панели)	Проверка появления панели на страницах приложения; первичный анализ SQL-запросов	—
4.	**5.4.2**	Запуск сервера разработки; финальные проверки	—	Применение select_related() и annotate() с F() для оптимизации запросов; сохранение функциональности списка и деталей бронирований	Сравнение SQL-запросов до и после оптимизации; выявление проблемы N+1 запросов	—
5.	**5.4.3**	Создание и применение миграций схемы и данных	Тестирование методов менеджера QuerySet: active(), cancelled(), completed(), for_date()	Добавление перечисления Status(TextChoices) и поля status в модель Booking; реализация BookingQuerySet и BookingManager; перенос значений из is_cancelled в status; удаление старого поля	—	Схемная миграция (поле status); миграция данных (перенос значений is_cancelled); схемная миграция (удаление is_cancelled)
6.	**5.4.4**	Создание и применение миграций	Проверка значения вычисляемого поля после изменения исходных данных	Добавление GeneratedField в модель Room для вычисляемого признака	—	Схемная миграция (вычисляемое поле GeneratedField)
7.	**5.4.5**	Создание и применение миграций	Проверка формирования URL методом get_absolute_url()	Добавление SlugField в модель Room; реализация get_absolute_url(); настройка маршрута с конвертером <slug:slug>	—	Схемная миграция (поле slug модели Room)
8.	**5.4.6**	—	—	Создание контекстного процессора room_context; регистрация его в TEMPLATES в settings.py	—	—

№	Пункт МУ	Терминале	Django Shell	Изменения в коде	Debug Toolbar	Миграции БД
9.	**6.1**	—	—	Проверка административной панели: отображение моделей, столбцов, поиска, фильтрации, inline-форм, локализации	—	—
10.	**6.2**	—	Проверка запросов через менеджер и QuerySet; тестирование фильтрации, сортировки, агрегации	Проверка корректности ORM-запросов и бизнес-логики (статусы, доступность помещений, статистика)	Проверка SQL-запросов на страницах списка и деталей бронирований	—
11.	**6.3**	—	—	Функциональная проверка пользовательских сценариев: открытие страниц, фильтрация, пагинация, статистика	—	—
12.	**6.4**	Запуск проверок проекта и автоматизированных тестов	—	Проверка отсутствия отладочного кода; согласованности статусов в коде, шаблонах и тестах; корректности маршрутов	—	Проверка состояния миграций (makemigrations --check)

7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

7.1. Контроль выполнения материалов Яндекс Практикума

Выполнение учебных материалов контролируется по завершённым заданиям курса и результатам применения изученных приёмов в проекте Roomly.

Необходимо подтвердить освоение следующих тем в рамках урока раздела «Базы данных и Django ORM»:

Урок 4 «Админ-зона Django»:

- Создание суперпользователя. Регистрация моделей в админке
- Локализация и перевод
- Тонкая настройка админ-зоны

Урок 5 «Django ORM. Получение информации из БД»:

- Django Debug Toolbar
- SELECT и FROM через ORM
- Фильтрация WHERE: методы filter() и exclude()
- Условия AND, OR, NOT. Q-объекты
- Сортировка ORDER BY, ограничение LIMIT и сдвиг OFFSET
- Получение отдельных объектов модели

- Запросы к связанным моделям
- Доработка проекта

6. Финальный проект спринта: публикации для Блогикум

- Подготовка к работе над проектом
- Проект. Приложение «Блогикум». Часть 2. Публикации
- Обратная связь

Если в учебном курсе используются дополнительные демонстрационные проекты, их структура и модели не должны механически переноситься в Roomly. Необходимо применять изученные приёмы к уже существующей предметной области системы бронирования помещений.

7.2. Чек-лист выполнения практической работы

Перед сдачей работы проверьте каждый пункт.

№	Пункт проверки	Выполнено
1	Модели Room, User и Booking зарегистрированы в Django Admin, настроены отображение связанных объектов, поиск, фильтрация и сортировка.	☐
2	Миграции созданы и применены после изменения моделей.	☐
3	Данные существующих бронирований корректно преобразованы при переходе на TextChoices.	☐
4	Менеджер и QuerySet бронирований содержат необходимые методы и используются в представлениях.	☐
5	Основные операции чтения и обработки данных выполняются средствами Django ORM.	☐
6	Фильтрация, сортировка и пагинация списка бронирований работают совместно.	☐
7	Свободные помещения определяются с учётом даты и статуса бронирования.	☐
8	Статистика корректно учитывает статусы ACTIVE, CANCELLED и COMPLETED.	☐
9	Подробная страница помещения открывается по slug, метод get_absolute_url() формирует правильный адрес.	☐
10	Контекстный процессор передаёт общие данные в шаблоны, если он предусмотрен текущей реализацией.	☐
11	Поля, вычисляемые через GeneratedField, корректно формируются базой данных, если это поле реализовано в проекте.	☐
12	Проверены запросы ORM и SQL с помощью Django Debug Toolbar.	☐
13	Проверены операции с моделями через Django Shell.	☐
14	Команда python manage.py check завершается без ошибок.	☐
15	Автоматизированные тесты выполнены успешно либо установлены причины их неуспешного выполнения.	☐
16	README.md соответствует текущему состоянию проекта.	☐
17	Изменения зафиксированы в Git.	☐

7.3. Таблица оценивания практической работы

За выполнение практической работы по развитию проекта Roomly студент получает 1,5 балла. Для оценивания используется таблица из 30 точечных показателей.

Оценивание проводится в форме демонстрации. Студент показывает соответствующий элемент проекта, выполняет указанное действие и отвечает на вопрос преподавателя. Опрос проводится последовательно по пунктам таблицы, выборочно. Демонстрация длится не более 5 минут и завершается на первом неправильном ответе либо его отсутствии.

Примечание. Показатели сформулированы на материале Roomly. Если студент выполняет работу в рамках собственного индивидуального проекта, демонстрация проводится на его предметной области с учётом реализованных моделей, связей и функциональности.

№	Показатель оценивания	Порядок демонстрации и ожидаемый результат
1	Продемонстрируйте настройку административной панели для работы с моделями предметной области	В Django Admin: откройте списки Room, User и Booking. Покажите настроенные столбцы, поиск, фильтры и сортировку. Для бронирований найдите запись по связанному помещению или пользователю и примените фильтр по статусу. Объясните, какие параметры ModelAdmin обеспечивают показанные возможности и как они связаны с полями моделей.
2	Продемонстрируйте работу со связанными объектами в административной панели	В Django Admin: откройте карточку помещения или пользователя и покажите связанные бронирования, если настроен inline. Измените доступное поле связанной записи либо создайте связанную запись через inline, сохраните изменения и проверьте результат. Объясните, как внешний ключ связывает объекты и почему удаление или изменение связанного объекта зависит от настройки on_delete. Если inline не реализован, продемонстрируйте другой предусмотренный проектом способ просмотра связи.
3	Продемонстрируйте изменение модели и применение миграций	В редакторе кода и терминале: покажите поле модели, измените один из его параметров или добавьте поле, например blank, null либо default, если это допустимо для данной модели. Выполните python manage.py makemigrations, откройте созданный файл миграции и объясните отражённое изменение. Затем выполните python manage.py migrate и покажите результат применения миграции. Объясните различие между созданием миграции и её применением к базе данных.
4	Продемонстрируйте преобразование существующих данных при изменении модели бронирования	В редакторе кода и терминале: откройте миграцию данных, связанную с переходом от is_cancelled к status. Покажите, как старые значения преобразуются в ACTIVE или CANCELLED, и объясните, как обрабатываются существующие записи. При наличии завершённых бронирований покажите, как они сохраняют соответствующий статус. Объясните, почему преобразование данных необходимо выполнять до удаления старого поля.
5	Продемонстрируйте работу с моделями через Django Shell	В терминале: выполните python manage.py shell, импортируйте Room, User и Booking, получите несколько объектов через objects.all() или фильтрацию. Покажите чтение и изменение поля объекта, сохраните изменение методом save() и повторно получите запись из базы данных. Объясните, почему изменения объекта в памяти и изменения, сохранённые в базе данных, не всегда происходят одновременно.

№	Показатель оценивания	Порядок демонстрации и ожидаемый результат
6	Продемонстрируйте создание, изменение и удаление записей средствами ORM	В Django Shell: создайте объект модели, сохраните его, измените одно из полей и выполните save(). Затем покажите удаление созданной тестовой записи методом delete(). Объясните, как ORM преобразует эти операции в действия с базой данных и как настройки связей влияют на удаление связанных объектов. Не удаляйте реальные данные проекта: используйте специально созданную тестовую запись.
7	Продемонстрируйте получение записей через QuerySet и фильтрацию по нескольким условиям	В редакторе кода или Django Shell: покажите запрос к модели Booking, например с использованием filter(). Добавьте условие по дате, помещению или пользователю и объясните, какие записи попадут в результат. Измените одно из условий и покажите изменение выборки.
8	Продемонстрируйте фильтрацию по полям связанных моделей	В редакторе кода или Django Shell: покажите ORM-запрос с обращением к связанному полю через двойное подчёркивание, например room__name или user__name. Измените условие поиска и получите новый результат. Объясните, как ORM формирует условие для связанной таблицы и чем такой запрос отличается от фильтрации только по внешнему ключу.
9	Продемонстрируйте работу собственного менеджера и методов QuerySet для бронирований	В редакторе кода: откройте bookings/models.py и покажите BookingQuerySet, его методы active(), cancelled(), completed() и for_date(), а также подключение менеджера к модели. В Django Shell: выполните, например, Booking.objects.active() и Booking.objects.for_date(выбранная_дата). Покажите результаты и объясните, почему повторно используемые условия вынесены в методы менеджера, а не дублируются в разных представлениях.
10	Продемонстрируйте обработку пользовательского ввода при формировании ORM-запроса	В редакторе кода: покажите представление списка бронирований, в котором значения даты, помещения и пользователя извлекаются из request.GET. Объясните проверку и преобразование значений перед использованием в запросе. В браузере: задайте корректные и некорректные параметры, например неправильную дату, и покажите предусмотренный результат обработки ошибки без необработанного исключения.
11	Продемонстрируйте динамическую сортировку результатов ORM-запроса	В редакторе кода: покажите словарь разрешённых полей сортировки и вызов order_by(). В браузере: измените параметр сортировки списка бронирований и сравните порядок записей. Объясните, почему нельзя без проверки передавать произвольное значение пользовательского параметра непосредственно в конструкцию сортировки.
12	Продемонстрируйте выбор необходимых полей через values()	В редакторе кода: покажите запрос списка бронирований, в котором через values() выбираются только необходимые поля, включая данные связанных помещения и пользователя. В Django Shell: выполните аналогичный запрос и покажите структуру возвращаемых словарей. Добавьте или удалите одно поле и объясните, как изменится результат и почему шаблон должен обращаться к тем ключам, которые реально возвращает запрос.
13	Продемонстрируйте вычисление дополнительных полей через annotate() и F()	В редакторе кода: откройте запрос, в котором через annotate() и F() добавляются поля, например room_name=F("room__name") или user_name=F("user__name"). Покажите, где эти поля используются далее, например в values() и шаблоне списка бронирований. Измените выражение или добавьте ещё одно поле, проверьте результат и объясните, как ORM получает значение из связанной таблицы.

№	Показатель оценивания	Порядок демонстрации и ожидаемый результат
14	Продемонстрируйте получение объекта с обработкой его отсутствия	В редакторе кода: покажите использование get_object_or_404() в представлении подробной страницы помещения или бронирования. В браузере: откройте страницу существующего объекта, затем обратитесь к несуществующему идентификатору или slug. Покажите корректный ответ 404 и объясните, почему несуществующий объект не должен приводить к необработанному исключению.
15	Продемонстрируйте формирование URL помещения через slug и get_absolute_url()	В редакторе кода: покажите поле slug, реализацию get_absolute_url() и маршрут с конвертером <slug:slug>. Объясните использование reverse() и именованного маршрута. В Django Shell или браузере: получите URL через room.get_absolute_url() либо откройте сформированную ссылку и покажите страницу соответствующего помещения. В Django Admin: продемонстрируйте переход к публичной странице помещения через ссылку просмотра объекта, если она предусмотрена настройками административной панели. Объясните, почему использование именованного маршрута предпочтительнее жёстко заданного адреса.
16	Продемонстрируйте использование GeneratedField, если поле реализовано в проекте	В редакторе кода: покажите объявление вычисляемого поля и выражение, на основании которого оно формируется, например определение признака большого помещения по вместимости. Объясните назначение GeneratedField и параметр db_persist. В Django Shell или Django Admin: измените исходное поле объекта и проверьте значение вычисляемого поля после сохранения и повторного получения объекта. Если GeneratedField не входит в фактическую реализацию проекта, этот показатель заменяется демонстрацией другого реализованного изменения модели и его миграции.
17	Продемонстрируйте формирование агрегированной статистики средствами ORM	В редакторе кода: покажите представление статистики, в котором используется aggregate() с Count() и условиями Q() для разных статусов. В браузере: откройте страницу статистики и сопоставьте выводимые значения с данными базы. Объясните разницу между общим количеством записей и количеством бронирований, удовлетворяющих условию по статусу.
18	Продемонстрируйте формирование групповой статистики и работу условий отбора групп	В редакторе кода или Django Shell: покажите реализованный запрос с values(), annotate() и, если предусмотрено, filter() или order_by(). Если проект использует группировку с Count(), объясните, что именно считается для каждой группы. Покажите результат и объясните, как изменится выборка при изменении условия или порядка сортировки. Если отдельная групповая статистика не реализована, продемонстрируйте другой используемый в проекте агрегирующий ORM-запрос.
19	Продемонстрируйте оптимизацию получения связанных объектов через select_related()	В редакторе кода: покажите запрос подробной страницы бронирования с select_related("room", "user"). Объясните, какие связи загружаются совместно с бронированием. В Django Debug Toolbar: откройте подробную страницу и вкладку SQL, найдите запрос получения бронирования и покажите соединения с таблицами помещения и пользователя. Объясните, почему обращение к связанным объектам в этом случае не требует отдельных запросов.
20	Продемонстрируйте анализ запросов списка бронирований с использованием Django Debug Toolbar	В браузере: откройте список бронирований, примените фильтр и перейдите на другую страницу. Откройте Django Debug Toolbar, вкладку SQL и покажите запросы, связанные со списком, фильтрацией и пагинацией. Объясните назначение запроса подсчёта записей и запроса получения текущей страницы. Укажите, какие дополнительные запросы ожидаемы, а какие могли бы свидетельствовать о повторной загрузке связанных объектов.

№	Показатель оценивания	Порядок демонстрации и ожидаемый результат
21	Продемонстрируйте совместную работу фильтрации, сортировки и пагинации	В браузере: откройте список бронирований, задайте фильтр по дате или помещению, выберите сортировку и перейдите на следующую страницу. Покажите, что условия фильтрации и сортировки сохраняются. В редакторе кода: покажите использование Paginator и механизм сохранения GET-параметров в ссылках навигации. Объясните, как изменение размера страницы влияет на число страниц и объём отображаемых данных.
22	Продемонстрируйте определение доступности помещения на выбранную дату	В редакторе кода: покажите ORM-запрос, который получает идентификаторы помещений с активными бронированиями на выбранную дату, и логику исключения этих помещений из списка свободных. В Django Shell или Django Admin: найдите активное бронирование и покажите, что соответствующее помещение исключается из списка свободных. Измените статус бронирования на отменённый, повторите проверку и объясните результат.
23	Продемонстрируйте работу контекстного процессора	В редакторе кода: откройте функцию контекстного процессора и покажите возвращаемый словарь, например с названием сайта и количеством помещений. Затем откройте settings.py и покажите регистрацию процессора в TEMPLATES. В браузере: откройте несколько страниц, использующих общий шаблон, и покажите переданные значения. Объясните отличие контекстного процессора от передачи данных через context в отдельном представлении.
24	Продемонстрируйте применение статусов TextChoices в модели и пользовательской части	В редакторе кода: покажите объявление статусов в модели Booking и методы QuerySet, работающие с ними. В Django Admin: измените статус тестового бронирования, затем откройте список бронирований и статистику. Покажите изменение поведения в соответствии с новым статусом и объясните, почему сравнение выполняется с константами Booking.Status, а не с произвольными строками по всему проекту.
25	Продемонстрируйте получение связанных данных без необоснованных дополнительных запросов	В редакторе кода: покажите запрос списка бронирований, использующий annotate() с F() и values() для получения названий помещения и пользователя. В Django Debug Toolbar: откройте список и покажите SQL-запрос. Объясните, почему для этого запроса select_related() не требуется, если данные уже выбираются одним запросом, и чем такой способ отличается от получения объектов моделей с последующим обращением к связанным атрибутам.
26	Продемонстрируйте корректность вычисляемого и связанного значения после изменения исходных данных	В Django Admin или Django Shell: измените вместимость помещения или данные связанного объекта, если это допустимо в предметной области. Сохраните изменение и повторно получите объект из базы данных. Покажите обновившееся значение, связанное поле или результат вычисления. Объясните, какие значения хранятся непосредственно, а какие формируются на основе других данных.
27	Продемонстрируйте работу именованных маршрутов и ссылок в шаблонах	В редакторе кода: покажите маршрут подробной страницы помещения и формирование ссылки через {% url %} либо get_absolute_url(). В браузере: перейдите из списка к карточке помещения и вернитесь к списку. Объясните, как изменение имени маршрута или передаваемого аргумента влияет на формирование ссылки и почему маршруты должны использоваться согласованно с представлениями.

№	Показатель оценивания	Порядок демонстрации и ожидаемый результат
28	Продемонстрируйте проверку корректности ORM-запроса и обработку ошибки базы данных	В редакторе кода: покажите участок, где выполняется запрос и предусмотрена обработка ошибки, если она требуется для данного сценария. Объясните, как приложение реагирует на исключение. В браузере или тесте: продемонстрируйте доступный безопасный сценарий проверки обработки ошибки либо покажите соответствующий автоматизированный тест. Не требуется намеренно повреждать рабочую базу данных; необходимо объяснить, какой результат увидит пользователь при предусмотренной ошибке.
29	Продемонстрируйте проверку моделей и миграций после изменения структуры данных	В терминале: выполните python manage.py makemigrations --check и python manage.py check. Объясните, что первая команда проверяет наличие изменений моделей, для которых ещё не созданы миграции, а вторая проверяет конфигурацию проекта. Если в проекте обнаружены несозданные миграции, покажите соответствующую модель и объясните, как подготовить и применить миграцию.
30	Продемонстрируйте проверку пользовательского сценария автоматизированным тестом	В редакторе кода: откройте тест, проверяющий один из основных сценариев, например фильтрацию активных бронирований или обработку отсутствующего объекта. Покажите проверяемое условие и ожидаемый результат. В терминале: выполните pytest -v либо python manage.py test, найдите результат соответствующего теста и объясните, какое поведение приложения он подтверждает.

7.4. Состав отчёта

Отчёт по практической работе должен содержать:

1. Тему и цель работы.
2. Краткое описание выполненных заданий Яндекс Практикума, относящихся к текущему этапу.
3. Описание настроек административной панели Roomly.
4. Краткое описание изменений моделей, миграций и менеджера `QuerySet`.
5. Описание проверенных ORM-запросов, фильтрации, сортировки, пагинации и статистики.
6. Результаты проверки доступности помещений с учётом статусов бронирований.
7. Результаты анализа SQL-запросов с помощью Django Debug Toolbar.
8. Результаты выполнения `python manage.py check` и автоматизированных тестов.
9. Подтверждение выполнения контрольной точки Git.
10. Выводы по результатам работы.

В качестве подтверждения результата приложите скриншоты, демонстрирующие фактическую работу приложения:

- список моделей и настроенный список записей в Django Admin;
- поиск или фильтрацию бронирований;
- карточку помещения или пользователя со связанными бронированиями, если реализован `inline`;

- список бронирований с фильтрацией и пагинацией;
- статистику по статусам бронирований;
- фрагмент модели или менеджера `QuerySet`, если он изменялся в рамках работы;
- SQL-запросы в Django Debug Toolbar;
- результаты выполнения проверок и автоматизированных тестов в терминале.

Скриншоты должны подтверждать реальную работу проекта. Не следует включать изображения, созданные вместо запуска приложения, или выдавать ожидаемый результат за фактически полученный.

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего предназначена административная панель Django?
2. Как зарегистрировать модель в административной панели?
3. Для чего используется класс `admin.ModelAdmin`?
4. Как с помощью `list_display` настроить столбцы списка записей?
5. Чем search_fields отличается от list_filter?
6. Для чего используются `ordering` и `list_per_page`?
7. Как настроить отображение связанных объектов с помощью `inline`?
8. Что такое `QuerySet` и в какой момент ORM-запрос обычно обращается к базе данных?
9. Чем отличаются методы `filter()` и `exclude()`?
10. Для чего используется объект `Q`?
11. Как с помощью ORM отсортировать записи и ограничить выборку?
12. Чем отличаются `get()` и `filter()` при получении объектов?
13. Для чего используется `get_object_or_404()` в представлении Django?
14. Чем отличаются select_related() и prefetch_related()?
15. Как с помощью ORM получить поля связанной модели?
16. Почему при работе со связанными объектами может возникать проблема `N`+1 запросов?
17. Как проверить корректность фильтрации и сортировки в приложении?
18. Почему отменённые бронирования не должны учитываться при определении занятости помещения и подсчёте активных бронирований?
19. Почему настройка фильтров в Django Admin не заменяет проверку бизнес-правил?
20. Какие проверки необходимо выполнить перед фиксацией изменений в Git?

9. ИТОГ

В ходе выполнения ПР10 необходимо закрепить навыки работы с административной панелью Django и Django ORM на примере индивидуального проекта Roomly.

Результатом работы должна стать настроенная административная панель для моделей `Room`, `User` и `Booking`, проверенная работа основных ORM-запросов и сохранение функциональности приложения после внесённых изменений.

Завершение практической работы подтверждается выполнением учебных заданий, проверкой пользовательских сценариев, запуском доступных автоматизированных тестов, обновлением документации и фиксацией изменений в Git.

Направление дальнейшего развития индивидуального проекта показано на рис. 6.

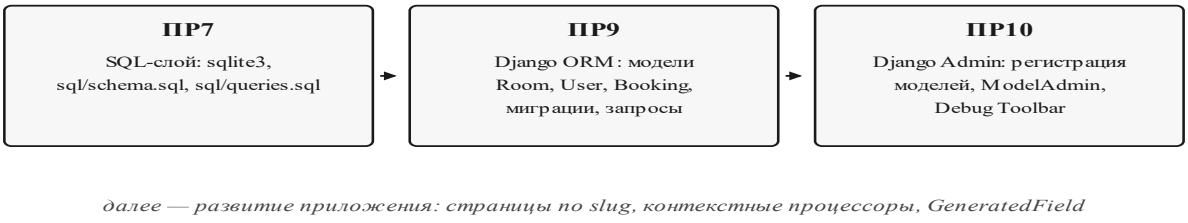


Рисунок 6 – Развитие индивидуального проекта